RESEARCH ARTICLE

A Verification-Centric Reference Pipeline for AI-Native Software Development

Brendan Edmonds[1]* Mark Utting[1]

[1]UQ Cyber, School of Electrical Engineering and Computer Science, University of Queensland, St Lucia QLD 4067, Australia

*Correspondence: Brendan Edmonds, b.edmonds@uq.edu.au

Abstract

As large language models make implementation code cheap to generate and regenerate, the economics of software development invert: the implementation becomes a disposable build output, and the durable, scarce artefact becomes the *specification* from which it is produced. The emerging generation of AI-native, specification-driven development lifecycles—exemplified by AWS AI-DLC, GitHub Spec Kit, and Kiro—has recognised this shift but stops short of exploiting it, because their specifications are unverifiable natural language, their generate-then-review loop has no machine gate, and their verification, where present, is circular: the same model that writes the code also writes the tests that judge it. This article proposes a verification-centric reference pipeline that supplies the missing piece: a machine-checkable formal-specification contract layer and a closed, counterexample-guided verification loop. The architecture is organised around the formal contract as the *pivot* between ambiguous human intent and precise machine verification, exploiting the dual role of a specification as both the precise *input* an agent builds against and the *oracle* its output is verified against. We develop four mechanisms in detail: an intent-elaboration layer that closes the *ratification gap* by having developers ratify behaviour rather than formal logic; a brownfield characterisation phase that triangulates code, issue trackers, and documentation to recover contracts and surface candidate defects; a synthesis–verification loop in which an independently authored contract gates agent-generated code over all inputs; and compositional change propagation that turns a callee specification change into a sound, run-free behavioural blast-radius and version-compatibility analysis. We show the architecture is buildable by mapping its trusted components onto externally validated prior artefacts—a heuristic JML inference engine, an OpenJML/Z3 verifier—and instantiating it as a proof of concept over a real three-stream corpus drawn from Apache Commons Lang. We specify a falsifiable evaluation built on tool-derived metrics; full empirical validation on external production codebases and an industrial case study are identified as future work.

Keywords: specification inference; formal verification; large language models; AI agents; design by contract; JML; software development lifecycle; brownfield software; API evolution; behavioural compatibility

1 Introduction

For most of the history of software engineering, source code has been the scarce, durable, and expensive artefact of the discipline. The cost of producing a working implementation dominated every other concern, and the surrounding apparatus—requirements, designs, tests, and documentation—existed largely to amortize and protect that investment. The recent maturation of large language models (LLMs) for code

generation has begun to invert this economic order. When a sufficiently capable agent can regenerate an implementation from a description in seconds and at negligible marginal cost, the implementation ceases to be a treasured asset and becomes closer to a disposable build output, akin to an object file recompiled on demand. What then becomes scarce and durable is not the code but the *specification of intent*: the precise statement of what the code must do, against which any number of cheap regenerations can be measured. This article takes the economic inversion seriously and asks what a software lifecycle reorganised around a machine-checkable specification as its source of truth would have to look like.

1.1 The rise—and the defects—of spec-driven, agentic lifecycles

The industry has not failed to notice the inversion. A cluster of emerging methodologies and tools now place a "specification" at the centre of an AI-assisted lifecycle in which agents generate, test, and revise code: GitHub's Spec Kit [44], Amazon's AI-Driven Development Life Cycle (AI-DLC) [4], and the Kiro spec-driven integrated development environment [5] are representative, and position papers have begun to argue that LLMs are reshaping the software development life cycle wholesale [78]. These approaches are genuine advances over unstructured prompting because they recognise that intent, not the keystroke-level act of typing code, is the durable artefact. Yet they share three structural defects that this article seeks to remedy.

First, the specification is expressed in *natural language* (NL). An NL specification cannot be mechanically compiled into code, only interpreted, so the central "compilation" step from intent to implementation is fundamentally unverifiable; there is no formal object against which the result can be checked. Second, the loop is *open*. The agent generates an artefact and a human, or at best another model, eyeballs it; there is no machine gate that an output must pass before it is admitted, and so regressions and subtle defects propagate silently. Third, where automated verification is present at all, it is typically *circular* or self-grading: the same model that writes the code also writes its tests or serves as its oracle, so the acceptance signal is drawn from the very distribution that produced the candidate. The reliability literature has repeatedly shown that such signals are fragile—benchmark suites are often too weak to detect plausible-but-wrong code, and tightening them lowers reported success rates sharply [62]—and that LLM code generation is prone to confident hallucination that survives superficial review [108]. The hazard is acute precisely when an LLM is asked to generate the oracle it will be judged against [64].

1.2 The proposed pipeline: a verification spine for AI-native development

The thesis of this article is that the missing element in these lifecycles is a *machine-checkable formal-specification contract layer* together with a *verification loop* that closes the gap between intent and implementation. The contract layer supplies the verification spine that agentic methodologies currently lack. We do not propose a new model or a new prompting trick; we propose a reference architecture that re-couples two intellectual lineages that the LLM era has split apart—classical specification-driven program synthesis, with its closed, counterexample-driven loop [67, 91], and description-driven generation, with its cheap NL intent [18, 53]—and we validate the architecture by instantiating it.

The pivot of the architecture is the formal contract, which mediates between the ambiguous human world above it and the precise machine world below it. Above the pivot lies *elaboration*: the transformation of NL intent into a formal contract, confirmed by a human. Below the pivot lie two operations on a now-precise artefact: *synthesis*, in which an agent produces code from the contract, and *verification*, in which a deductive verifier checks the code against the contract over *all* inputs rather than a finite sample. Two principles distinguish this organisation from the spec-driven tools surveyed above. The first is the *dual role of the specification*: the same formal contract serves simultaneously as the *input* the agent builds against and the *oracle* the agent's output is verified against. Because the specification plays both roles, the cost of authoring

it—historically the bottleneck that crippled formal methods—is the single cost that must be eliminated, and automated inference is what eliminates it. The second is the *independent-authority constraint*: the contract that code is verified against must not be authored by the same agent instance or role that writes the code, on pain of reintroducing the self-grading circularity that the architecture exists to remove. The elaborator role is maximally faithful and strict and never sees the implementation; the implementer role merely satisfies the contract.

Establishing a human-owned oracle is harder than it appears, and naming the difficulty precisely is one of this article's contributions. We term it the *ratification gap*: the specification is the oracle and must therefore be correct and human-owned, yet the human reached for NL in the first place precisely because they cannot or will not write formal contracts, so asking them to confirm formal text they cannot read makes their authority a fiction. The architecture resolves this by a discipline of *ratifying behaviour, not logic*—rendering a candidate contract as concrete, checkable examples ("given $x = -1$, this contract says the result is 0; is that correct?") whose ratified instances become *pinned oracles* the contract must satisfy—and by *Socratic, decision-surfacing elaboration*, in which the agent surfaces only the boundary decisions it was forced to resolve (null and empty inputs, overflow, ordering, duplicates, concurrency, error-versus-exception) rather than the full formal text. To guard against an elaborator that quietly weakens the contract to make it trivially satisfiable—"sort" silently relaxed to "is a permutation," which the identity function satisfies—a second, independent *adversarial elaboration* agent searches for scenarios consistent with the NL intent but violating the proposed contract, the elaboration-layer dual of counterexample-guided synthesis, supplemented by vacuity and non-triviality checks that repurpose our own prior "ensures true" punt detection as a weakening signal. Throughout, every obligation in a contract is tagged with its *provenance* (human-ratified, inherited-default, inferred-unconfirmed, example-pinned, ticket-stated, doc-stated, or code-inferred) and its *assurance* level (proved, bounded-checked, tested, or unchecked), which is what makes the independent-authority constraint auditable and the word "verified" honest. Where intent is not formally specifiable or verification does not terminate, the architecture degrades gracefully from full proof to bounded check to test, recording the level rather than silently dropping the obligation.

1.3 Brownfield reality and compositional propagation

A reference architecture that assumes a clean slate would be of little use, because most software already exists. For pre-existing (brownfield) codebases the contract is *recovered*, not authored, and the existing code occupies a peculiar dual position: it is simultaneously the best available evidence of intent and the prime suspect, because it encodes whatever bugs it contains. We therefore introduce a *characterisation phase* that runs before any user input, triangulating three streams of evidence—the code (what the system does), the issue tickets (why, including known bugs and edge cases), and the documentation (the declared intent)—and reconciling them into a candidate contract with per-clause provenance, flagging the disagreements between streams. It must be stated precisely what such verification proves: checking code against a *code-derived* specification is characterisation, a faithful-description baseline and a toolchain sanity check; it is nearly circular and does *not* prove the absence of bugs. Bugs surface instead as *disagreements*—the code violating its own inferred pattern, or the code contradicting ticket- or doc-derived intent—so the characterisation phase cannot be gated on "zero errors"; finding code-versus-intent violations is the success outcome, and its output is a characterisation report listing the recovered specification, the candidate bugs, and what could not be verified, which becomes the opening agenda of the human dialogue. This motivates a *reordering* of the lifecycle loop: a first, retroactive pass establishes a trusted baseline on existing code before any change is made, and only subsequent, prospective passes implement new features. Because whole legacy applications cannot be formalised at once, the architecture adopts a *frontier* strategy, specifying only the module under change, the public API surface, and the modules referenced by active tickets, and treating the remainder as unspecified under weak inherited defaults.

It is at these module boundaries that the thesis programme's central mechanism plugs in. Because formal contracts compose, a change to a callee's specification *recomputes* the proof obligations of its callers and flags which callers break, *without running them*. Internal refactoring and dependency or library version upgrades become the same operation, and the perennially expensive question "which of my call sites does this upgrade break behaviourally?" becomes a first-class, machine-answered lifecycle event—a blast-radius or behavioural-compatibility analysis. This lifts the compositional refinement established in our prior work [33] from a property of the inference engine into a lifecycle capability, and it directly serves the broader programme goal of supporting formal-methods adoption and library version compatibility.

1.4 Relation to prior work in this programme

This is the fifth article in a research programme on automated specification inference and its uses. The instrument throughout is the JML-Inferrer, a Java 21 tool built on JavaParser that infers Java Modeling Language (JML) [58] pre- and postconditions, loop and class invariants, and assignable clauses by abstract-syntax-tree heuristics, with verification by OpenJML extended static checking [23] discharged through the Z3 SMT solver [27], augmented by a custom OpenJML fork that adds recursive function definitions for the `\sum`, `\product`, and `\num_of` quantifiers. The four prior articles supply the trusted, already-validated components on which the present architecture's proof-of-concept is built: Article 1 of this programme showed that automatically inferred JML specifications materially improve LLM-based unit-test generation [31]; Article 2 showed that JML specifications can be embedded into compiled Java artefacts and OpenAPI documents at full roundtrip fidelity [32]; Article 3 showed that heuristic inference admits compositional refinement across method boundaries, a second weakest-precondition pass adding on the order of 18,854 additional well-formed preconditions on the study corpus [33]; and Article 4 integrated the inference pipeline into continuous-integration and continuous-delivery (CI/CD) systems with diff-only analysis, content-hash caching, and fail-open semantics [34]. The present article reuses these as trusted building blocks and contributes the lifecycle architecture that binds them together.

1.5 Research questions and contributions

The proposed architecture (contribution (i) below) frames a single overarching design question—what lifecycle architecture supplies the missing verification spine for agentic, spec-driven development, and what roles, artefacts, and invariants must it enforce so that the formal contract can act simultaneously as input intent and verification oracle without self-grading. That design question is decomposed into six research questions, each operationalised in the evaluation design (section 11).

- **RQ1 (Ratification effort).** Can ratifying behaviour through pinned-oracle examples let a non-expert legitimately own a formal contract, and at what human cost relative to confirming formal text?
- **RQ2 (Decision surfacing and adversarial soundness).** Does Socratic boundary-decision surfacing capture the decisions where defects concentrate, and does adversarial elaboration detect specification weakening?
- **RQ3 (Provenance, assurance, and independent authority).** Do per-obligation provenance and assurance tags, together with the independent-authority constraint, make a "verified" claim auditable?
- **RQ4 (Synthesis–verification loop).** Does the counterexample-guided loop discharge obligations on the method classes the inferrer supports, and how does tiered assurance degrade where it does not?
- **RQ5 (Brownfield characterisation).** For pre-existing codebases, can multi-source triangulation of code, tickets, and documentation recover a provenance-tagged contract and surface genuine code-versus-intent discrepancies as candidate bugs on a real three-stream corpus?

- **RQ6 (Compositional propagation).** Can compositional contracts make behavioural version-compatibility a first-class lifecycle event, identifying which callers a callee-specification change breaks without executing them?

The corresponding contributions are: (i) a layered reference architecture with the formal contract as the pivot between the human and machine worlds, the dual-role specification, and the independent-authority constraint (section 5 develops it); (ii) an elaboration discipline that resolves the ratification gap by ratifying behaviour rather than logic, with adversarial elaboration and provenance- and assurance-tagged obligations; (iii) a brownfield characterisation phase and loop reordering that treats disagreement detection, not error-freedom, as the success criterion; (iv) the promotion of compositional refinement to a behavioural-compatibility, blast-radius capability; and (v) a proof-of-concept instantiation that reuses the programme's validated assets as trusted components—the JML-Inferrer for draft contracts, the OpenJML fork and Z3 as the oracle that never grades itself—instantiated as a partially built proof of concept on Apache Commons Lang, in which the synthesis–verification loop is exercised against fixed contracts (milestone M1) while contract generation and conversational ratification (milestones M2–M3) are specified as a staged build plan. The corpus uniquely couples code with a public issue tracker (the "LANG-####" Apache JIRA project) and rich Javadoc, yielding a genuine three-stream dataset on which real code-versus-intent discrepancies can be exhibited.

We are deliberate about what is and is not claimed. This article *proposes and partially instantiates* a reference architecture; it argues for the architecture's feasibility and realism rather than a controlled effect size. Several limitations are load-bearing and stated openly: inferred specifications skew weak because they describe what code does rather than what it should, making ratification real labor that we mitigate with inherited default contracts and API-surface prioritisation; inferred specifications are sometimes over-strong and yield false candidate bugs, which is why every discrepancy is surfaced as a *candidate* for human adjudication; NL-to-formal translation without a human is imperfect, so the resulting clauses remain unconfirmed and drive the dialogue rather than gating it; linking tickets and documentation to code is noisy and credible only on a curated module; verification scalability and decidability limits, including Z3 timeouts on multiplicative reasoning, array theory, and rich preconditions, are real and known; and, most importantly, a wrong specification verified against is worse than none, which is exactly why the independent-authority constraint and human ratification are not optional decorations but the architecture's structural load-bearers. Full empirical validation on external production codebases and an industrial action-research case study, with the attendant human-research ethics approval, is future work.

1.6 Roadmap

The remainder of the article is structured as follows. section 3 synthesizes the related work across program synthesis, specification inference, verified code generation, the test-oracle problem, and behavioural version compatibility, positioning the proposed pipeline against each. The subsequent sections present the layered reference architecture and its invariants; the elaboration layer and its resolution of the ratification gap; the brownfield characterisation phase and loop reordering; the synthesis–verification loop and its compositional change-propagation capability; and the proof-of-concept instantiation, including the verify-as-a-service, fixed-contract synthesis, contract-generation, and conversational-ratification milestones evaluated on Apache Commons Lang. The article closes with a discussion of threats to validity, limitations, and the future industrial-validation programme.

2 Background and Foundations

The reference pipeline proposed in this article assembles several mature technologies into a new lifecycle role, and its design choices are intelligible only against the formal-methods machinery they reuse. This

section establishes that machinery. It proceeds from the question of *what a specification is* and how design by contract renders one machine-checkable in the Java Modeling Language (JML); through *how such a contract is discharged* by deductive verification, extended static checking (ESC), OpenJML, and an underlying Satisfiability Modulo Theories (SMT) solver, together with the decidability and scalability limits that constrain what can be proved; to *how a contract can be obtained cheaply* through specification inference, of which the author's JML-Inferrer instrument is the concrete realisation carried forward into this work; and finally to the *generators* of code—large language models (LLMs) and the agents built on them—whose output the pipeline is designed to gate. Throughout, the intent is didactic: each foundation is introduced with the precise terminology used in the remainder of the paper, so that the contributions of section 5 can be stated without re-deriving their substrate.

2.1 Specifications and design by contract

A *specification* is a statement of intended behaviour that is independent of any particular implementation. In the formal-methods tradition this statement is a predicate, or a pair of predicates, over program states. The axiomatic basis is Hoare logic, in which a Hoare triple $\{P\}\, S\, \{Q\}$ asserts that if the precondition P holds before the statement S executes and S terminates, then the postcondition Q holds afterward [48]. Dijkstra's predicate-transformer semantics recasts this as the weakest precondition $\mathrm{wp}(S, Q)$, the weakest predicate on the initial state guaranteeing that S terminates and establishes Q (total correctness; the partial-correctness reading of the triple above corresponds to the weakest *liberal* precondition), giving a calculus for reasoning about correctness mechanically rather than by inspection [29, 30]. The refinement calculus extends this into a methodology in which an abstract specification is transformed, step by provably correct step, into executable code [10, 73]; that tradition is the intellectual ancestor of the contract-to-code *synthesis* discussed later, with the crucial difference that the pipeline of this article delegates the transformation to a fallible agent and recovers soundness by verifying the result rather than by trusting the derivation.

Design by Contract (DbC), introduced by Meyer for Eiffel, operationalizes these ideas for working object-oriented software [69, 70]. A method is governed by a contract comprising a precondition that the caller must establish, a postcondition that the method guarantees in return, and class invariants that every public method must preserve. The arrangement assigns *obligations* and *benefits* between caller and callee: the caller is obliged to satisfy the precondition and benefits from the postcondition; the callee benefits from assuming the precondition and is obliged to deliver the postcondition. This caller/callee duality is foundational to the present work, because it is exactly what makes contracts *compose*: a change to a callee's contract recomputes the obligations of every caller, the property the thesis programme exploits for version-compatibility analysis (section 2.3).

JML is the behavioural interface specification language for Java in which the pipeline expresses its contracts [58]. JML annotations are written in specially marked comments—so they are ignored by the Java compiler—and use a superset of Java's expression syntax. A `requires` clause states a precondition and an `ensures` clause a postcondition; `invariant` declares a class invariant; an `assignable` (frame) clause bounds the locations a method may modify, which is essential for sound modular reasoning because it tells the verifier what a call *cannot* change. Postconditions may refer to the pre-state value of an expression through `\old(e)` and to the returned value through `\result`, and may quantify over ranges with `\forall` and `\exists`. Listing 1 illustrates the vocabulary on a small bounded-search method; the contract states that the array is non-null, that a non-negative result indexes an element equal to the key, and that a negative result certifies the key is absent.

```
//@ requires a != null;
//@ ensures \result >= 0 ==> (\result < a.length && a[\result] == key);
//@ ensures \result <  0 ==> (\forall int k; 0 <= k && k < a.length; a[k] != key);
//@ assignable \nothing;
int indexOf(int[] a, int key) { ... }
```

Listing 1: A JML method contract using the core clauses the pipeline manipulates.

Two properties of this representation matter for the pipeline. First, a JML contract is *machine-checkable*: unlike a natural-language docstring, it has a formal semantics against which an implementation can be mechanically judged. Second, it is *partial*: a contract constrains behaviour only to the extent its clauses are written, so an under-specified contract may be satisfied by implementations the author never intended—a weakness this article treats as a first-class hazard under the headings of vacuity and spec weakening (section 5). The closest contemporary relative of JML is Spec#, which embeds contracts in C# and discharges them with a verifying compiler [12]; the design considerations transfer, but JML's tooling maturity for Java is what makes it the substrate here.

2.2 Deductive verification, ESC, OpenJML, and SMT

Possessing a machine-checkable contract is useful only if an implementation can be *checked* against it across all inputs. Deductive program verification does this by reducing the question "does this code satisfy this contract?" to a set of logical formulas—*verification conditions* (VCs)—whose validity entails correctness. The VCs are generated by a weakest-precondition computation over the method body [30]: the verifier propagates the postcondition backward through each statement, accumulating the obligations that must hold, and emits a formula asserting that the precondition implies them. Loops break this backward walk because their effect over an unknown number of iterations cannot be enumerated; the engineer (or an inference tool) must supply a *loop invariant*, a predicate true on entry, preserved by each iteration, and strong enough that together with the loop's exit condition it entails the obligation the loop must establish, which the verifier uses in place of unrolling. The need for loop invariants is the single largest source of the annotation burden that motivates specification inference.

Extended Static Checking is the pragmatic engineering of this idea pioneered by ESC/Java [40]. ESC trades completeness for automation and modularity: it analyses each method in isolation, assuming the contracts of the methods it calls rather than inlining them, and it discharges the resulting VCs automatically without interactive proof. OpenJML is the modern realisation of ESC for Java built on JML [22, 23]; it parses JML annotations, generates VCs from method bodies under their contracts, and hands those VCs to an automated theorem prover. That prover, in the configuration used by this article's instrument, is the Z3 SMT solver [27], which determines the satisfiability of formulas over combined background theories—linear integer and real arithmetic, arrays, bit-vectors, and uninterpreted functions—that match the constructs arising from Java programs; the quantifier-free fragments of these theories are decidable, whereas the quantified formulas arising from JML contracts are handled by heuristic instantiation and may return `unknown` or fail to terminate. cvc5 is a comparable alternative [11]. OpenJML asks Z3 whether the negation of each VC is satisfiable: an `unsat` answer certifies the VC and hence the obligation, while a satisfiable answer yields a model that OpenJML reports as a *counterexample*—a concrete input witnessing the violation. This counterexample is not a diagnostic convenience but a structural asset of the pipeline: it is a precise, machine-generated witness of failure that the synthesis loop feeds back to the code generator—typically a far more informative signal than "a test failed," though under modular checking such a witness can be spurious (for instance, reflecting a loop invariant too weak to carry the proof rather than a reachable defect).

It must be stated precisely what such a proof does and does not establish, because overclaiming here would undermine the entire argument. A successful ESC run proves that the implementation satisfies its contract for *all* inputs admitted by the precondition—a guarantee no finite test suite can give. It does *not* prove the contract captures what the developer wanted; that is the province of the intent layer. Equally important are the limits. ESC verifies modularly under assumed callee contracts, so its guarantee is only as strong as those contracts. Program verification is undecidable in general, and even the decidable frag-

ments Z3 targets carry combinatorial cost: in practice the solver times out on nonlinear (multiplicative) arithmetic, on rich reasoning over the array theory, and on contracts with deeply nested quantifiers. The thesis programme's own corpus exhibits exactly these failure modes, and the pipeline's design accommodates them through *tiered assurance* (section 5) rather than pretending verification always terminates. The author's instrument additionally relies on a custom OpenJML fork that adds recursive function definitions (`define-fun-rec`) for the JML aggregation operators `\sum`, `\product`, and `\num_of`, which the stock toolchain does not discharge; this fork is reused as a trusted component in the proof-of-concept of section 10. Comparable deductive ecosystems—Dafny's auto-active verifier [59], the KeY interactive prover for Java [1], Frama-C for C [55], and Verus for Rust [57]—share the same VC-to-SMT architecture and the same fundamental limits, confirming that the constraints discussed here are intrinsic rather than artefacts of one tool.

2.3 Specification inference and the JML-Inferrer

The chief obstacle to deploying contract-based verification is the cost of *authoring* the contracts. Specification inference attacks this bootstrap cost by deriving candidate specifications automatically. Two traditions exist. Dynamic inference, exemplified by Daikon, observes program executions and reports invariants that held over the traces as *likely* program properties [36]; the inferred properties are only as representative as the test inputs and require subsequent validation, and integrating them with static checkers is itself a research thread [76]. Static inference instead reasons about the code without running it: Houdini conjectures a large pool of candidate annotations and iteratively retracts those a checker refutes [39]; abstract interpretation computes sound over-approximations of reachable states [25]; and dedicated invariant-learning frameworks such as ICE [43] and neural approaches like Code2Inv [89] target the loop-invariant problem specifically. A separate strand mines specifications from execution traces as API-protocol automata [6].

The instrument carried into this work, the *JML-Inferrer*, occupies the static end of this space. It is a Java 21 tool built on the JavaParser library that analyses a method's abstract syntax tree (AST) and emits JML annotations—preconditions, postconditions, loop invariants, class invariants, and `assignable` clauses—by heuristic pattern matching over AST structure rather than by a complete program analysis. A null-check or array access on a parameter induces a non-nullness precondition; recognised loop shapes (counting up or down, accumulating, linear search, running maximum or minimum) induce matching invariants and result postconditions; numeric guards induce range and overflow preconditions. The heuristic basis is a deliberate trade: the inferrer favors recall of plausible, syntactically valid JML over the soundness guarantees of abstract interpretation, on the premise that an automated checker (section 2.2) will reject any inferred clause that does not hold. This instrument is not introduced here; it is the validated artefact of the author's prior programme, reused as the brownfield contract-bootstrapping component of the proposed pipeline.

Four results from that programme are load-bearing for the present article and are cited as the author's own prior work. Article 1 established that automatically inferred JML specifications materially improve LLM-based unit-test generation, the first evidence that inferred contracts have downstream value for AI-assisted development [31]. Article 2 showed that JML specifications can be embedded into compiled Java artefacts and into OpenAPI service descriptions with full roundtrip fidelity, so a contract can travel with the code it governs [32]. Article 3 demonstrated that heuristic inference admits *compositional refinement* across method boundaries—a second weakest-precondition pass over the call graph recovers preconditions a single-method pass misses, contributing on the order of 18,854 additional well-formed preconditions on the experimental corpus [33]. Because contracts compose along the caller/callee obligation structure of DbC (section 2.1), this same mechanism lets a callee-spec change recompute caller obligations *statically*, which is the technical bridge from inference to the version-compatibility capability of section 5. Article 4 integrated the inference pipeline into continuous-integration systems with diff-only analysis, content-hash caching, and fail-open semantics, establishing that the toolchain runs at the cadence of ordinary development [34].

Together these results supply the reused, trusted components on which the proof-of-concept of section 10 is built, and they ground the present article's claim that the missing piece is not the verification machinery but its *lifecycle organisation*.

2.4 LLMs and agents as code generators

The fourth foundation is the code generator the pipeline gates. LLMs trained on source code can map a natural-language description to plausible code: Codex demonstrated function-level generation from docstrings and introduced the *pass@k* functional-correctness metric [18], with MBPP and AlphaCode extending the evidence to entry-level and competition-level tasks respectively [9, 60]. These systems inverted the economics of classical synthesis—natural-language intent is cheap and abundant—but estimate correctness against finite, model-adjacent test suites rather than proving it against a contract. That estimate is fragile: benchmark suites are often too weak to expose plausible-but-wrong solutions, and strengthening them lowers reported pass rates [62]; and generated code is prone to confident fabrication of non-existent APIs and subtly inconsistent logic that survives superficial review [108].

The frontier has since moved from isolated functions to *agents* that operate over whole repositories. SWE-bench reframed evaluation around resolving real GitHub issues with project test suites as the oracle [53]; SWE-agent showed that a tailored agent–computer interface markedly improves repository navigation and editing [104]; and AutoCodeRover paired LLM reasoning with structure-aware search and fault localization to localize and patch issues [107]. A complementary line equips agents with iterative self-correction—Self-Refine [66], Reflexion [88], execution-trace-driven self-debugging [20], and the test-driven flow of AlphaCodium [85]—while multi-agent frameworks such as MetaGPT [49], ChatDev [83], and AutoGen [101] coordinate role-specialized agents over a development workflow, as surveyed recently for software engineering [63]. These advances are real, but they preserve two structural defects: the specification of intent remains natural-language issue text, and the acceptance gate remains an executable test suite—frequently the project's own, which the agent may also consult or edit, so the model effectively grades itself. This circularity is acute in oracle generation, where the same model authors both the code and the tests meant to judge it [64].

These defects are being institutionalized rather than resolved. Emerging *spec-driven* lifecycles—AWS's AI-Driven Development Lifecycle [4], GitHub's Spec Kit [44], and the Kiro IDE [5], all reflecting the broader repositioning of LLMs within the software lifecycle [78]—reorganise development around a specification, but that specification is natural language, the loop from specification to code is unverifiable, and verification, where present, remains self-grading. The contribution of this article, developed in section 5, is to interpose a machine-checkable formal contract as the pivot of these lifecycles: a single artefact that serves simultaneously as the input intent an agent builds against and the oracle its output is verified against by OpenJML over all inputs, with an independent-authority constraint and provenance- and assurance-tagged obligations ensuring the verifier never grades its own author. The foundations assembled here—contracts and their composition, ESC and its limits, inference and its prior validation, and the agentic generators and their circularity—are exactly the pieces that pipeline re-couples.

3 Related Work

The pipeline proposed in this article sits at the confluence of several research traditions that have, until recently, evolved largely in isolation: the generation of code by large language models, the formal specification and verification of programs, the automated inference of specifications, and the empirical study of how software is documented, evolved, and kept compatible across versions. This section surveys each of these traditions in turn. For each we identify the works most relevant to a verification-centric development lifecycle and, at the close of each subsection, locate the precise boundary at which existing work stops and the

contribution of this article begins. We then draw the threads together in a concluding synthesis (section 3.11) that states the gap the proposed pipeline fills.

3.1 LLM-based code generation and program synthesis

The automatic production of executable code from a higher-level description is a long-standing goal of computer science, and the recent confluence of classical program synthesis with large language models (LLMs) has dramatically reshaped its landscape. Understanding where this article's reference pipeline sits requires distinguishing two intellectual lineages that the pipeline deliberately re-couples: *specification-driven* synthesis, in which a precise formal artefact defines the target, and *description-driven* generation, in which an LLM maps natural-language (NL) intent to plausible code.

The synthesis lineage is the older of the two. Deductive synthesis derives a program from a constructive proof that a formal specification is satisfiable [67], while inductive and example-driven approaches infer programs from input–output behaviour, as in programming-by-example systems that synthesize string transformations from a handful of demonstrations [46]. Counterexample-guided inductive synthesis (CEGIS) and the closely related *sketching* paradigm partition the problem into a synthesizer that proposes a candidate and a verifier that either accepts it or returns a counterexample to refine the search [91]. The defining property of this lineage is that the target is a machine-checkable contract and the loop is *closed*: a candidate is admitted only when an oracle certifies it. That oracle, however, has historically been the bottleneck—formal specifications are expensive to author, and synthesis scales poorly beyond small, well-circumscribed problems.

The description-driven lineage emerged with the application of transformer language models to source code. Codex demonstrated that a model trained on public repositories could generate functionally correct programs from docstrings, and introduced the HumanEval benchmark together with the now-standard *pass@k* metric for functional correctness [18]; the contemporaneous MBPP benchmark established a complementary set of entry-level Python tasks [9]. Competition-level results followed, with AlphaCode reaching median human performance on programming contests by massive sampling and filtering [60]. These systems inverted the economics of the synthesis lineage—NL intent is cheap and abundant—but at the cost of abandoning the closed loop: correctness is estimated against a finite, model-adjacent test suite rather than proved against a contract. Subsequent scrutiny exposed the fragility of this estimate. Benchmark test suites are frequently too weak to detect plausible-but-incorrect solutions, and strengthening them materially lowers reported pass rates [62]. More fundamentally, LLM code generation is prone to *hallucination*: confidently produced fabrications such as non-existent APIs, intent conflicts, and inconsistent context dependencies, which evade superficial review precisely because the surrounding code is well-formed [108]. Because the generator and its informal oracle share a model and a training distribution, the verification is effectively self-grading.

The frontier of the description-driven lineage has moved from function-level snippets to repository-level, *agentic* software engineering. SWE-bench reframed evaluation around resolving real GitHub issues against full codebases with project test suites as the oracle [53], and a wave of agents followed: SWE-agent showed that a purpose-built agent–computer interface substantially improves an LLM's ability to navigate and edit a repository [104], while AutoCodeRover combined LLM reasoning with structure-aware code search and spectrum-based fault localization to localize and patch issues [107]. These agents are impressive, but they preserve and indeed amplify the two structural defects of the lineage: the specification of intent remains the NL issue text, and the acceptance gate remains an executable test suite—typically the project's own tests, which the agent may also be permitted to consult or modify, reintroducing circularity at the repository scale.

The gap this article addresses lies precisely at the seam between these lineages. Agentic generation has solved the authoring-cost problem that crippled classical synthesis, but in doing so it discarded the machine-checkable contract and the closed, counterexample-driven loop that made synthesis trustworthy. This ar-

ticle's contribution is to restore that loop in an LLM-native setting by interposing a *formal-specification contract layer* that serves the dual role of input intent and verification oracle, with the contract verified against *all* inputs by OpenJML extended static checking rather than sampled by tests. Crucially, the oracle is not the generator: an independent-authority constraint and provenance- and assurance-tagged obligations make the "verified" claim auditable, directly answering the self-grading critique that the reliability literature raises [62, 108]. The synthesis–verification loop is CEGIS-shaped [91], but operates at the granularity of agentic edits [104, 107] and degrades gracefully when verification does not terminate. Finally, where the agentic benchmarks are greenfield-issue-centric [53], this article targets the brownfield reality of most software through multi-source intent triangulation—reconciling code, tickets, and documentation into a provenance-tagged contract—and lifts the author's prior compositional inference [33] into a lifecycle capability, so that a callee-spec change recomputes caller obligations without re-running them.

3.2 Agentic Software Engineering and Spec-Driven AI-Native Lifecycles

The integration of large language models (LLMs) into software construction has progressed rapidly from single-shot code completion to autonomous agents that plan, edit, execute, and repair across an entire repository. A first generation of techniques established that an LLM's output improves markedly when it is embedded in an iterative loop rather than emitted in one pass. *Self-Refine* demonstrated that a single model can critique and rewrite its own output without additional training [66], while *Reflexion* showed that verbalized feedback retained in an episodic memory buffer steers an agent toward better subsequent attempts [88]. For code specifically, *Self-Debugging* taught models to inspect execution traces and explain their own faults [20], and *AlphaCodium* recast competitive-programming synthesis as a multi-stage "flow" in which the model reasons, generates, and then repairs candidate solutions against tests [85]. These approaches share a common architecture—generate, observe a signal, regenerate—that prefigures the synthesis loop this article formalizes; their limitation, however, is the nature of that signal. Each closes its loop on either model self-assessment or a finite, model-authored test suite, so the feedback is both circular and incomplete: the same model that wrote the code also adjudicates it, and passing the chosen examples says nothing about untested inputs.

A second strand scaled the loop to full software engineering tasks against real repositories. *SWE-agent* showed that a carefully designed agent–computer interface lets an LLM navigate, edit, and test a codebase to resolve genuine GitHub issues [104], evaluated on the now-standard SWE-bench benchmark of real-world issues and their reference patches [53]. In parallel, multi-agent frameworks decompose development across specialized roles: *MetaGPT* encodes standardized operating procedures across product-manager, architect, and engineer roles [49]; *ChatDev* organizes a virtual software company whose agents converse through a structured "chat chain" spanning design, coding, and testing [83]; and *AutoGen* provides a general substrate for orchestrating conversing agents and tool use [101]. Comprehensive surveys document the breadth of this turn from LLMs to LLM-based agents for software engineering [63]. Across these systems the dominant correctness oracle remains test execution—reference tests in the benchmark case, or model-generated tests otherwise—which inherits the classical incompleteness of testing: it samples the input space and demonstrates the presence of behaviour, never its absence over all inputs.

Most directly related is the emerging family of *spec-driven* and *AI-native* development lifecycles, which reorganise the process so that a specification, rather than code, becomes the primary human-authored artefact. GitHub's *Spec Kit* formalizes a four-phase workflow—specify, plan, tasks, implement—anchored by a "constitution" of immutable project principles [44]. Amazon's *AI-Driven Development Lifecycle* (AI-DLC) restructures the lifecycle into inception, construction, and operations phases with built-in quality gates and human approvals, including a reverse-engineering step for brownfield codebases [4], and the *Kiro* IDE places the requirements-to-design-to-tasks spec workflow at the centre of the editor [5]. These efforts correctly identify that, when an agent can regenerate an implementation cheaply, the durable artefact is the

specification, and broader position papers argue that LLMs will reshape the SDLC accordingly [78]. Their decisive shortcoming, however, is that the specification is expressed in *natural language*. Consequently the specification-to-code step is a *compilation without a checker*: there is no mechanical relation guaranteeing that the generated implementation satisfies the stated intent, the loop remains open (generate, then review by eye), and where verification appears at all it is the circular, self-grading test execution noted above. The natural-language artefact is also too imprecise to support automated reasoning over change: these lifecycles offer no way to compute, without re-running anything, which dependents a modification might break.

This article addresses precisely that missing verification spine. In contrast to the surveyed work, it proposes a *machine-checkable formal-contract layer* as the pivot between ambiguous human intent and precise machine behaviour, in which the contract serves the dual role of input to synthesis and oracle for verification. Verification is performed by OpenJML Extended Static Checking over *all* inputs rather than a sampled suite, closing the open loop with sound counterexamples instead of failing test indices, and an *independent-authority constraint* forbids the agent that writes the code from authoring the contract it is judged against, eliminating the self-grading circularity endemic to the prior loops. Because formal contracts compose, the approach further treats a callee-specification change as a recomputation of caller proof obligations—turning "which call sites does this upgrade break?" into a first-class, run-free analysis that the natural-language lifecycles cannot express. Finally, recognising that most software is brownfield, the article contributes a multi-source characterisation phase that triangulates code, issue tickets, and documentation to recover a provenance-tagged candidate contract and to surface code-versus-intent disagreements as candidate bugs, going beyond the simple reverse-engineering step offered by AI-DLC.

3.3 Design by Contract and Specification-Driven Development

The notion that a program component should be governed by a precise, machine-checkable agreement between supplier and client predates the present interest in agentic software construction by several decades, and it is from this lineage that the formal-contract layer proposed in this article descends. The semantic foundations were laid by Hoare's axiomatic treatment of program correctness, in which a triple relates a precondition, a command, and a postcondition [48], and by Dijkstra's predicate-transformer calculus, which renders the weakest precondition of a statement with respect to a desired postcondition computable [30]. These two results are the technical bedrock on which both deductive verification and the compositional change-propagation analysis used later in this article rest: the weakest-precondition transformer is precisely the mechanism by which a change to a callee's contract can be pushed back through its call sites.

Meyer reified these ideas into an engineering discipline he named Design by Contract (DbC), embedding preconditions, postconditions, and class invariants as first-class, executable constructs in the Eiffel language [69, 70]. The decisive move was to treat the contract not as external documentation but as a durable program artefact with defined obligations on each party and defined blame when an obligation is violated. This framing—contract as the authoritative statement of intent, code as its fulfillment—directly anticipates the economic inversion this article builds upon, in which a cheaply regenerable implementation becomes a build output and the contract becomes the scarce, durable asset. DbC's principal limitation, however, is that its assertions are checked only at runtime against the inputs that happen to occur; the guarantee is sampled, not universal.

The behavioural interface specification tradition closes part of this gap by enriching contracts to a point at which static proof becomes tractable. The Java Modeling Language (JML) marries Eiffel-style contracts with the model-based specification heritage of Larch and elements of the refinement calculus, yielding a notation expressive enough for full functional specification of Java classes [58]. OpenJML discharges such specifications by Extended Static Checking (ESC), translating a method together with its contract into verification conditions checked by an SMT solver, thereby establishing correctness over *all* inputs rather than a sampled subset [22, 23]. Spec# [12] pursued the same agenda for C#, demonstrating that a contract layer

can be retrofitted onto a mainstream object-oriented language with a verifying compiler. Underpinning the static-proof tradition is the refinement calculus of Back and Morgan [10, 73], which formalizes the stepwise transformation of an abstract specification into executable code with each step preserving correctness—the correctness-by-construction ideal that the synthesis-and-verification loop in this article approximates pragmatically, replacing manual refinement steps with agent-generated candidates that are filtered by a verifier.

A persistent obstacle to all of the above is the cost of authoring formal contracts in the first place, and a substantial body of work addresses this by *inferring* specifications rather than demanding them. Dynamic detection, exemplified by Daikon, observes executions and reports likely invariants [36], while Houdini infers candidate annotations and retains those an underlying checker can prove [39]. More recently, large language models have been applied to specification synthesis: SpecGen drives an LLM through conversational refinement and mutation, selecting candidates a verifier accepts [65], and related efforts translate natural-language intent into temporal or behavioural specifications [24]. The author's own prior work fits squarely in this inference tradition, recovering JML by AST heuristics and showing that such specifications materially improve downstream artefacts and admit compositional refinement across method boundaries [31, 33].

What this collected literature does not supply is the missing element for the emerging spec-driven, agentic lifecycle. The classical DbC and JML traditions assume a human author who can and will write formal contracts, an assumption the natural-language–first agentic workflow explicitly contradicts; inference work, conversely, recovers contracts that faithfully describe what code *does* rather than what it *should*, and—crucially—does not address *who* owns the resulting contract when that same contract must serve as the oracle against which generated code is judged. None of these strands treats the specification simultaneously as the input the agent builds against and the oracle it is verified against, nor do they confront the brownfield reality in which a contract must be triangulated from code, issue tickets, and documentation with explicit provenance, conflicts surfaced as candidate bugs rather than suppressed. This article positions its machine-checkable contract layer, its independent-authority and ratification constraints, and its compositional change-propagation analysis precisely at that gap, promoting the inference and verification machinery surveyed here into a governed lifecycle pivot rather than an isolated tool.

3.4 Dynamic and static specification and invariant inference

The pipeline proposed in this article depends on the ability to recover a draft formal contract from existing code, a capability with a long lineage in the specification- and invariant-inference literature. That literature divides naturally into dynamic and static traditions, and more recently into a learning-based strand; understanding where each is strong and where it is brittle clarifies precisely what our inference instrument contributes and, equally, what it deliberately does not claim.

The dynamic tradition observes a running program and reports properties that held over the executions seen. Its canonical instrument is Daikon, which instruments a target, records variable values at program points, and reports the templated properties (equalities, ordering, bounds, linear relationships) that survived every observed trace [36]. Daikon established the central conceptual point that our brownfield characterisation phase reuses: a specification can be *recovered* from behaviour rather than authored from scratch, lowering the bootstrap cost that has historically deterred adoption of formal contracts [69]. The same conceptual move underlies specification mining, in which finite-state protocols governing API usage are learned from execution traces [6]. The defining limitation of every dynamic technique, however, is soundness: an inferred property is only *likely* true, because it summarises the sampled executions, not all of them. This is exactly the distinction our architecture makes load-bearing. A code-derived clause is, in our terminology, a `code-inferred` obligation of `unchecked` assurance until OpenJML discharges it across all inputs, at which point it is promoted to `proved`. Dynamic inference supplies a hypothesis; it does not supply an oracle.

The static tradition derives invariants by sound over-approximation of program semantics. Abstract

interpretation provides the foundational framework, computing fixpoints over abstract domains so that every reported property provably holds on every execution [25]. Where abstract interpretation infers invariants from first principles, Houdini takes the complementary route of *guess-and-check*: it postulates a large pool of candidate annotations and uses the ESC/Java verifier to refute the unprovable ones, retaining only the inductive remainder [39]. Houdini is the historical antecedent of the verifier-in-the-loop discipline our synthesis–verification loop generalises, and its candidate-refinement structure prefigures the counterexample-guided refinement (CEGIS) shape we adopt for code synthesis. The static methods buy soundness at the price of expressiveness and scale: rich abstract domains do not always converge, and the decidability and capacity limits of the underlying decision procedure bound what can be discharged. Our own honest reporting of Z3 timeouts on multiplicative reasoning, array theory, and rich preconditions is the contemporary form of precisely this limit, and it is the reason our tiered-assurance mechanism degrades full proof to bounded check to test rather than silently dropping an obligation.

A third, recent strand learns invariants and specifications directly. Data-driven frameworks such as ICE pose invariant synthesis as learning from examples, counterexamples, and implication pairs supplied by a verifier [43], and neural approaches reformulate the same problem as graph- or trace-conditioned prediction [89]. Since 2023 the focus has shifted to large language models (LLMs): models have been fine-tuned to predict Daikon-style invariants statically [81], prompted to emit candidate loop invariants that symbolic tools then check [54], and augmented with learned re-ranking to triage many generated candidates against a sound verifier [17]. These works share a structural commitment with our pipeline: the generative model is never trusted on its own, but is paired with a sound checker that admits only what it can prove. Yet they remain narrow—focused on the single artefact of a loop invariant for an isolated verification task—and they do not address provenance, the human-ratification problem, or evolution.

Across all three strands, the literature treats inference as a terminal deliverable: the inferred property is the output. Our contribution repositions inference as the *opening move* of a verification-centric lifecycle. First, recovered clauses are not presented as fact but carried as provenance- and assurance-tagged obligations, so the near-circular nature of verifying code against a code-derived spec is made explicit rather than overclaimed as bug-freedom. Second, we triangulate the code-derived draft against two further intent streams—issue tickets and documentation—and treat *disagreement* between streams as the signal of interest, surfacing candidate bugs that single-stream inference cannot. Third, because the inferred contracts are compositional, a change to a callee specification recomputes caller obligations and flags behavioural incompatibilities without execution, promoting our prior compositional-refinement result (Article 3 of this programme, [33]) from an inference technique to a lifecycle capability for version-compatibility analysis. No prior inference system combines recovery, multi-source reconciliation, sound verification across all inputs, and compositional change propagation into a single contract layer; that integration is the gap this article addresses.

3.5 LLM-Based Specification, Contract, and Formal-Spec Inference

The proposition that executable contracts can serve as the durable, machine-checkable artefact of a software system long predates the present interest in large language models (LLMs). Design by contract [69] established preconditions, postconditions, and invariants as first-class obligations that bind callers and callees, while dynamic invariant detection, exemplified by Daikon [36], demonstrated that likely specifications could be *recovered* from program behaviour rather than authored by hand. These two ideas—the contract as a verifiable interface and the specification as an inferable artefact—anchor the present article, but classical inference is confined to properties witnessed in observed executions and offers no bridge from the informal human intent that motivates a specification in the first place. Bridging that gap is precisely what the recent wave of LLM-based specification work attempts, and it is against this body of work that the contribution of this article must be positioned.

A first thread translates unstructured natural language directly into formal artefacts. The nl2spec framework [24] maps English requirements to temporal logic and, recognising the inherent ambiguity of natural-language requirements, lets users iteratively correct sub-translations rather than redraft an entire formula. Closer to the contract setting, Endres et al. [35] study *nl2postcond*, prompting LLMs to convert code-adjacent natural language into formal-method postconditions, and report that the generated postconditions both correctly capture intent on a large fraction of problems and catch real historical bugs from Defects4J—direct evidence that an intent-derived oracle can disagree productively with an implementation. This last observation is foundational to the brownfield characterisation phase developed in this article, yet nl2postcond stops at producing a candidate assertion; it neither ratifies that assertion with the human nor guards against the synthesizing model silently weakening the intent it was asked to formalise.

A second, larger thread targets the specifications needed to make automated verification succeed, particularly the loop invariants that are the classical bottleneck of deductive verification. Pei et al. [80] show that fine-tuned models can predict invariants statically at a quality comparable to dynamic analysis, and a sequence of systems—Loopy [54], Lemur [100], and AutoSpec [98]—couple an LLM proposer to a sound checker so that only machine-validated invariants survive. Lemur, in particular, formalizes the LLM-plus-reasoner loop as a sound calculus, while AutoSpec drives synthesis with static analysis and per-round verification to avoid error accumulation. SpecGen [65] extends this conversational, feedback-driven scheme to full JML method specifications for Java, falling back to mutation-based search when the model's output fails the verifier. The defining discipline of this thread is that the verifier, not the model, is the arbiter of soundness—a discipline this article adopts wholesale by reusing OpenJML and Z3 as a trusted oracle the synthesizing agent never grades.

A third thread closes the loop between specification, code, and proof. Baldur [38] generates and repairs whole proofs with LLMs, feeding failed attempts and their error messages back into generation. Clover [94] reduces correctness checking to mutual consistency among code, documentation, and formal annotations, using a deductive verifier together with an LLM, and the verified-Dafny synthesis of Misu et al. [71] generates specification and implementation together so the code can be proved against its own spec. These systems realise a counterexample-guided generate-and-check loop and confirm its feasibility, but they share a structural limitation that this article makes its central concern: the specification and the code it grades typically originate from the *same* model, so the verification is, in the terminology developed here, self-grading. Where the specification is itself the output of the model under test—as in nl2postcond, Clover, and Dafny synthesis—a model that misreads the intent will produce an oracle that excuses its own implementation, and consistency among model-authored artefacts is not correctness relative to intent.

The work surveyed here thus establishes every component this article builds upon—inference of draft specifications, NL-to-formal translation, verifier-checked invariant synthesis, and the CEGIS-shaped synthesis loop—yet leaves four gaps that the proposed pipeline addresses jointly. First, none separates the authority that writes the contract from the agent that writes the code; this article makes *independent authority* an explicit, provenance-auditable constraint. Second, none confronts the *ratification gap*: an inferred or NL-derived contract is treated as ground truth even though the human who supplied the intent cannot read the formal text, whereas this article ratifies *behaviour* through concrete pinned examples and surfaces only the boundary decisions an agent had to resolve. Third, the surveyed systems are overwhelmingly greenfield, generating fresh code against fresh specs, whereas real software is brownfield; this article recovers contracts by triangulating three disagreeing evidence streams—code, issue tickets, and documentation—and treats the disagreements as candidate bugs rather than errors to suppress. Fourth, prior work verifies a method in isolation, whereas this article promotes the compositional refinement of our prior work [33] to a lifecycle capability, recomputing caller obligations when a callee contract changes so that internal refactors and dependency-version bumps become the same machine-checkable blast-radius analysis [34]. The novelty lies not in any single inference technique but in assembling these into a verification-centric reference architecture in which a provenance- and assurance-tagged formal contract, never authored by the implementer,

serves simultaneously as the agent's input and its oracle.

3.6 Deductive verification and verified program synthesis

The proposal in this article rests on a long tradition of deductive program verification, in which a program is proved to satisfy a contract written in a machine-checkable specification language. The foundational idea is Hoare logic [48] and Dijkstra's weakest-precondition calculus [29], which reduce correctness to the discharge of verification conditions, and design-by-contract [69], which made pre/postconditions and class invariants a unit of software construction. The Java Modeling Language (JML) [58] is the direct descendant of this lineage for Java, and is the contract language this article adopts. Practical tooling clusters into two families. *Extended static checking* trades completeness for automation: ESC/Java [40] and its successor OpenJML [22, 23] translate annotated Java into verification conditions discharged by a Satisfiability Modulo Theories (SMT) solver such as Z3 [27] or cvc5 [11]. *Auto-active verifiers* ask the programmer to supply more annotation but aim at full functional correctness: Dafny embeds specifications, loop invariants, and termination metrics in a programming language designed for verification [59]; Frama-C with its ACSL contract language targets industrial C [55]; KeY uses an interactive/symbolic-execution prover for Java [1]; and the recent Verus reuses Rust's linear types and borrow checker to verify systems code at scale [57]. This article reuses the OpenJML–Z3 combination, including a custom fork that adds recursive function definitions for `\sum`, `\product`, and `\num_of`, as a *trusted oracle* rather than proposing a new prover; the deliberate constraint is that the verifier never grades artefacts of its own authorship.

A parallel tradition concerns the synthesis of programs that are correct by construction. Counterexample-guided inductive synthesis (CEGIS) [92, 93] is the canonical loop: a candidate is proposed, a verifier searches for a counterexample, and the witness is fed back to refine the next candidate. The SyGuS formulation [3] standardized the syntax-guided variant. This generate–check–refine structure is precisely the shape the present article gives to its synthesis–verification loop, with two adaptations: the candidate generator is a large language model (LLM) rather than an enumerative search, and the SMT counterexample is surfaced as a concrete behavioural witness for the synthesis agent. The proposal also lifts the CEGIS idea *above* the implementation layer into *adversarial elaboration*, where a second, independent agent searches for scenarios consistent with the natural-language intent but violating the proposed contract—a counterexample search against specification weakening rather than against code.

The most active recent strand couples LLMs with verifiers so that generated code or proofs are machine-checked rather than merely sampled and inspected. Clover reduces correctness to consistency among code, docstrings, and formal annotations, checked with Dafny and an LLM, and notably tolerates no false positives on adversarial inputs [94]. Other work synthesizes verified Dafny methods directly [71], repairs proofs in Verus by reasoning over counterexamples [103], drives Dafny verification with inference-time feedback loops [75], and benchmarks the broader task of generating verified code from formal specifications ("vericoding") [7]. In adjacent territory, LLM-guided interactive theorem proving over Lean has reached competition-level mathematics via retrieval [105] and reinforcement learning [2]. These systems share an architectural commitment this article endorses: a sound external checker, not the generator's own judgment, is the arbiter of correctness. They differ from the present proposal in three respects that define its gap. First, almost all of them assume the formal specification is *given*—hand-written by the user, or, as in Clover and Self-Spec-style approaches, authored by the same model that writes the code, which reintroduces the circularity (the same agent grading its own output) that the proposed *independent authority constraint* forbids. Second, they operate almost exclusively in the *greenfield* setting of small, self-contained benchmark methods (CloverBench, DafnyBench, MBPP-DFY), whereas most real software is brownfield: the contract must be *recovered* from existing code, issue tickets, and documentation, with disagreements between those streams treated as candidate bugs rather than as noise to be smoothed away. Third, they verify a single method against a fixed contract and stop; none treats a callee-specification change as a lifecycle event that

recomputes caller obligations and flags which call sites break without running them.

It is this last capability that connects deductive verification to the thesis of the programme. Specification inference can lower the authoring cost that has historically limited adoption—Daikon recovers likely invariants dynamically [36], and the present authors' JML-Inferrer recovers JML contracts statically by AST heuristics and shows that a second weakest-precondition pass adds compositional preconditions a single pass misses [33]. The contribution of this article is to assemble these threads—a trusted SMT-backed oracle, a CEGIS-shaped synthesis loop, an LLM generator held at arm's length from the contract, brownfield multi-source characterisation, and compositional change propagation—into a single verification-centric reference pipeline in which the machine-checkable contract, not the disposable code, is the durable source of truth.

3.7 The Test-Oracle Problem and LLM-Based Test Generation

A test exercises a program; an *oracle* decides whether the observed behaviour is correct. Barr et al. [13] survey this oracle problem and establish that, while input generation is now largely automatable, the oracle remains the binding constraint on test automation: without a trustworthy arbiter of correctness, generated tests can only detect crashes and uncaught exceptions, not functional deviation. Their taxonomy distinguishes *specified* oracles, derived from a formal description of intended behaviour, from *derived* and *implicit* oracles inferred from artefacts or general expectations. This article positions its formal-contract layer squarely as a *specified* oracle, but one that is recovered and ratified rather than assumed to pre-exist.

A long line of work attacks the oracle problem by sidestepping the need for an absolute correctness criterion. Metamorphic testing checks *relations* between multiple executions—for example, that permuting a list and sorting it yields the same result as sorting and permuting—rather than asserting any single output; Chen et al. [19] and Segura et al. [87] survey the technique and its reach into domains where no conventional oracle exists. Metamorphic relations are expressive but partial: they constrain behaviour without fully pinning it, and an implementation can satisfy every relation while still being wrong. A complementary tradition *infers* oracles from program behaviour. Daikon detects likely invariants by observing values across executions and reporting properties that held over the observed runs [36]; the resulting invariants are candidate oracles, but they describe what the code *did*, not what it *should* do, and are unsound by construction. This description-versus-prescription gap is exactly the tension this article confronts in its brownfield characterisation phase, where a code-derived contract is treated as evidence of intent and prime suspect simultaneously.

Search-based and feedback-directed generators sharpened the gap by automating inputs at scale while leaving the oracle largely implicit. EvoSuite evolves whole test suites toward coverage targets and, lacking a specification, emits *regression* assertions that merely capture current behaviour [42]; Randoop generates sequences by feedback-directed random testing and flags only contract violations and crashes [79]. Both expose the central weakness: a test suite with self-generated assertions is a snapshot, not a correctness judgment, and it will faithfully ratify a buggy implementation. Learning-based oracle synthesis followed, treating assertion generation as a translation task. ATLAS learns to emit assert statements for a test and its focal method via neural machine translation [97], and AthenaTest fine-tunes a transformer on mined focal-method/test pairs [96]. These systems learn the *form* of plausible oracles from human-written corpora but inherit whatever (in)correctness those corpora encode, and they offer no machine- checkable guarantee that a generated assertion reflects intended behaviour.

The arrival of large language models (LLMs) has accelerated this work without resolving the underlying problem. TestPilot generates and adaptively repairs unit tests by re-prompting the model with failing tests and error messages [86]; ChatTester iteratively refines ChatGPT-produced tests and improves over earlier neural baselines on compilability and coverage [106]; broad empirical studies report that LLM-generated suites achieve competitive coverage and superior readability yet still suffer correctness and compilation defects [90]. Crucially, recent analyses of *LLM-driven oracle generation* document a structural hazard: when the same model both implements the code and writes the asserting oracle, the oracle tends to encode

the model's own (possibly mistaken) expectations, yielding tests that pass by construction [64]. This is the self-grading, circular-verification defect that motivates the present work.

The gap, then, is not a shortage of test-generation machinery but the absence of an oracle that is at once *machine-checkable*, *prescriptive*, and *independently authored*. Sampling-based oracles—metamorphic relations, inferred invariants, learned or LLM-written assertions—judge correctness on a finite set of executions and, when produced by the implementing agent, collapse into circularity. This article instead elevates a formal JML contract to the oracle and discharges it with OpenJML over *all* inputs rather than samples, separating the elaborator role that authors the contract from the implementer role that satisfies it (the independent-authority constraint). Where prior oracle-inference work stops at unsound, code-derived descriptions, the proposed pipeline tags each obligation with provenance and assurance, triangulates code against tickets and documentation to surface bugs as *disagreements*, and feeds OpenJML counterexamples back into a CEGIS-shaped synthesis loop—turning the inferred-invariant tradition of Ernst et al. [36] and the LLM-test tradition of Schäfer et al. [86] from a description-and-eyeball workflow into a verified, prescriptive, change-propagating contract layer.

3.8 API Evolution, Breaking-Change Detection, and Behavioural Compatibility

The discipline of managing how a software interface changes over time is the empirical backdrop against which this article's compositional change-propagation mechanism must be positioned. The governing convention is semantic versioning [82], which encodes a behavioural promise in the version string: a major increment signals an incompatible change, a minor increment signals backward-compatible additions, and a patch increment signals backward-compatible fixes. A substantial body of empirical work has examined whether practice honors this promise. Raemaekers et al. [84] analysed seven years of release history in Maven Central and found that roughly one third of all releases introduce at least one breaking change, frequently without the corresponding major increment, undermining the version string as a compatibility signal. Ochoa et al. [77] later replicated and refined this analysis over 119,879 library upgrades and 293,817 clients, reporting a more optimistic 83.4% compliance rate and a rising trend over time, but crucially observing that most breaking changes touch code no client actually uses—only a small fraction of clients are affected in practice. Decan et al. [28] broadened the lens across seven packaging ecosystems, characterising backward-incompatible updates and dependency staleness as a shared, structural hazard of large dependency networks rather than an ecosystem-specific accident. Collectively this literature establishes that the version string is a weak, statistically unreliable proxy for the property developers actually care about: whether *their* call sites still behave correctly.

A second strand attacks the detection problem directly. Tools such as APIDiff [15] and the framework Maracas underpinning Ochoa et al. [77] compare two library versions at the level of types, methods, and fields, computing a delta of signature-level changes and, in the case of Maracas, the set of client locations each change impacts. Xavier et al. [102] performed a large-scale historical study of such syntactic breaking changes and their client impact, and Brito et al. [16] catalogued the reasons developers introduce them. These signature-diffing techniques are mature and precise, but they share a definitional ceiling: they detect changes to the *shape* of an interface, not to its *behaviour*. A method whose signature is untouched but whose contract has silently shifted—an off-by-one boundary, a newly thrown exception, a changed null-handling policy—is invisible to them.

This ceiling motivates the literature on *behavioural* backward incompatibility, which is the closest prior work to the present article. Mostafa et al. [74] manually studied 296 behavioural backward incompatibilities across popular Java libraries, showing that changes with untouched signatures are both common and disproportionately damaging precisely because no diffing tool flags them. Foo et al. [41] proposed efficient static checking of library updates to surface such hazards without full re-execution, and Jezek and Dietrich [52] demonstrated experimentally that detection rates vary substantially across existing tools, with

many real incompatibilities escaping every one. Most recently, Jayasuriya et al. [51] quantified the phenomenon at scale, finding that 11.58% of dependency updates carry client-impacting breaking changes and that 2.30% of updates introduce behavioural breaks detectable only when client tests are executed. The recurring methodological move in this strand is to recover behaviour dynamically—through the client's own test suite, regression runs, or learned invariants in the style of Daikon [36]—which inherits the fundamental limitation of testing: it observes sampled inputs, never the whole input space, and so can confirm the presence of an incompatibility but never its absence. A complementary, recent direction uses large language models to *repair* breaking dependency updates [50] once detected, but presupposes that detection has already occurred and offers no machine-checkable guarantee that the repair preserves behaviour.

The decisive gap across all three strands is that compatibility is treated as an *ex post*, sampled, and largely syntactic property: a version string is consulted, a signature diff is computed, or a test suite is re-run after the fact. None grounds compatibility in a machine-checkable behavioural contract that covers all inputs, and none answers the question a developer actually poses at upgrade time—"which of my call sites does this change break, and why?"—with a proof obligation rather than a failed test. The pipeline proposed here closes this gap by making the formal contract the unit of compatibility. Because contracts compose, a callee-spec change mechanically recomputes each caller's proof obligations and flags the broken call sites *without running them*, turning behavioural-compatibility analysis into a static, exhaustive, counterexample-bearing operation. This builds directly on the compositional refinement established in our prior work [33], in which a second weakest-precondition pass recovers preconditions a single pass misses, and on its integration into continuous-integration workflows [34]. In this framing an internal refactor and a third-party version bump are the *same* operation—blast-radius analysis over a contract graph—and the foundational notion of an interface as a behavioural contract [69] is finally enforced rather than merely documented.

3.9 Legacy code comprehension and specification recovery

Most software in production is not written from scratch but inherited, and the reference pipeline proposed in this article must therefore confront brownfield codebases in which intent is implicit, distributed, and partly erroneous. The disciplines of legacy code comprehension and specification recovery have a long history of grappling with precisely this condition, and they supply both the conceptual vocabulary and the technical building blocks on which the present work draws. We group the relevant literature into four strands: the contractual stance that motivates recovering machine-checkable obligations at all; behaviour-preservation and characterisation practices for safely manipulating opaque code; dynamic and static specification mining; and the recent turn to learned, language-model–based recovery of formal contracts.

The contractual stance originates with Meyer's Design by Contract [69], which recast a software component's interface as a set of mutual obligations—preconditions, postconditions, and invariants—between caller and callee. This framing is foundational to the present article: the contract layer it proposes is exactly a set of such obligations, and the compositional propagation it inherits from Article 3 of this programme [33] is a direct consequence of contracts composing across method boundaries. Meyer assumed contracts would be authored by the developer up front, however, whereas the brownfield reality is that the contract must be *recovered* from code that predates any contractual discipline. The terminology for that recovery activity was rationalized by Chikofsky and Cross [21], whose taxonomy distinguishes reverse engineering, design recovery, and reengineering; their notion of *design recovery*—reconstructing higher-level abstractions from artefacts and domain knowledge beyond the code itself—anticipates the multi-source triangulation (code, tickets, documentation) that this article's startup phase performs.

The behaviour-preservation strand is exemplified by Feathers' influential treatment of legacy systems [37], which defines legacy code operationally as code lacking tests and introduces *characterisation tests*: tests that pin down what the code currently does rather than what it should do, establishing a regression baseline before any change. This is conceptually identical to what we call the characterisation phase, and

to the precise claim we make about what verification against a code-derived specification actually proves—namely a faithful description of current behaviour, not its correctness. The crucial difference is one of medium and coverage: a characterisation test fixes behaviour at sampled inputs, whereas a recovered formal contract checked by Extended Static Checking (ESC) constrains behaviour over *all* inputs, and disagreements between the recovered contract and independently sourced intent become candidate bugs rather than silently captured behaviour.

Specification mining sought to automate this recovery. Ammons et al. [6] coined the term and inferred finite-state API-protocol specifications from execution traces under the assumption that common behaviour is usually correct behaviour. In parallel, Ernst and colleagues' Daikon system [36] dynamically detected likely value invariants—ranges, ordering, linear relationships—over observed runs, and Nimmer and Ernst [76] integrated Daikon with ESC/Java so that dynamically proposed invariants could be statically confirmed. This propose-then-verify pairing is the closest antecedent to our pipeline's structure, yet it differs in two decisive ways. First, dynamic mining is bounded by test coverage and produces invariants that may be accidental, whereas our drafts are inferred statically from the abstract syntax tree by the JML-Inferrer and discharged by OpenJML over all inputs. Second, and more fundamentally, neither Daikon nor classical specification mining addresses *provenance*: a mined invariant is an undifferentiated guess, with no record of whether it reflects ratified human intent, documented behaviour, or merely incidental regularity in the traces.

The recent language-model strand attacks the natural-language gap that mining never could. Endres et al. [35] translate informal intent into formal postconditions capable of catching real Defects4J bugs; SpecGen [65] generates verifiable specifications through conversational prompting and mutation, outperforming Daikon and Houdini; and a broader line of work probes contract synthesis at scale [56]. These results validate the feasibility of the elaboration layer above this article's pivot, but they treat specification generation as a one-shot, self-graded translation, leaving open the very tensions this article foregrounds: the contract is taken as trusted output rather than ratified against pinned behavioural examples, the generating model is not held independent of the implementer, and a single assurance level is assumed rather than a tiered degradation that records what could only be tested or bounded-checked.

The gap this article addresses is thus the synthesis these strands have not achieved. Prior recovery work mines or generates specifications, but it does not (i) reconcile code-, ticket-, and documentation-derived intent into a single provenance- and assurance-tagged contract whose internal *disagreements* are surfaced as candidate bugs; (ii) treat the recovered contract as a non-trusted artefact that drives a Socratic, example-based ratification dialogue rather than gating on zero errors; (iii) enforce an independent-authority separation between the agent that elaborates the contract and the agent that satisfies it; or (iv) promote compositional contract propagation [33] into a lifecycle event that flags which callers a library or dependency change breaks behaviourally without executing them. The reference pipeline integrates legacy comprehension, machine-checkable contracts, and change-impact analysis into one verification-centric loop, with the recovered contract—not the disposable code—as the durable artefact.

3.10 Requirements and Intent Extraction, Traceability, and Natural-Language-to-Formal Translation

A pipeline that treats a machine-checkable contract as the durable source of truth must first answer where that contract comes from. In brownfield settings the contract is not authored from a blank page but *recovered* from three competing streams of evidence—code, issue tickets, and documentation—each of which a substantial literature has studied in isolation. This subsection surveys that literature thematically and positions the gap that the present article addresses.

Traceability link recovery. The foundational problem of relating informal artefacts to code was framed by Antoniol et al., who recovered links between source code and free-text documentation using information-

retrieval (IR) models on the premise that programmers embed meaningful names in identifiers [8]. De Lucia et al. extended this line into an integrated artefact-management environment using latent semantic indexing, demonstrating that IR-based recovery could be embedded in a working development tool rather than run as a one-off batch analysis [26]. A decade of such work was consolidated by Borg et al., whose systematic mapping catalogued the dominant vector-space and topic-model techniques and, tellingly, the persistent precision–recall ceiling that purely textual similarity imposes [14]. The field's research agenda was crystallized by Gotel et al.'s grand-challenge roadmap, which named ubiquitous, trustworthy, and *purposed* traceability as the long-horizon goal and observed that links created without a downstream consumer are rarely maintained [45]. More recent work has displaced shallow IR with learned semantics: Guo et al. applied recurrent architectures with domain word embeddings to inject artefact semantics into the tracing decision [47], and Lin et al. showed that pretrained BERT models (T-BERT) transfer effectively to the low-resource trace-link setting, materially improving link accuracy over IR baselines [61]. Across this body of work, however, a recovered link asserts only *relatedness*; it does not commit to any machine-checkable claim about behaviour, and so cannot by itself serve as a verification oracle.

Mining requirements and intent from issue trackers. Issue trackers are now the de facto repository of just-in-time requirements. Merten et al.'s grounded analysis of open-source trackers established that most requirements-relevant content—feature clarification, edge cases, rationale, and bug-driven constraints—is buried in unstructured discussion rather than in structured fields [68]. The scale and noisiness of this evidence stream are well documented by curated corpora such as the public Jira dataset of Montgomery et al., spanning millions of issues, comments, and inconsistently labelled links [72]. This work substantiates two assumptions of the present pipeline's characterisation phase: that tickets are a genuine source of *why*—the intent and known edge cases that code alone cannot reveal—and that linking tickets to code is inherently heuristic and noisy, credible only on a curated module rather than across a whole legacy application.

Documentation–code consistency. A parallel thread treats documentation not as a source to be mined but as a claim to be checked against code. Tan et al.'s `@tComment` inferred null/exception properties from Javadoc and generated tests to expose comment–code inconsistencies, surfacing defects in either the comment or the body [95]. Wen et al.'s large-scale study confirmed that such inconsistencies are common, persistent, and frequently co-located with bugs [99]. This is precisely the disagreement-detection value the present article elevates from a one-off analysis to a first-class lifecycle signal: a divergence between code and its documentation is a *candidate bug*, not an error to be silently reconciled.

Natural-language-to-formal translation. Bridging informal intent to a checkable artefact is the closest prior art to the elaboration layer. Cosler et al.'s `nl2spec` translated unstructured English to temporal logic with LLMs and—crucially—introduced an interactive loop in which sub-formulas are mapped back to natural-language fragments for human correction [24]. Endres et al. studied whether LLMs can turn natural-language intent into formal method postconditions, validating the generated contracts by their power to discriminate buggy from correct code and catching dozens of real Defects4J faults [35]. These results directly motivate the present pipeline—inference can remove the spec-authoring bootstrap cost—but they also expose the residual risk it must manage: LLM-produced formal text is sometimes wrong or vacuous, so it cannot be a trusted oracle absent independent authority and human ratification.

The gap. Each thread above optimises one edge of the intent problem—link recovery relates artefacts, issue mining extracts intent, consistency checking flags divergence, and NL-to-formal translation produces a candidate spec—but none composes them into a single, provenance-tagged, machine-checkable contract that serves simultaneously as the agent's input and its verification oracle. None reconciles the three evidence

streams into a contract whose clauses carry per-source provenance and graded assurance, nor treats inter-stream disagreement as the deliberate success outcome of a brownfield startup phase. Critically, prior NL-to-formal work validates a spec against a fixed code sample, whereas the present article verifies code against the recovered contract over *all* inputs via OpenJML extended static checking, and propagates a callee-spec change to recomputed caller obligations (building on Article 3 of this programme [33]). The contribution surveyed against here is therefore not a better link recovery or translation technique, but the architectural integration that turns recovered intent into a verifiable, compositional contract layer.

3.11 Synthesis: the gap this article addresses

The ten preceding subsections traverse a wide arc—from the deductive synthesis and refinement calculi that first treated a program as derivable from its specification [10, 67, 73], through the modern wave of large-language-model (LLM) code generation and its weak-oracle critique [18, 62, 108], the agentic and multi-agent frameworks that now drive repository-scale change [49, 63, 104], the design-by-contract and deductive-verification lineage that supplies a machine-checkable notion of correctness [23, 58, 69], the spec-inference and loop-invariant literature [36, 39, 54], the counterexample-guided synthesis and verified-codegen efforts that close a proof loop around the model [71, 91, 94], the test-oracle and circularity hazards of self-graded generation [13, 64], the behavioural-compatibility and breaking-change work that frames version evolution [41, 51, 74], to the requirements-extraction, traceability, and natural-language-to-formal translation that govern where a recovered contract comes from [8, 35, 68]. Rather than restate those surveys, this closing subsection draws the threads together and states precisely the gap the present article fills.

Each line of work optimises one edge of the problem the emerging spec-driven lifecycles leave open, and each does so in isolation. The verification community has long possessed a machine-checkable contract layer with independent authority over an implementation [23, 59, 69], but it presupposes that a human writes the formal contract by hand and says little about how an agentic lifecycle should obtain, ratify, or evolve that contract. The counterexample-guided synthesis tradition [3, 91] and its LLM-era descendants [71, 75, 94] close a proof loop around a generator, but typically against a contract that is fixed in advance or, worse, drafted by the same model that writes the code—reintroducing the self-grading circularity that the weak-oracle and oracle-generation critiques warn against [62, 64]. The natural-language-to-formal translators [24, 35, 65] produce a candidate contract from informal intent and even validate it against sampled buggy code, but they neither confront the ratification problem—that the human chose natural language precisely because they will not read formal text—nor verify the synthesized code against that contract over *all* inputs. The spec-mining and invariant-inference tools [6, 36, 89] recover likely properties from code or traces, but a property mined solely from an implementation describes what the code *does*, not what it *should* do, and so cannot by itself distinguish a faithful contract from an encoded bug. Finally, the breaking-change and behavioural-compatibility literature [50, 51, 74, 77] measures and sometimes repairs incompatibilities after the fact, but does not treat a contract change as a first-class lifecycle event whose blast radius can be computed without running a single client.

Crucially, no prior line of work combines these capabilities into a single reference development pipeline. The gap this article addresses is precisely that integration, along five dimensions that no existing system unites:

1. **A machine-checkable formal-contract layer with independent authority.** The contract against which code is verified is a set of provenance- and assurance-tagged obligations expressed in the Java Modeling Language (JML) and discharged over all inputs by OpenJML extended static checking [23, 58], and—unlike self-graded LLM pipelines [64]—it is authored by an elaborator role that never sees the implementation, so the verifier never grades its own author.

Table 1: Capability coverage of prior research lines against the five dimensions of the proposed reference pipeline. A check (✓) denotes substantial support; a dash (—) denotes that the line does not address the dimension. No prior line spans all five.

Prior line	Checkable contract	Behaviour ratification	Weakening detection	Multi-source characterisation	Compat. propagation
Deductive verification [23, 59]	✓	—	—	—	—
CEGIS / verified codegen [91, 94]	✓	—	✓	—	—
NL-to-formal [24, 35]	✓	—	—	—	—
Spec / invariant mining [6, 36]	—	—	—	✓	—
Traceability & intent mining [8, 68]	—	—	—	✓	—
Breaking-change analysis [51, 77]	—	—	—	—	✓
This article	✓	✓	✓	✓	✓

2. **Example/Socratic ratification that closes the ratification gap.** Rather than asking a human to confirm formal text they cannot read, the pipeline renders each contract as concrete checkable examples and surfaces only the boundary decisions it had to resolve, turning ratified examples into pinned oracles. The interactive NL-to-formal loops of [24, 35] map fragments back to language for correction but stop short of ratifying *behaviour* rather than *logic*.

3. **Adversarial spec-weakening detection.** A second, independent agent searches for scenarios consistent with the stated intent yet permitted by the proposed contract—the elaboration-layer dual of counterexample-guided inductive synthesis [3, 91]—repurposing vacuity and non-triviality checks (the author's prior "ensures true" punt detection [33]) as a weakening signal that no NL-to-formal translator currently emits.

4. **Brownfield multi-source characterisation with honest characterisation semantics.** The pipeline recovers a draft contract by triangulating three evidence streams—code, issue tickets, and documentation—building on traceability and intent-mining research [8, 68, 95] but, unlike that research, reconciling the streams into a per-clause provenance-tagged contract and treating inter-stream *disagreement* as the deliberate success outcome. It is also explicit that verifying code against a code-derived spec is characterisation, not proof of correctness—a precision the spec-mining literature [6, 36] rarely states.

5. **Compositional propagation for version compatibility.** Because contracts compose, a callee-spec change recomputes caller proof obligations and flags behaviourally broken call sites without execution, promoting the compositional refinement validated in Article 3 of this programme [33] into a lifecycle capability. Internal refactors and dependency version bumps become the same operation—a blast-radius analysis the breaking-change literature [50, 51, 77] measures empirically but does not derive from a verified contract.

These five capabilities are not novel in isolation; the contribution is their composition into a coherent reference architecture in which the formal contract is the pivot between the ambiguous human world and the precise machine world, simultaneously the agent's input and its verification oracle. The remainder of this article specifies that architecture and demonstrates its feasibility by instantiation over a real three-stream corpus, reusing the validated assets of the surrounding programme—inferred draft contracts [31], lossless artefact embedding [32], compositional refinement [33], and continuous-integration deployment [34]—as trusted components, while the trusted verifier never grades itself. Table 1 summarises which capability each major prior line supplies and where it stops short of the integrated pipeline.

4 Problem Analysis: Why Agentic Lifecycles Need a Verification Spine

The emerging generation of AI-native software lifecycles—typified by AWS AI-DLC [4], GitHub Spec Kit [44], and the Kiro spec-driven IDE [5]—share a common and genuinely attractive premise: that natural language (NL) specifications, rather than handwritten code, should be the primary artefact a developer authors, with AI agents responsible for translating those specifications into running implementations. This premise is supported by a strong empirical trend. Repository-level coding agents now resolve real issues against full codebases [53, 104, 107], and the broader position literature anticipates a corresponding restructuring of the software development lifecycle around agentic generation [63, 78]. The thesis of this article is that the premise is correct but its current realizations are structurally unsound: they elevate the specification to the role of source of truth while leaving the specification, and the loop that consumes it, in a form that cannot be machine-checked. This section analyses precisely why, isolates three named defects shared across these lifecycles, frames the economic inversion that makes the defects consequential, and motivates the reference architecture developed in the remainder of the article.

4.1 Three structural defects of current spec-driven lifecycles

Examined as a control system rather than as a developer experience, the AI-DLC, Spec Kit, and Kiro lifecycles exhibit three defects that compound one another. They are stated here as named problems because the architecture in section 5 is organised as a direct response to each.

Defect 1: the specification is natural language, so spec-to-code is an unverifiable compilation. In each of these lifecycles the durable input is a NL document—a prompt, a "spec" file, or a requirements note. The agent's job is to "compile" that document into code. But because the source language of this compilation is NL, there is no formal relation between source and target that any tool can check. A conventional compiler can be tested, and in principle verified, because both its input and output have a defined semantics; a NL-to-code agent has, on the input side, only the diffuse and underdetermined meaning of a human sentence. The consequence is that the central transformation of the lifecycle is, by construction, outside the reach of mechanical verification. This is qualitatively different from the situation in classical program synthesis, where the input is itself a formal contract and the synthesized program can be certified against it [67, 91].

Defect 2: the loop is open—generate then eyeball. Having produced code, these lifecycles return it to a human for inspection, or run a finite, human-selected test suite. There is no *machine gate* that admits an implementation only when it provably meets the intent. We call this the *open loop*: the feedback path that should close "code back onto specification" is completed, if at all, by human judgment or by sampled tests. The reliability literature has documented in detail how unreliable that closure is. Functional-correctness estimates from test suites are systematically optimistic because the suites are too weak to catch plausible-but-wrong solutions, and strengthening them lowers reported pass rates substantially [18, 62]. LLM-generated code is moreover prone to confident fabrication—non-existent APIs, intent conflicts, subtle off-by-one and boundary errors—that survives review precisely because the surrounding code is well-formed [108]. An open loop over a fallible generator is not a quality process; it is a presentation layer.

Defect 3: where verification exists, it is circular—self-grading. The few points at which these lifecycles do attempt automated acceptance rely on tests, and those tests are typically generated by the same model, in the same session, against the same NL understanding, that produced the code. The generator thereby writes its own oracle. Because the test author and the implementation author share a model and a training distribution, an error in the model's understanding of the intent is replicated identically in code and in oracle, and the two agree—wrongly—without contradiction. The hazard has been observed directly in LLM-driven

oracle and assertion generation, where the test and the code drift together away from the true intent [64, 97]. We call this the *self-grading* or circularity problem. It is the deepest of the three defects because it can make verification appear to succeed while certifying nothing.

These defects are not independent failures but a single missing component viewed from three angles. The missing component is a *verification spine*: a machine-checkable specification that is (a) related to code by a checkable relation, closing Defect 1; (b) the gate of a closed loop, closing Defect 2; and (c) owned independently of the implementation agent, closing Defect 3.

4.2 A motivating example: weakening "sort" to "is a permutation"

The defects above are easiest to see in a concrete failure. Consider a developer whose NL intent is "sort the array ascending." Suppose the agent is asked, as in current spec-driven flows, to record this intent as a formal contract and then to satisfy it. A faithful postcondition for an in-place sort over `int[] a` has two conjuncts: the result is sorted, and the result is a permutation of the input. Listing 2 shows the faithful contract beside a contract the agent might propose if it is permitted to author the oracle it will later be graded against.

```
//@ ensures (\forall int i; 0 <= i && i < a.length-1; a[i] <= a[i+1]);
//@ ensures isPerm(\old(a), \result); // result is a permutation of the input
int[] sort(int[] a) { ... }

//@ ensures isPerm(\old(a), \result); // "is a permutation" -- orderedness dropped
//@ assignable \nothing;
int[] sort(int[] a) { return a; }     // identity satisfies the weakened contract
```

Listing 2: A faithful sort contract (top) and a silently weakened contract (bottom). The weakened contract drops the orderedness conjunct; the identity function satisfies it.

Here `isPerm` denotes a user-defined `pure` model predicate, since JML provides no built-in permutation operator. The weakened contract retains only the permutation conjunct. It is still verifiable; indeed it is verifiable by the identity function, which returns the input unchanged (a permutation of itself). Under a circular pipeline the same model that drafted this contract then writes `return a`, the verifier reports success, and the lifecycle declares the task done. Nothing in the open loop catches the omission: the code provably satisfies the contract, the contract is plausible NL-adjacent text, and a human glancing at a green check has no reason to object. This is the self-grading defect (Defect 3) producing a spec-weakening pathology that no amount of code-side verification can detect, because the code is correct *relative to the weakened contract*. The error lives entirely in the specification, and is invisible unless the specification is itself adversarially scrutinized against the original intent—a mechanism we develop as adversarial elaboration in section 5.

A symmetric pathology arises in brownfield code. Consider a pre-existing method whose Javadoc states "returns the index of `key`, or -1 if absent," but whose body, through an off-by-one in a refactor, returns 0 rather than -1 on the empty array. Here the code and the documentation *disagree*, and that disagreement is exactly the bug. Inferring a contract from the code alone [33, 36] would faithfully encode `result == 0` for the empty case and verify it green—characterising the bug rather than finding it. The bug surfaces only when the code-derived clause is reconciled against the documentation-derived intent and the conflict is flagged. Both examples make the same point: verifying code against the wrong specification is not merely unhelpful, it is actively misleading, and a wrong specification verified against is worse than no specification at all.

4.3 The economic inversion: code is disposable, the specification is durable

The defects of section 4.1 have always existed in informal practice; what makes them urgent now is an inversion in the cost structure of software. When an implementation can be regenerated by an agent in

seconds and at negligible marginal cost, the implementation ceases to be the scarce, durable artefact it was for the preceding half-century of software engineering. Code becomes a *disposable build output*—a derivative that can be discarded and re-synthesized whenever requirements shift, the surrounding library set changes, or a faster model becomes available. The artefact that retains value across regenerations is the precise statement of *what the code must do*: the specification. This is the economic inversion, and it has a direct architectural corollary. A lifecycle that treats fluid, regenerable code as its source of truth and an unchecked NL note as a transient input has the durability gradient backwards. The lifecycle must instead be reorganised so that the machine-checkable specification is the persistent source of truth and code is the regenerable projection of it.

This inversion is what converts the three defects from chronic annoyances into load-bearing failures. If code were the durable artefact, an occasional spec-weakening or a stale comment would be a local blemish to be patched in the implementation. Once the specification is the durable artefact—the thing every regeneration is built against and graded by—a defect in the specification is inherited by every future implementation. The cost of an unverifiable, self-graded, open-loop specification is therefore no longer paid once; it compounds. The classical disciplines that anticipated this regime—Hoare logic and weakest-precondition reasoning [30, 48], Design by Contract [69, 70], and refinement calculus [10, 73]—already located correctness in the relation between a specification and an implementation rather than in the implementation alone. The contribution of this article is to make that relation the operational pivot of an AI-native lifecycle, using a contract language and verifier that are already mature for Java: JML [58] discharged by OpenJML extended static checking [22, 23] over the Z3 SMT solver [27].

4.4 The ratification gap: the central tension of the intent layer

Reorganising the lifecycle around a machine-checkable contract immediately raises a tension that, left unresolved, would reintroduce every defect it was meant to remove. We call it the *ratification gap*. The argument is short and unforgiving. If the formal contract is the oracle against which all code is graded, then the contract must be correct, and—because correctness is relative to human intent, which no tool can divine—the contract must ultimately be owned and confirmed by a human. Yet the human chose NL in the first place precisely because they cannot, or will not, author and read formal contracts. Asking that same human to confirm a page of quantified JML they cannot decode does not establish human authority over the oracle; it manufactures the *appearance* of authority while the human rubber-stamps text they do not understand. A pipeline built on fake ratification is no better grounded than the open loop it replaces: the self-grading agent is simply relabeled as "human-approved."

The ratification gap is the central design problem of the intent layer, and the architecture in section 5 is structured around refusing the false resolutions of it. It does not ask the human to read formal logic; instead it ratifies *behaviour, not logic*, rendering each contract as concrete, checkable examples—"given `x = -1`, this contract says the result is `0`; is that correct?"—so that the human exercises genuine authority over observable behaviour while the formal text remains the machine's concern. It surfaces, Socratically, only the boundary decisions the agent had to resolve (null and empty inputs, overflow, ordering, duplicates, concurrency, error-versus-exception), rather than burying the human in the full contract. And it enforces an *independent-authority constraint* so that the contract the code is graded against is never authored by the agent that writes the code. Table 2 summarises the correspondence between the defects analysed here and the mechanisms that answer them, each defined in the architecture that follows.

4.5 Why inference makes the reorganisation feasible

A final objection must be met before the architecture can be credible: if formal contracts are so expensive to author that developers reach for NL, does reorganising the lifecycle around them not simply relocate the cost

Table 2: The three defects of current agentic lifecycles, the named problems they give rise to, and the architectural mechanism (section 5) that responds to each.

Defect	Named problem	Responding mechanism
NL spec, unverifiable compilation	Unverifiable intent	Formal contract as pivot; verify all inputs
Generate-then-eyeball	Open loop	CEGIS-shaped synthesis–verification loop
Same model writes code + oracle	Self-grading / circularity	Independent-authority constraint
Human cannot read formal text	Ratification gap	Ratify behaviour, not logic (pinned oracles)
Spec weaker than intent	Spec weakening	Adversarial elaboration; vacuity checks
Code-derived spec only describes	Near-circular characterisation	Multi-source intent triangulation

rather than remove it? The answer, and the reason this programme is positioned to attempt the reorganisation, is that specification *inference* removes the authoring bootstrap. The author's prior work in this programme has established the necessary instrument and its lifecycle fit: inferred JML specifications materially improve LLM-based unit-test generation [31]; those specifications can be embedded into compiled artefacts and OpenAPI documents at full roundtrip fidelity [32]; heuristic inference admits compositional refinement, with a second weakest-precondition pass adding roughly 18,854 additional well-formed preconditions across the study corpus [33]; and the inference pipeline integrates into CI/CD with diff-only analysis, content-hash caching, and fail-open semantics [34]. Inference supplies a draft contract automatically, so the human's task collapses from *authoring* formal text to *ratifying* behaviour—exactly the operation the ratification gap demands be cheap. The compositional result is especially consequential for the lifecycle, because it means a change to one method's contract recomputes the proof obligations of its callers without re-running them, turning internal refactors and dependency version bumps into the same first-class, statically answerable question: which of my call sites does this change break? Section 5 develops the layered model that turns these capabilities into a verification spine, and section 10 reuses the inferrer, the OpenJML fork, and the Z3 oracle as trusted components of a proof-of-concept instantiation, so that the verifier never grades itself.

5 A Verification-Centric Reference Pipeline: Overview and Design Principles

The preceding survey establishes a precise gap: agentic, specification-driven lifecycles have solved the authoring-cost problem that crippled classical synthesis, but in restoring the cheapness of natural-language (NL) intent they discarded the two properties that made synthesis trustworthy—a machine-checkable contract and a closed, counterexample-driven acceptance loop. This section presents the spine of the article's response: a *verification-centric reference pipeline* that reintroduces a machine-checkable formal-specification contract layer and a verification loop into the AI-native lifecycle. The pipeline is offered as a reference architecture—a coherent set of design principles, a layered model, and an end-to-end data flow—validated by instantiation rather than asserted as a proven methodology. The subsequent sections detail each layer; the purpose here is to fix the terminology, the unit of work, and the constraints that the rest of the article respects.

5.1 The economic inversion and its consequence

The architecture rests on a single observation about the changed economics of software production. When an AI agent can regenerate a conforming implementation cheaply and repeatedly, the implementation ceases to be the scarce, durable artefact of the lifecycle and becomes a disposable build output, much as object code is a disposable output of a compiler. What remains scarce, and what must therefore be curated, versioned, and trusted, is the precise statement of what the software is required to do. The lifecycle must consequently

be reorganised around a *specification as the durable source of truth*, with code positioned downstream as a derived, regenerable artefact. This is not a rhetorical flourish: it is the load-bearing premise from which every design principle below follows. If the specification is the source of truth, then it must be both precise enough to drive synthesis and trustworthy enough to serve as an acceptance oracle—and those two demands, in tension, generate the architecture.

5.2 Design principles

Five principles govern the pipeline. They are stated here in summary and elaborated, with their attendant mechanisms, throughout the article.

The formal contract as pivot. A machine-checkable formal contract sits at the centre of the architecture as the *pivot* between the ambiguous human world and the precise machine world. Above the pivot lies the inherently informal activity of deciding what is wanted; below it lies the mechanically checkable activity of producing and certifying code. The contract is the single point at which informality is converted into formality, and it is the only artefact that crosses that boundary. Concretely, the pivot is expressed in the Java Modeling Language (JML) [58], the behavioural interface specification language whose pre/postcondition discipline descends directly from Hoare logic [48], Dijkstra's weakest-precondition calculus [30], and Meyer's Design by Contract [69].

The dual role of the specification. The contract plays two roles simultaneously. It is the *input*—the precise intent the synthesis agent builds against—and it is the *oracle*—the contract against which the agent's output is formally verified. This duality is what makes the pipeline self-consistent: there is exactly one statement of intent, and code is judged against the very artefact that produced it. The historical obstacle to this arrangement was the cost of authoring formal contracts; the pipeline removes that bootstrap cost by drawing on automated inference to supply draft contracts, building directly on the author's prior work showing that heuristic inference produces specifications useful to downstream LLM tasks [31, 33].

Independent authority. Because the contract is the oracle, verification is meaningful only if the contract is not authored by the same agent (instance or role) that writes the code. If a single model both proposes the contract and satisfies it, the verifier still soundly proves that the code meets the contract, but that proof certifies nothing about *intent*: a shared misreading is encoded in both the contract and the code, which then agree without exposing it. This is the self-grading circularity that undermines test-suite-based acceptance in current agentic systems and, more acutely, LLM-driven oracle generation [64]. The *independent authority constraint* therefore partitions the lifecycle into an elaborator role—maximally faithful and strict, never permitted to see the implementation—and an implementer role, whose only task is to satisfy the contract handed to it. The trusted verifier, OpenJML extended static checking discharged by the Z3 SMT solver [23, 27], never grades its own output.

Tiered assurance and graceful degradation. Not all intent is formally specifiable, and even formalizable obligations do not always verify: deductive verification rests on SMT solving, which is undecidable in general and times out in practice on multiplicative reasoning, array theory, and richly quantified preconditions. The architecture therefore does not treat "verified" as binary. It defines a ladder of assurance—full proof, then bounded checking, then testing—and *degrades* an obligation down this ladder when the level above does not terminate, always *recording* the level achieved and never silently dropping the obligation. Graceful degradation is what allows a verification-centric pipeline to remain usable on real code rather than only on the fragment that the solver happens to discharge.

Table 3: The two tags carried by every obligation. Provenance records the origin and thus the authority of an obligation; assurance records the strongest verification level achieved. Together they make a "verified" claim auditable and the independent-authority constraint enforceable.

Tag	Value	Meaning
Provenance	`human-ratified`	confirmed by a human via a behavioural example
	`example-pinned`	a concrete case the contract must satisfy
	`inherited-default`	from an organisation/codebase policy contract
	`ticket-stated`	extracted from an issue tracker
	`doc-stated`	extracted from documentation or Javadoc
	`code-inferred`	inferred from the implementation by heuristics
	`inferred-unconfirmed`	machine-derived, awaiting human adjudication
Assurance	`proved`	discharged by OpenJML extended static checking
	`bounded-checked`	verified up to a finite bound
	`tested`	exercised by generated/existing tests only
	`unchecked`	recorded but not yet verified at any level

Provenance-tagged obligations. Finally, the architecture makes the independent-authority constraint and the tiered-assurance record *auditable* by attaching, to every obligation, a tag recording where it came from and how strongly it has been checked. This is the mechanism that keeps the word "verified" honest: a reader of a verification report can see which obligations are human-owned, which are machine-guessed, and which were proved as opposed to merely tested. The obligation, so tagged, is the architecture's unit of work, and the next subsection defines it precisely.

5.3 The obligation as the unit

A contract in this architecture is not a monolithic block of formal text but a *set of obligations*, each an individually addressable proof goal (a precondition, a postcondition, a loop invariant, a class invariant, or an assignable clause) carrying two orthogonal tags. The first, *provenance*, records the obligation's origin and hence its claim to authority; the second, *assurance*, records the strongest verification level the toolchain has achieved for it. Table 3 enumerates the admissible values.

The two tags do real architectural work. Provenance operationalizes the independent-authority constraint: an obligation that the code is verified against must trace to an authority other than the implementer—ideally `human-ratified` or `inherited-default`—and an obligation tagged `code-inferred` or `inferred-unconfirmed` is understood to describe what the code *does*, not what it *should* do, and therefore carries no independent authority to certify that same code. Assurance operationalizes tiered degradation: when full proof is unavailable, the obligation is not discarded but retagged `bounded-checked` or `tested`, preserving a truthful account of what is and is not known. Taken together, the tags convert "the system verified the code" from an opaque assertion into a structured, inspectable claim—which obligations, from which authorities, were established to which level.

5.4 The layered model

The pipeline organizes its activity into layers stratified by the pivot. Figure 1 depicts the arrangement. Above the pivot sits the *intent* layer and the *elaboration* layer, which together convert human-owned, ambiguous intent into a formal contract; below the pivot sit the *synthesis* layer, which produces code from the contract, and the *verification* layer, which certifies code against it.

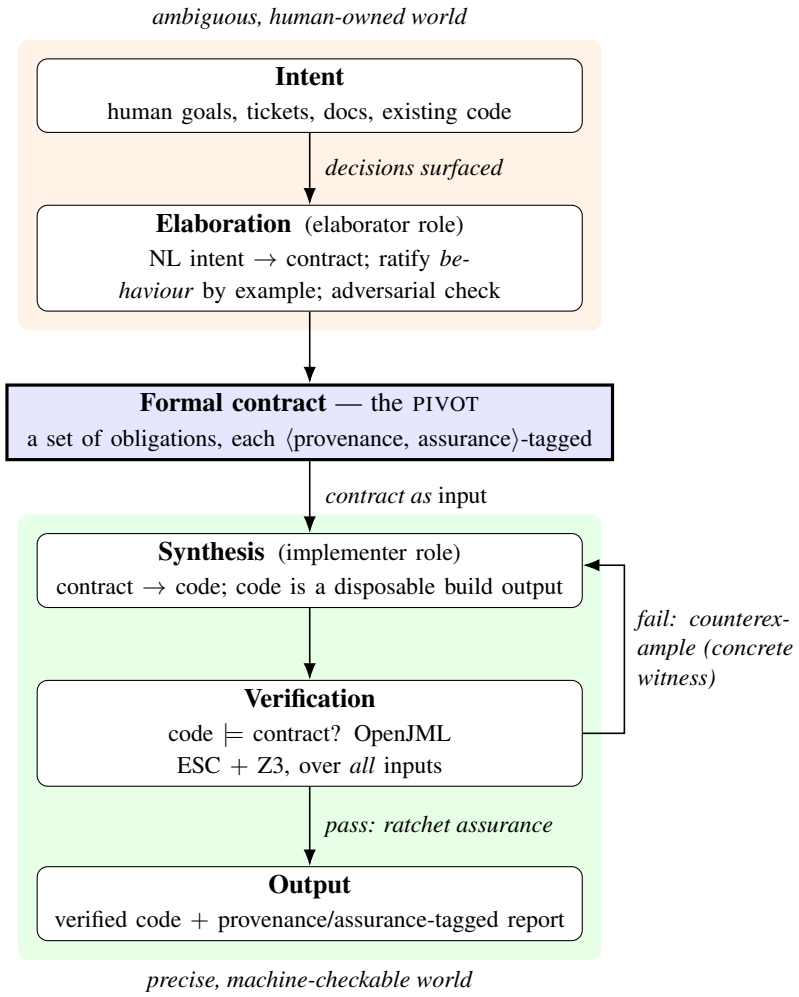

Figure 1: The layered reference architecture. The formal contract is the pivot between the ambiguous, human-owned world (above) and the precise, machine-checkable world (below). Elaboration converts natural-language intent into a contract under human ratification of behaviour; synthesis and verification form a counterexample-guided loop below the pivot, and the verifier never grades a contract it authored.

The stratification is deliberate and load-bearing. Each layer answers a distinct question and is subject to distinct correctness criteria, and the pivot is the only place where an artefact changes ontological status from informal to formal.

The *intent* and *elaboration* layers, above the pivot, own the problem that informal intent must be rendered as a precise contract without forfeiting human authority over what is wanted. Their central difficulty—the *ratification gap*—is that the human used NL precisely because they cannot or will not author formal contracts, so asking them to confirm formal text they cannot read makes their authority illusory. The architecture's response, developed in the layer's dedicated section, is to never ask the human to confirm logic: the contract is rendered as concrete behavioural examples ("given $x = -1$, this contract yields $result = 0$; is that correct?"), and ratified examples become *pinned oracles* the contract must satisfy. Elaboration is further made trustworthy by surfacing the boundary decisions the agent had to resolve—null and empty inputs, overflow, ordering, duplicates, concurrency, error-versus-exception—and by an adversarial second agent that searches for scenarios consistent with the NL intent yet violating the proposed contract, detecting specification weakening before the contract is trusted.

The *synthesis* and *verification* layers, below the pivot, own the mechanical problem of producing code and certifying it. Their loop is counterexample-guided in the manner of CEGIS [91]: the implementer emits code, OpenJML attempts to verify that the code models the contract over *all* inputs rather than a sample, and on failure the concrete counterexample witness—a far richer signal than "a test failed"—is returned to the implementer to drive regeneration, capped at a bounded number of iterations, after which the obligation is degraded and reported per the assurance ladder. Because the two roles are separated by the independent-authority constraint, the verifier never certifies an artefact whose contract it also authored.

5.5 End-to-end data flow

The architecture supports two entry modes that share the same layers but differ in how the contract first comes into being. In the *greenfield* mode, intent originates with a human and flows top-down: NL intent enters the elaboration layer, is converted to a contract under behavioural ratification, crosses the pivot, and drives the synthesis–verification loop. In the *brownfield* mode—the common case for real software—no contract exists and the existing code, documentation, and issue tickets must be mined to *recover* one. A startup characterisation phase, detailed in its own section, runs before any user input: it infers a draft contract from the code, extracts intent claims from tickets and documentation, and reconciles these three evidence streams into a candidate contract with per-clause provenance, flagging the disagreements between streams as candidate bugs. Crucially, verifying code against a code-derived contract is *characterisation*, not a correctness proof—it is nearly circular and establishes only a faithful-description baseline plus toolchain sanity; genuine correctness-relative-to-intent arises only where the ticket- and doc-derived streams disagree with the code and a human ratifies the resolution. The brownfield mode thus reorders the loop: a first, *retroactive* pass resolves recovered discrepancies and establishes a trusted baseline, after which *prospective* passes handle new changes.

A single further capability spans both modes and connects the architecture to the thesis programme's core concern with library and version compatibility. Because the contract is a composition of obligations, a change to one method's contract mechanically recomputes the proof obligations of its callers and flags which callers break *without running them*—and an internal refactor and a dependency version bump are, from the verifier's standpoint, the same operation. The author's prior compositional refinement result [33] is thereby promoted from an inference technique to a first-class lifecycle event: the question "which of my call sites does this upgrade break behaviourally?" becomes a blast-radius analysis the pipeline answers from contracts alone, a capability the broader literature on behavioural breaking changes has long sought [51, 74].

5.6 Reuse of validated components

The architecture is realizable today because its trusted core already exists and has been independently validated. The proof-of-concept instantiation, presented later, reuses the JML-Inferrer to bootstrap draft contracts [31, 33], the custom OpenJML fork with Z3 as the trusted oracle [23, 27], the artefact-embedding machinery for carrying specifications alongside code [32], and the Dockerized verification pipeline already integrated into continuous integration [34]. The only new software is an orchestrator together with LLM calls for elicitation, contract drafting, and code synthesis; the verifier that grades the result is a pre-existing, independent component that never grades itself. This separation between newly written, untrusted orchestration and reused, trusted verification is the architecture's most important engineering commitment, and it is what permits the article to claim feasibility and realism for the reference pipeline while reserving full empirical validation on external production codebases for future work. The sections that follow take each layer in turn, beginning with the intent and elaboration layer and its ratification gap.

6 The Intent Elaboration Layer

The layered model developed earlier in this article places a machine-checkable formal contract at the pivot between the ambiguous human world and the precise machine world. Everything below that pivot—synthesis of code from the contract and verification of code against it—can in principle be made sound, because both the agent and the verifier operate over the same formal object. Everything above the pivot is harder, because it must carry intent across the boundary between a human who reasons informally and a machine that reasons formally, and it must do so without quietly substituting the machine's reading of intent for the human's. This is the work of the *intent elaboration layer*: the process by which natural-language (NL) intent becomes a formal contract that the human accepts as a faithful statement of what they meant. It is the hardest layer in the pipeline precisely because its correctness cannot be delegated to a solver. Z3 can prove that code satisfies a contract, but no solver can certify that the contract says what the human wanted; that judgment is irreducibly human, and the entire assurance argument of the pipeline rests on it. This section defines the central tension of the layer, develops the mechanisms that resolve it, gives the in-layer elaboration loop as an algorithm, and states the layer's named research contribution.

6.1 The Ratification Gap

The dual role of the specification, established earlier, is that the contract is simultaneously the agent's *input*—the precise intent it builds against—and its *oracle*—the property its output is verified against. As an oracle the contract is load-bearing in a way that a test suite never is: verification covers all inputs, not a sample, so a contract that misstates intent will be discharged confidently against code that does the wrong thing. A wrong specification verified against is therefore worse than no specification at all, because it converts the absence of a counterexample into false assurance. It follows that the contract must be correct with respect to intent and must be owned by the human, since only the human is the authority on what was meant.

Here the layer encounters its defining obstacle, which we term the *ratification gap*. The human supplied intent in natural language for a reason: they cannot, or will not, write formal contracts. Asking that same human to confirm formal text—to read a quantified JML postcondition and attest that it captures their intent—makes the human's authority a fiction. They will sign whatever they are shown, because they cannot evaluate it, and the signature certifies nothing. The naive elaboration loop, in which an LLM drafts a contract and the human is asked "is this correct?", thus produces the appearance of human ownership while transferring real authority to the model that drafted the text. Because that model is the same kind of system whose output the contract is meant to police, the loop is circular in the most damaging way: the artefact that is supposed to ground the pipeline's assurance is itself ungrounded. Resolving the ratification gap without abandoning either human ownership or formal precision is the problem the remaining mechanisms of this layer are designed to solve.

6.2 Ratify Behaviour, Not Logic

The first and most important mechanism is a discipline: *ratify behaviour, not logic*. The human is never asked to confirm formal text. Instead the candidate contract is rendered into concrete, checkable behavioural claims that any competent reader of the problem domain can evaluate without reading a line of JML. A contract clause is exercised at representative and boundary inputs, and the resulting input–output pairs are presented as questions in the human's own terms: "given an input of -1, this contract says the result is 0; is that correct?"; "given an empty list, this contract says an exception is thrown rather than zero returned; is that what you want?". The human reasons about behaviour, which is what they understand, rather than about the predicate-logic encoding of behaviour, which they do not.

Each behavioural claim the human ratifies becomes a *pinned oracle*: a concrete input–output (or input–exception) pair that the contract is henceforth required to satisfy, recorded independently of the formal text.

Table 4: The boundary-decision catalogue. For each method, the elaborator determines whether NL intent settles each category; unresolved categories are surfaced to the human as behavioural questions.

Category	Representative question
Null inputs	Reject `null` arguments, or treat as a valid value?
Empty inputs	Empty collection/string: return identity, or error?
Overflow	Saturate, wrap, or throw on arithmetic overflow?
Ordering	Is output order specified, or unconstrained?
Duplicates	Are duplicate elements removed, preserved, or rejected?
Concurrency	Is the method required to be thread-safe?
Error vs. exception	Signal failure by return value (sentinel) or by exception?

Pinned oracles serve three functions. They are the operational definition of human ratification, replacing an unverifiable signature on formal text with a verifiable set of agreed cases. They constrain the formal contract directly, because any subsequent reformulation—whether by the elaborator or by a later compositional pass—must continue to satisfy every pinned oracle, and a reformulation that does not is rejected mechanically rather than silently accepted. And they are themselves machine-checkable: the pinned oracle "$f(-1) = 0$" can be discharged against the formal contract without human involvement, turning ratification into a persistent, re-checkable artefact rather than a one-time conversation. Example-grounded ratification is, in effect, the elaboration-layer analogue of programming by example [46]: concrete instances anchor an otherwise underdetermined formal object. The difference is that here the examples do not synthesize the contract—inference and the LLM draft do that—but *ratify* it, supplying the human authority that synthesis alone cannot. Pinned oracles are, however, necessarily a finite sample, so they do not by themselves establish human authority over the contract's behaviour on *every* input; what they establish is authority over a strategically chosen, catalogue- and adversarially-guided set of boundary cases, while the contract's universal force on unpinned inputs is borne by the formal generalisation and the adversarial search. This is a different posture from the sampled-oracle weakness criticised earlier: there the unsampled region is simply unverified, whereas here a finite sample *anchors* a universal contract that is then verified over all inputs, so the residual risk is contract-misstatement on unpinned inputs—a narrower hazard than test-suite coverage gaps, though not an eliminated one.

6.3 Socratic, Decision-Surfacing Elaboration

Behavioural ratification answers how to confirm a contract; it does not by itself decide which questions to ask. Natural-language intent is radically underdetermined at exactly the points where defects concentrate, and a faithful elaborator must resolve those points to produce any contract at all. Rather than resolve them silently and present a finished contract, the elaborator practices *Socratic*, decision-surfacing elaboration: it makes explicit the ambiguities it was forced to resolve and asks the human only about those. The premise is that the human's scarce attention should be spent on genuine decisions, not on confirming the obvious.

The decisions worth surfacing are not arbitrary; they fall into a small, recurring *boundary-decision catalogue* that the elaborator walks for each method. Table 4 lists the categories. For each one the elaborator determines whether the NL intent settles the decision; where it does not, the unresolved decision is surfaced as a behavioural question in the sense of section 6.2. This converts an open-ended "is this contract right?" interrogation, which the human cannot answer, into a bounded sequence of concrete adjudications, each of which the human can. It also makes the elaboration auditable: the catalogue is a checklist, and a contract can be reported as having every catalogue entry either resolved by stated intent, resolved by an inherited default (section 6.6), or ratified by the human.

6.4 Adversarial Elaboration and Vacuity Checks

Surfacing decisions and ratifying behaviour guard against a contract that omits intent, but they do not guard against a contract that is technically consistent with the stated intent yet too weak to be useful. An LLM asked to formalise "sort the list" may produce a postcondition asserting only that the output is a permutation of the input—true of the sorting it was asked for, but also satisfied by the identity function. The drafting model has, perhaps inadvertently, weakened the specification to one its own implementation can trivially meet. Because the same family of models drafts the contract and synthesizes the code, this failure mode is systematic rather than accidental, and behavioural ratification will not always catch it: a human who agrees that "the output is a permutation of the input" is a true statement may not notice that it is an *incomplete* one.

The mechanism that addresses this is *adversarial elaboration*, the elaboration-layer dual of counterexample-guided inductive synthesis (CEGIS) [91]. Where CEGIS pits a synthesizer against a verifier that searches for inputs on which the candidate program fails, adversarial elaboration pits the contract drafter against a second, independent agent that searches for *scenarios consistent with the NL intent but violating the proposed contract's evident purpose*—concretely, behaviours a reasonable reading of the intent would forbid that the drafted contract nonetheless permits. For the sorting example, the adversary exhibits the identity function as a witness that satisfies the permutation-only contract while plainly contradicting "sort", signaling that the postcondition must be strengthened with an orderedness clause. The adversary's witnesses are themselves behavioural and are fed back into the ratification dialogue, so spec weakening is surfaced as a concrete "the contract currently permits this; did you intend to permit it?" question.

Adversarial elaboration is complemented by automatic *vacuity and non-triviality checks*, which detect the degenerate cases of weakening without an adversary in the loop. Two patterns are flagged: a postcondition that is logically equivalent to `true`, which constrains nothing, and a precondition that is unsatisfiable, which makes the contract vacuously dischargeable because no input ever reaches it. This repurposes a signal already present in our prior inference work, where the inferrer emits an `ensures true` clause as an honest *punt*—an explicit admission that it could not infer a meaningful postcondition for a method [31]. In the elaboration layer the same syntactic signal is reinterpreted as a weakening alarm: an `ensures true` obligation, or any clause provably equivalent to it, marks a method whose contract carries no behavioural content and must either be strengthened or be explicitly accepted as unconstrained. Listing 3 illustrates the two patterns.

```
//@ ensures true;                    // flagged: no behavioural content
public int normalise(int x) { ... }

//@ requires n > 0 && n < 0;         // flagged: unsatisfiable -> vacuous
//@ ensures \result == n;
public int echo(int n) { ... }
```

Listing 3: Vacuity patterns flagged by non-triviality checks: a postcondition equivalent to `true` (no constraint) and an unsatisfiable precondition (vacuously dischargeable).

6.5 The Independent Authority Constraint

Adversarial elaboration already requires a second agent, and this reflects a constraint that governs the whole layer: the *independent authority constraint*. The contract that code is verified against must not be authored by the same agent—the same model instance, and more importantly the same role—that writes the code. If a single agent drafts both the oracle and the implementation, then a misreading of intent corrupts both identically, the implementation satisfies the corrupted oracle, and verification certifies the misreading. This is the self-grading circularity that the survey of prior LLM verification work identified as its recurring structural weakness, and it is the reason the layer enforces a role separation rather than merely recommending it.

The separation assigns two distinct roles with deliberately opposed objectives. The *elaborator* role is maximally faithful and maximally strict: its objective is to capture the human's intent as precisely as

possible, it conducts the Socratic dialogue and owns the pinned oracles, and—critically—it never sees the implementation. The *implementer* role has the narrow objective of satisfying the contract handed to it; it does not negotiate the contract and has no incentive to weaken it, because it never authored it. Because the elaborator is blind to the code, it cannot tailor the oracle to excuse a particular implementation, and because the implementer cannot edit the contract, it cannot relax the oracle to admit a deficient implementation. The verifier, reused unchanged from our prior work as a trusted component, grades the implementer against the elaborator's contract and is authored by neither. The independent authority constraint removes *self-grading* circularity—an agent cannot tailor the oracle to excuse its own code—but it does not by itself decorrelate a shared *misreading of intent* across two roles of the same underlying model, which can encode the same error in both contract and code. That residual is broken not by role separation but by human ratification, whose authority lies outside the model distribution, and is further reduced by drawing the adversarial elaborator from a different model or prompt lineage. The constraint is thus necessary but not sufficient; the provenance tagging introduced next is what makes adherence to it auditable rather than merely asserted.

6.6 Provenance, Assurance, and Inherited Default Contracts

A contract in this pipeline is not a monolithic block of formal text but a *set of obligations*, each carrying two tags. The *provenance* tag records where the obligation came from: `human-ratified`, `inherited-default`, `inferred-unconfirmed`, `example-pinned`, `ticket-stated`, `doc-stated`, or `code-inferred`. The *assurance* tag records how strongly it has been established against the implementation: `proved`, `bounded-checked`, `tested`, or `unchecked`. Provenance makes the independent authority constraint auditable, because one can mechanically confirm that every obligation the implementer was graded against carries a provenance other than the implementer's own authorship. Assurance makes the pipeline's "verified" claim honest: an obligation that timed out in the solver and fell back to bounded checking is recorded as `bounded-checked`, never silently promoted to `proved`. The two tags together turn a contract into a self-describing assurance record rather than an opaque oracle of unknown trustworthiness.

The provenance value `inherited-default` names the most team- and process-flavored artefact of the layer: *inherited default contracts*. These are organisation- or codebase-level policy obligations that individual elaborations inherit automatically—for example, "public methods reject `null` arguments unless explicitly stated otherwise," "monetary values are never negative," or "returned collections are never `null`." Such policies encode decisions an organisation has already made once and does not wish to re-adjudicate per method. They sharply reduce the ratification burden, because the elaborator asks the human only about *departures* from policy rather than re-litigating every boundary decision in the catalogue of section 6.3; a method that conforms to the null-rejection default needs no null question at all. Inherited defaults also make ratification labor scale with the genuinely novel content of each method, which is the practical answer to the objection that example-grounded ratification is too costly to apply across a large codebase. Where a method must depart from a default, the departure is itself surfaced as a behavioural question and, once ratified, retags the affected obligation from `inherited-default` to `human-ratified`.

6.7 The In-Layer Elaboration Loop

The mechanisms above combine into a single elaboration loop, given as the numbered procedure below. The loop takes the NL intent and the applicable inherited defaults and terminates with a contract every clause of which is provenance-tagged and either resolved by stated intent, resolved by an inherited default, or ratified through a behavioural example. The adversarial and vacuity checks run before the human is consulted, so the human adjudicates a contract that has already been hardened against the most common weakening failures; the human's questions therefore concern genuine intent decisions rather than drafting defects.

1. **Seed.** Obtain a draft contract: for greenfield methods, an LLM few-shot draft from the NL intent; for

brownfield methods, the inferrer's recovered draft (section 6 situates this within the broader characterisation phase). Attach inherited default obligations applicable to the method's scope.

2. **Surface decisions.** Walk the boundary-decision catalogue (table 4); mark each category as resolved-by-intent, resolved-by-default, or unresolved.

3. **Adversarial check.** Invoke the independent adversary to seek a behaviour consistent with the NL intent but permitted by the draft contract that a reasonable reading would forbid. If a witness is found, record it as a candidate weakening and add the implied strengthening to the agenda.

4. **Vacuity check.** Flag any obligation equivalent to `ensures true` or guarded by an unsatisfiable precondition; add each flag to the agenda.

5. **Behavioural ratification.** For every unresolved decision, candidate weakening, and vacuity flag, render the relevant contract behaviour as a concrete input–output example and ask the human in domain terms. Record each ratified answer as a pinned oracle and retag the affected obligation's provenance to `human-ratified`.

6. **Reconcile and re-check.** Reformulate the contract to satisfy all pinned oracles and ratified strengthenings. Verify mechanically that every pinned oracle is discharged by the reformulated contract; if any is not, return to step 3 with the violated oracle as a new agenda item.

7. **Terminate.** When the agenda is empty and all pinned oracles are discharged, emit the provenance-tagged contract for handoff across the pivot to the synthesis and verification layers.

The loop is bounded in practice by the size of the boundary-decision catalogue and the inherited defaults, which together cap the number of questions a single method's elaboration can generate. It is also resumable: because pinned oracles persist independently of the formal text, a later compositional pass that recomputes a caller's obligations re-enters the loop only at step 6, re-checking the reformulated contract against the already-ratified oracles rather than re-interrogating the human.

6.8 Research Contribution of the Layer

The named research contribution of the intent elaboration layer is a *ratification protocol that establishes genuine human authority over a formal contract without requiring the human to read formal text*. Its components are individually defensible and jointly novel: behavioural ratification through pinned oracles replaces an unverifiable signature with a re-checkable set of agreed cases; Socratic, catalogue-driven elaboration bounds the human's attention to genuine boundary decisions; adversarial elaboration and vacuity checks, repurposing the inferrer's `ensures true` punt [31] as a weakening alarm, harden the contract against the spec-weakening to which a self-drafting model is prone; the independent authority constraint forbids the implementer from authoring its own oracle; and provenance- and assurance-tagged obligations, refined by inherited organisational defaults, make both the independence of the authority and the honesty of the "verified" claim auditable. Together these resolve the ratification gap that prior LLM-based specification work leaves open, in which an inferred or NL-derived contract is treated as ground truth despite never having been genuinely confirmed by the human whose intent it claims to encode. The remaining sections take the contract this layer produces as given and develop the synthesis and verification machinery below the pivot.

7 Brownfield Characterisation and Multi-Source Intent Triangulation

The reference pipeline described in the preceding sections presumes that a machine-checkable contract exists before an agent writes code, so that the contract can act simultaneously as the synthesizing agent's input and

as the oracle against which its output is verified. For a new method written from scratch, that contract is *elaborated*: natural-language (NL) intent is lifted across the formal pivot through a human-confirmed dialogue. The overwhelming majority of real software, however, is not new. It is a pre-existing codebase carrying years of accreted behaviour, an issue tracker recording the bugs and edge cases that behaviour has provoked, and documentation declaring what the behaviour was intended to be. In this brownfield setting the contract cannot be authored; it must be *recovered*. This section develops the startup phase that performs that recovery before any user input is solicited, and it is at pains to state precisely—and without overclaiming—what the verification performed during that phase does and does not establish.

The defining difficulty of the brownfield case is that the existing code occupies two roles at once. It is the single best evidence of what the system actually does, and it is simultaneously the prime suspect, because whatever bugs the system contains are encoded in exactly that code. A contract inferred from the code alone therefore describes the implementation faithfully, including its defects. Verifying the implementation against such a contract is consequently *characterisation* rather than validation, and the remainder of this section is organised around that distinction: section 7.1 describes the three evidence streams and the disagreement detection that is the real source of value; section 7.2 states what the verification proves; section 7.3 defines the characterisation report and the retroactive-then-prospective loop reordering; and section 7.4 records the honest limitations.

7.1 Three Evidence Streams and Disagreement Detection

The startup phase ingests three artefact families and treats each as an independent stream of evidence about intent (fig. 2). The first stream is the *code* itself, which answers the question of *what the system does*. From it the JML-Inferrer of our prior work [31, 33] derives a draft contract by abstract-syntax-tree heuristics—preconditions from parameter null checks and bounds guards, postconditions from return-value patterns, loop invariants for the max/min, linear-search, and summation shapes the tool already handles, and class invariants and assignable clauses—and the compositional second pass of Article 3 [33] strengthens that draft with the additional preconditions a single pass misses. The second stream is the *issue tracker*, typically Jira, which answers the question of *why*: it records reported defects, the edge cases users actually exercised, and the rationale for past changes. The third stream is the *documentation*, principally Javadoc but also design notes and API references, which answers the question of *declared intent*—what the authors claimed the code should do. Intent claims are extracted from the latter two streams by an LLM augmented with retrieval over the relevant artefacts, building on the demonstrated capability of models to translate code-adjacent NL into formal postconditions [35, 65] and, more broadly, into formal contracts [24, 56].

These three streams are then reconciled into a single candidate contract in which every clause carries a *provenance* tag recording the stream it came from: `code-inferred`, `ticket-stated`, or `doc-stated`, alongside the elaboration-time tags (`human-ratified`, `inherited-default`, `inferred-unconfirmed`, `example-pinned`) introduced earlier in the pipeline. The reconciliation is deliberately not a merge that smooths away conflict. Its primary product is *disagreement detection*: the points at which the three streams make incompatible claims about the same clause. Recovering a clean, consistent contract is the unremarkable case; the engineering value of triangulation lies in the inconsistent cases, because each one is a hypothesis that something is wrong—either the code, or the documentation, or the recorded understanding of the system. This reframing of conflict-as-signal mirrors the long tradition of detecting comment–code inconsistency [95, 99] and of mining specifications from heterogeneous sources [6, 36], but it elevates the inconsistency itself, rather than any one reconciled artefact, to the role of primary output.

Table 5 gives a small worked example drawn from the kind of method the demonstration corpus contains. Each row is a single recovered clause, shown beside what the code implies, what the documentation states, and what the issue tracker records, together with the verdict the triangulation assigns. The rows illustrate the three outcomes the phase can reach: agreement across streams (a clause the recovery is confident in),

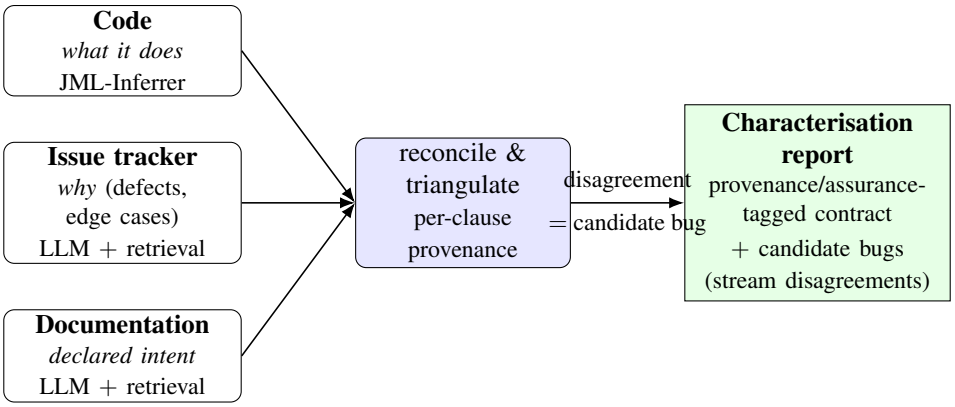

Figure 2: Multi-source intent triangulation in the brownfield startup phase. Three evidence streams—code (*what it does*), the issue tracker (*why*), and documentation (*declared intent*)—are reconciled into a single provenance-tagged candidate contract. The primary product is not the merged contract but the *disagreements* between streams, each a hypothesis that the code, the documentation, or the recorded understanding is wrong.

Table 5: Worked example of multi-source intent triangulation for a string-trimming utility, in the style of an Apache Commons Lang `StringUtils` method. Each clause is recovered independently from the three streams; the verdict records whether they agree, and a disagreement is recorded as a candidate bug rather than suppressed.

Clause	**Code-inferred**	**Doc-stated**	**Ticket-stated**	**Verdict**
Null input handling	`\result == null` on null input	returns `null` for null input	(no mention)	*Agreement*: confidence clause
Empty-string result	`\result.isEmpty()` when input is blank	returns the empty string for blank input	(no mention)	*Agreement*: confidence clause
Unicode whitespace	trims only `c <= 0x20` (ASCII)	"removes whitespace" (unqualified)	LANG-ticket requests Unicode-aware trimming	*Disagreement*: candidate bug
Idempotence	`trim(trim(s)) == trim(s)` holds	(no mention)	(no mention)	*Unconfirmed*: drives dialogue

silence in the intent streams (a code-inferred clause that no human source confirms or denies, which therefore remains unconfirmed and drives the dialogue), and outright disagreement (a candidate bug).

7.2 What the Startup-Phase Verification Actually Proves

It is essential to be exact about the epistemic status of any verification performed during characterisation, because the phrase "the code verified" invites a far stronger reading than is warranted. When the recovered contract is dominated by `code-inferred` clauses, verifying the code against that contract is *nearly circular*: the specification was derived from the very implementation it is now used to grade, so a successful proof establishes only that the implementation is self-consistent and that the toolchain—the JML-Inferrer, `AnnotationToJMLConverter`, OpenJML, and Z3—is operating correctly [23, 27]. This is a genuine and useful outcome: it furnishes a faithful-description baseline and a sanity check on the instrument, in the same spirit as the characterisation tests that legacy-code practitioners write to pin existing behaviour before changing it [37]. It is emphatically *not* a proof that the code is free of bugs. A defect that the code imple-

ments consistently will be faithfully described by a code-inferred clause and will verify without complaint.

Bugs surface, instead, as *disagreements*, and they do so by two distinct mechanisms. The first is internal: the code violates a pattern that the inference over-generalised from the rest of the code, so that the proof of a too-strong inferred clause fails. Here the failure is a known hazard of heuristic inference—an over-strong clause [39]—but the location of the failure is also a plausible anomaly worth a human's attention. The second, and more valuable, mechanism is external: the code disagrees with a `ticket-stated` or `doc-stated` clause. Because the intent streams encode what the system was *supposed* to do, a clause that the code cannot satisfy is direct evidence of a code-versus-intent gap, exactly the productive disagreement that intent-derived oracles have been shown to expose against real historical defects [35]. Correctness relative to intent, then, never comes from the code stream; it comes from the intent streams and, ultimately, from the human ratification that the elaboration layer provides. Nothing the startup phase emits is a trusted oracle: a clause translated from documentation or a ticket without a human in the loop remains `*-unconfirmed` and is used to *drive* the dialogue, not to gate any decision.

A direct consequence is that the startup phase cannot be gated on the absence of verification errors. In a greenfield synthesis loop a failed proof is a defect to be repaired before proceeding; in brownfield characterisation a code-versus-intent violation is the *success outcome*, because the phase has done its job of locating a candidate bug. Demanding zero errors would amount to demanding that the recovery find nothing, which is precisely the wrong objective for a phase whose purpose is to expose the discrepancies that a subsequent human dialogue will adjudicate.

7.3 The Characterisation Report and Loop Reordering

The output of the startup phase is therefore not a green build but a *characterisation report*. The report has three parts. The first is the recovered contract itself, with every clause carrying its provenance tag and an *assurance* tag (`proved`, `bounded-checked`, `tested`, or `unchecked`) recording how far verification got, so that the strength of each claim is auditable and the degradation from full proof to bounded check to test under Z3's decidability limits is recorded rather than hidden. The second part is the agenda of candidate bugs: the N disagreements detected across streams, each presented as a concrete clause with its conflicting evidence and, where verification produced one, a counterexample witness. The third part is the coverage statement—what could be specified and verified, and what could not, whether because no clause could be inferred, because verification did not terminate, or because the module lay outside the specified frontier. These discrepancies become the *opening agenda* of the human dialogue: the brownfield phase does not silently absorb the codebase, it hands the engineer a prioritised list of places where code, documentation, and recorded intent fail to agree.

This reframing forces a reordering of the verification loop relative to the greenfield case. In greenfield use the loop is purely *prospective*: a new feature is elaborated, synthesized, and verified. In brownfield use the *first* pass over an existing system is *retroactive*. Before the pipeline changes anything, it resolves the recovered discrepancies—confirming intent where the streams agreed, fixing the bugs the disagreements exposed, and correcting documentation that the code has outgrown—and in doing so it establishes a trusted, ratified baseline contract for the touched surface. Only once that baseline exists do subsequent passes become prospective, elaborating and verifying new changes against it. The ordering is deliberate and has a process rationale: the tool earns trust on the existing code, by surfacing real defects and producing a contract the team has ratified, before it is permitted to author or verify any change. A pipeline that began by generating new code against an unratified, code-derived contract would inherit every latent bug as a "verified" obligation.

Because a whole legacy application cannot be formally specified—verification does not scale to hundreds of thousands of lines, and the decidability limits documented for the underlying solver are real—the startup phase adopts a *frontier* adoption strategy. It specifies only the module currently being touched, the

public API surface, and the modules referenced by the active tickets, treating the remainder as unspecified and governed only by weak inherited-default contracts. The boundary of the specified frontier is exactly where the compositional change-propagation machinery of our prior work [33] plugs in: the contracts on the boundary methods are the interface across which caller obligations are recomputed when a callee contract changes, so that growing the frontier and propagating a version-compatibility change are, mechanically, the same operation [34]. Frontier adoption thus makes the brownfield phase tractable without abandoning the compositional thesis that motivates the programme.

7.4 Honest Limitations

The characterisation phase rests on two inference steps that are individually imperfect, and their limitations must be stated plainly. The first is that specifications inferred from code skew *weak*: they describe what the code does, not what it should do, and a weak contract is a poor oracle. This is why the intent streams and human ratification are load-bearing, and why the phase prioritizes the public API surface and leans on inherited-default contracts to keep the ratification labor bounded rather than attempting to ratify every inferred clause. The same inference is occasionally *wrong* in the opposite direction, over-generalising into a clause the code does not satisfy and so producing a *false* candidate bug; the report mitigates this by labelling every entry on the agenda a *candidate*, to be confirmed or dismissed by the human, never an asserted defect.

The second imperfect step is the translation of tickets and documentation into formal clauses. Done without a human, NL-to-formal translation is unreliable [24, 56], which is exactly why those clauses are retained as `*-unconfirmed` and used only to drive dialogue. Compounding this, linking a ticket or a documentation fragment to the precise code element it concerns is a noisy traceability problem: issue references, commit links, and path heuristics are partial, and the information-retrieval and learned approaches to traceability recovery [8, 14, 61] are credible only on a curated module, not across an arbitrary repository. The phase therefore claims realism on a well-scoped frontier, not coverage of an entire production system. Finally, the verifier's own scalability and decidability limits—Z3 timeouts on multiplicative reasoning, array theory, and rich preconditions—bound what can be proved rather than merely tested, and the assurance tags make that boundary visible in the report. Taken together, these limitations are the reason the brownfield phase is designed to produce an *agenda* for a human dialogue rather than a verdict: a wrong contract verified against would be worse than no contract at all, and the provenance tagging, the independent authority constraint, and the human ratification of behaviour are precisely the safeguards that keep an unconfirmed recovered clause from being mistaken for a trusted oracle.

8 The Synthesis–Verification Loop

Below the formal-contract pivot established in the layered model (section 5), the ambiguous human world has been left behind: the contract is a precise, machine-readable artefact, and the agent's task is no longer to interpret intent but to satisfy an obligation. This is the region of the lifecycle in which classical correctness machinery applies without dilution, and it is where the proposed pipeline supplies the verification spine that emerging agentic lifecycles lack. The mechanism is a closed loop shaped like counterexample-guided inductive synthesis (CEGIS) [91, 93]: a synthesis agent proposes an implementation, a sound verifier checks the implementation against the contract over all inputs, and any failure is returned to the synthesis agent not as a verdict but as a concrete counterexample that drives the next attempt. This section sets out the loop, argues why a counterexample is a categorically stronger feedback signal than a failing sampled test, explains how inference supplies loop invariants alongside the body so that the verifier can actually discharge the obligation, and specifies what happens when verification does not converge: a tiered-assurance degradation that records the level of guarantee achieved rather than silently weakening the claim.

Algorithm 1 The synthesis–verification loop for a single method.

Require: contract C (tagged obligations); iteration cap N; context Γ
Ensure: proved body, or degraded result with recorded assurance level

$feedback \leftarrow \emptyset$
for $i \leftarrow 1$ **to** N **do**
  $b \leftarrow \textsc{Synthesize}(C, \Gamma, feedback)$ {implementer agent}
  $inv \leftarrow \textsc{InferInvariants}(b, C)$ {JML-Inferrer supplies loop invariants}
  $r \leftarrow \textsc{Verify}(b \cup inv, C)$ {OpenJML/Z3, all inputs}
  if $r =$ `proved` **then**
    return $(b,$ `proved`$)$
  else if $r =$ `counterexample`(x) **then**
    $feedback \leftarrow feedback \cup \{x\}$ {concrete witness, not a verdict}
  else
    break {undecided: solver timeout/incapacity}
  end if
end for
return $\textsc{Degrade}(b, C)$ {bounded check, then test; record level}

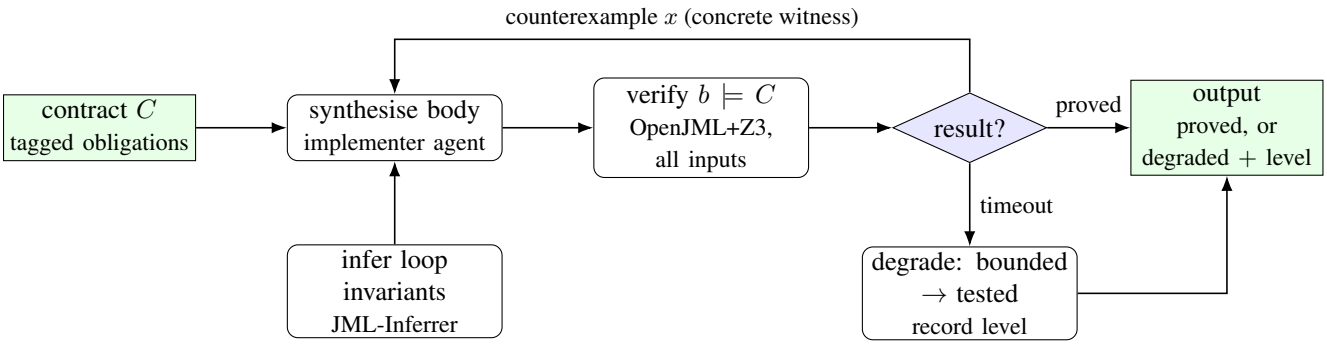

Figure 3: The synthesis–verification loop of algorithm 1. A counterexample is fed back to the synthesiser as a concrete witness rather than a verdict; the JML-Inferrer supplies discharging loop invariants alongside the body; and on solver non-termination the obligation is degraded down the assurance ladder rather than dropped.

8.1 The loop and its feedback signal

The structure of the loop is given as algorithm 1 and depicted in fig. 3. The synthesis agent receives the contract C for a method—a set of provenance- and assurance-tagged obligations, of which the verification-relevant subset comprises preconditions, postconditions, frame (`assignable`) clauses, and any class invariants—together with the surrounding type context. It emits a candidate body. The trusted verifier, OpenJML's Extended Static Checking (ESC) discharged by the Z3 SMT solver [23, 27], then attempts to prove that the body satisfies C. The verifier is a Hoare-logic checker [48] operating by weakest-precondition predicate transformation [29]: it reduces the verification condition to a logical formula whose validity it asks Z3 to decide. Three outcomes are possible. If the formula is valid, the body is proved correct against C and the loop terminates successfully. If it is invalid, Z3 returns a satisfying assignment to its negation—a concrete input state that drives the implementation into a violation of some obligation. If the solver neither proves nor refutes within the configured resource budget, the obligation is undecided and the loop falls through to degradation (section 8.3).

The reason this loop is worth building, rather than the test-and-eyeball loop of current agentic tools, lies entirely in the quality of the feedback signal on the failure path. Contemporary self-improving code agents—Self-Refine [66], Reflexion [88], self-debugging from execution traces [20], and test-driven flows such as AlphaCodium [85]—close their loop with a sampled test suite. When such a suite passes, it certifies only the finitely many inputs it happened to exercise; the weakness of automatically generated suites as correctness oracles is well documented, with EvalPlus showing that a large fraction of code accepted by the original HumanEval and MBPP tests fails under denser inputs [9, 18, 62]. A failing test, moreover, reports only that input 37 produced the wrong answer; it carries no guarantee that fixing input 37 does not break input 38, and it says nothing about the unsampled region. By contrast, a successful ESC proof certifies the obligation over the *entire* input domain, and a failed proof yields a counterexample that is a precise diagnostic: a fully instantiated pre-state in which the current body and invariant cannot discharge the obligation. Under modular checking such a witness may reflect an inadequate loop invariant rather than a reachable bug, but it remains a witnessed cause in the verifier's own logic rather than a sampled symptom. The synthesis agent is therefore handed not a symptom but a witnessed cause, in the same currency the verifier itself reasons in. This is the same discipline that distinguishes verifier-coupled code agents such as Clover [94], AI-assisted Dafny synthesis [59, 71], and counterexample-driven proof repair [75, 103] from their test-only counterparts; the present pipeline brings that discipline to mainstream Java through JML and OpenJML [22, 58] rather than to a bespoke verification language.

A further, structural property makes the signal trustworthy: the verifier is never the agent. The contract against which the body is checked is authored by an independent elaborator role and the verifier is the unmodified OpenJML/Z3 toolchain, so no instance that wrote the code also grades it. This independent-authority constraint (section 5) is what separates the loop from the circular, self-grading arrangement in which one model writes both the implementation and the tests that judge it—a hazard increasingly recognised for LLM-generated oracles [64]. The verifier discharges a contract it did not write, against code it did not write, over all inputs; that is the property that earns the word *verified*.

8.2 Inferring loop invariants alongside the body

A counterexample-guided loop over a sound verifier is only useful if the verifier can in fact discharge the obligations the synthesis agent must meet, and for any method containing a loop this is not automatic. ESC reasons about loops inductively: it requires a loop invariant strong enough to imply the postcondition at loop exit yet preserved by every iteration of the body. A correct body with no invariant, or with too weak an invariant, fails to verify even though it is correct—the proof, not the code, is incomplete. Demanding that the LLM emit both a correct body and a discharging invariant compounds two hard problems, and recent invariant-by-LLM work confirms that invariant generation is itself an error-prone, check-in-the-loop activity [17, 54, 81].

The pipeline sidesteps this by separating the two concerns. The body is synthesized by the agent; the loop invariant is supplied, where possible, by the JML-Inferrer, the AST-heuristic instrument developed across the earlier articles of this programme. The inferrer already emits invariants for exactly the loop families that the proof-of-concept targets first—running maximum and minimum, linear search for an index or a boolean, and accumulating sums—and emits them in the direction-aware, inductively preserved form that ESC needs (for an increasing counter, the bound $i \geq init$ together with the quantified property established so far). listing 4 shows the shape for a running-maximum loop: the inferrer attaches both the frame bound and the universally quantified property that makes the postcondition discharge at exit. Because invariant inference runs on the candidate body the agent just produced (line 4 of algorithm 1), it adapts to whatever loop the agent chose rather than presupposing one, and when the inferred invariant is too weak the verifier's failure is itself a counterexample that re-enters the loop. The division of labor is deliberate: the agent contributes the algorithmic content, in which generative breadth is an asset, while the inferrer contributes

Table 6: The tiered-assurance ladder applied on the verification path. Each obligation is tagged with the highest level achieved; degradation is recorded, never silent.

Level	Coverage	Triggered when
`proved`	all inputs	ESC discharges the verification condition
`bounded-checked`	inputs up to size k	unbounded query times out; bounded query decides
`tested`	sampled inputs	no proof obtainable; generated tests pass
`unchecked`	none	not formally specifiable or all checks inconclusive

the proof scaffolding, in which a sound, deterministic heuristic is more dependable than a sampled guess. This reuses Article 3's compositional inference as a synthesis-time component rather than as a terminal report [33].

```
//@ requires a != null && a.length > 0;
//@ ensures (\forall int k; 0 <= k && k < a.length; \result >= a[k]);
int max(int[] a) {
  int m = a[0];
  //@ maintaining i >= 1 && i <= a.length;
  //@ maintaining (\forall int k; 0 <= k && k < i; m >= a[k]);
  //@ decreasing a.length - i;
  for (int i = 1; i < a.length; i++)
    if (a[i] > m) m = a[i];
  return m;
}
```

Listing 4: A loop body synthesized by the agent (running maximum) annotated with the loop invariant supplied by the inferrer; the quantified clause is what lets ESC discharge the postcondition at loop exit.

8.3 Termination, the iteration cap, and tiered assurance

The loop must terminate even when synthesis does not converge, for two independent reasons. The synthesis agent may simply fail to produce a satisfying body within a bounded number of attempts, and even when the body is correct the verifier may not decide the verification condition: Z3 is incomplete on the formula classes that arise from nonlinear (multiplicative) arithmetic, array theory, and richly quantified preconditions, and on these it returns neither a proof nor a counterexample but exhausts its resource budget. Both eventualities are bounded by the iteration cap N on the synthesis side and by a per-call solver timeout on the verification side. Neither limit is incidental; both are properties the pipeline reports honestly rather than engineering around, because the decidability and scalability ceiling of SMT-backed verification is real and shared by every tool in this class [11, 23, 27].

On persistent failure the pipeline does not discard the obligation and does not silently lower the bar. It applies *tiered assurance with graceful degradation*: it attempts, in order, full proof, then bounded verification (the same checker restricted to inputs up to a fixed size, which converts an intractable unbounded query into a decidable finite one), then a generated test, and it records on the obligation the highest level actually achieved, drawn from the assurance vocabulary $\langle$`proved`, `bounded-checked`, `tested`, `unchecked`$\rangle$ introduced with the provenance- and assurance-tagging mechanism (section 5). table 6 summarises the ladder. The crucial discipline is that the degradation is *visible*: a method that could only be bounded-checked carries that fact in its contract, so a downstream reader—human or tool—never mistakes a partial guarantee for a total one. An honest `tested` tag is preferable to a dishonest `proved` one, and recording `unchecked` where nothing could be established is preferable to dropping the obligation, because a silently dropped obligation is indistinguishable from one that was never required.

When the loop terminates without a full proof, the pipeline reports the specific failing obligation together with the counterexample or timeout that defeated it, rather than a bare rejection. This keeps the failure actionable: a recorded counterexample documents exactly which behaviour the agent could not realise, a recorded timeout documents a known decidability limit rather than a code defect, and in both cases the obligation remains in the contract at its true assurance level. Read against the broader argument of the article, this is what allows the loop to be deployed against real software at all. Not every intent is formally specifiable and not every specifiable intent is decidable; a lifecycle that insisted on `proved` for everything would verify only toy programs, whereas one that recorded its assurance level honestly can admit the full spectrum from machine-checked guarantee down to tested baseline while remaining truthful about which is which. The synthesis–verification loop is therefore not a binary gate but a graduated one, and it is precisely this graduation—the willingness to say exactly how strongly each obligation is held—that makes a verification-centric agentic lifecycle realistic rather than aspirational.

9 Compositional Change Propagation and Version Compatibility

The mechanisms developed so far treat a contract as a property of a single method: a set of provenance- and assurance-tagged obligations against which one implementation is synthesized and verified. Software, however, is not a flat collection of independent methods but a graph of callers and callees, and the durable value of a machine-checkable contract layer is realised only when that graph is exploited. This section promotes the compositional refinement established in Article 3 of this programme [33] from an inference-time accuracy gain to a lifecycle capability. The central observation is that because formal contracts compose along the call graph, a change to a callee's contract mechanically *recomputes* the proof obligations of every caller, and the set of callers whose obligations can no longer be discharged is exactly the set of callers that the change breaks—provided the compared contracts are strong enough to express the relied-upon behaviour and the verifier decides each obligation; where either condition fails, a caller is reported as undetermined rather than safe, so the analysis is sound in flagging breaks but not complete (a limitation made precise later in this section). Critically, this set is identified by re-running the verifier, not by re-running the program: it is a static, behavioural blast-radius analysis (fig. 4). The same machinery answers two questions that the prevailing tooling treats as unrelated—"does this internal refactor break any of my call sites?" and "does this dependency upgrade break any of my call sites?"—because, at the level of contracts, an internal refactor and a third-party version bump are the same operation. This identity is the concrete link between the two halves of the thesis core: formal-methods adoption and library/version compatibility are served by a single underlying mechanism.

9.1 Contracts Compose Along the Call Graph

Design by contract [69] casts a method specification as a two-party agreement: the caller must establish the callee's precondition, and in return the callee guarantees its postcondition. Modular deductive verification operationalizes this agreement by discharging each method against the *contracts* of the methods it invokes rather than against their bodies. When the proposed pipeline verifies a caller c that invokes a callee m, OpenJML replaces the body of m with its contract $(pre_m, post_m)$: at the call site it asserts pre_m as a proof obligation on c and assumes $post_m$ for the continuation. The correctness of c therefore depends on m *only* through $(pre_m, post_m)$, never through the implementation of m. This is the standard soundness story of modular verification, and it is what makes verification scale; here it is repurposed as the engine of change analysis. The contract is the abstraction barrier, so any change confined behind that barrier—one that preserves $(pre_m, post_m)$—is, by construction, invisible to every caller, and any change that crosses the barrier—one that alters $(pre_m, post_m)$—propagates to callers in a precisely characterizable way.

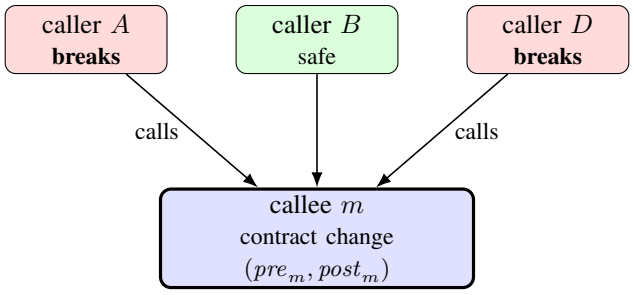

Figure 4: Compositional change propagation as behavioural blast-radius analysis. A change to callee m's contract recomputes the proof obligations of its callers; the callers whose obligations the new contract no longer discharges (A, D) are flagged statically; callers the verifier cannot decide are reported as undetermined. An internal refactor and a third-party version bump are, at the level of contracts, the same operation.

Article 3 of this programme established that the heuristic inference instrument admits a second, weakest-precondition pass that strengthens caller contracts using the contracts of their callees, yielding roughly $18{,}854$ additional well-formed preconditions on the evaluation corpus beyond a single pass. That result was reported as evidence that compositional reasoning recovers specification content a local analysis misses. The present article reads the same mechanism in the opposite direction. If a downstream change to $(pre_m, post_m)$ would have altered the precondition that the second pass computed for c, then that change is, by definition, a change that affects c's obligations. The dependency edges that the compositional pass traverses to *build* specifications are the same edges along which a change *propagates*. Recomputation is thus not new machinery but the existing compositional pass re-evaluated under a perturbed callee contract.

9.2 Change Propagation as Re-Verification

Let G be the contract-level call graph in which an edge $c \to m$ records that caller c invokes callee m under a known set of call-site arguments and path conditions. A proposed change replaces the callee contract $(pre_m, post_m)$ with $(pre'_m, post'_m)$. The propagation procedure is then a localized re-verification:

1. **Identify the affected frontier.** Collect the immediate callers of m in G. Only these methods have a proof that mentions the changed contract directly; deeper callers are reached transitively only if a frontier caller's own contract changes as a result.

2. **Recompute and re-discharge.** For each affected caller c, re-run the modular verification of c with the callee modelled by $(pre'_m, post'_m)$. Two failure modes are distinguished. A *precondition break* arises when c established pre_m at the call site but cannot establish the strengthened pre'_m; the verifier reports the unprovable call-site assertion. A *postcondition break* arises when c's own postcondition relied on the old $post_m$ and is no longer discharged under the weaker $post'_m$.

3. **Propagate transitively.** If re-verification of c requires a change to c's own contract—for example, a strengthened precondition that c must now impose on *its* callers—enqueue c as a changed callee and repeat. Because each step either discharges or strengthens a finite contract, and the graph is finite, the procedure terminates at a fixed point: the transitive set of breaking call sites together with the induced contract changes.

Two properties of this procedure deserve emphasis. First, it never executes the code. The decision of whether caller c is broken is the decision of whether an OpenJML proof obligation is discharged by Z3,

quantified over *all* inputs satisfying the path conditions, rather than over a sampled test suite. Where test-based regression detection answers "did any of the inputs I happened to run change behaviour?", contract re-verification answers "can any input within the specified domain now reach a violated obligation?". Second, the analysis is directional and asymmetric in a way that mirrors the classical theory of behavioural subtyping and the contravariance of refinement. *Weakening* a precondition or *strengthening* a postcondition is a safe change: every caller that satisfied the old contract satisfies the new one, so no call site is flagged, and the change is verifiably backward compatible. *Strengthening* a precondition or *weakening* a postcondition is a potentially breaking change, and the procedure reports exactly which callers it breaks and why. This gives a developer not merely a binary compatible/incompatible verdict but a ranked, justified work list of call sites, each accompanied by the specific obligation that fails—the same concrete-witness signal that drives the synthesis–verification loop of section 8.

9.3 One Mechanism, Two Lifecycle Events

The procedure of section 9.2 is agnostic to *why* $(pre_m, post_m)$ changed. When the changed method is internal to the codebase under development, the analysis is an impact analysis for a refactor: a developer who tightens a guard, broadens an accepted range, or narrows a returned set sees immediately which internal call sites must adapt. When the changed method belongs to a third-party library and the change is observed by comparing the contract attached to version v with that attached to version v', the very same analysis becomes a behavioural compatibility check for a dependency upgrade. The question "which of my N call sites does this upgrade break behaviourally?" is thereby promoted to a first-class, machine-answerable lifecycle event.

This framing addresses a well-documented and costly gap in current dependency practice. Semantic versioning [82] is the dominant convention for signalling compatibility, yet large-scale studies of Maven Central repeatedly find that version numbers are unreliable predictors of actual breakage: a substantial fraction of releases labelled non-breaking introduce incompatible changes [77, 84], and dependency networks evolve fast enough that clients are exposed to such changes continually [28]. The tooling that does exist operates predominantly at the level of *signatures*—method removal, parameter-type changes, visibility reductions—as exemplified by APIDiff and related detectors [15, 52]. Signature-level analysis, however, is blind to *behavioural* breaking changes: a method whose signature is untouched but whose contract has shifted, for instance one that begins rejecting an input it previously accepted or that stops guaranteeing a postcondition a caller relied upon. Such behavioural incompatibilities are both prevalent and damaging to clients [51, 74], and static checking of library updates has been pursued precisely because recompilation alone does not surface them [41]. Recent work even applies LLMs to *repair* clients broken by dependency updates [50]; the contract-propagation analysis proposed here is complementary, supplying the precise, per-call-site diagnosis of *what* broke and *why* that such a repair step requires as input. By making the unit of comparison the formal contract rather than the signature, the pipeline detects exactly the behavioural incompatibilities that semantic-versioning labels and signature diffs miss, and it does so soundly over the input domain rather than over a regression sample. The contract attached to a compiled artefact—the embedding mechanism validated in Article 2 of this programme [32]—is what makes the library side of this comparison feasible without library source, and the diff-only, cache-backed CI integration of Article 4 [34] is what makes re-running the analysis on every upgrade economical.

9.4 A Worked Version Step

Consider a utility library that, at version v, exposes an integer-parsing helper modelled by the contract in listing 5. The method accepts any string and returns a sentinel of -1 when the argument is not a valid integer; its postcondition records both the success and the sentinel cases.

Table 7: Classification of callee-contract changes by the re-verification procedure. Safe changes flag no callers; breaking changes yield a justified per-call-site work list.

Change to callee contract	Compatibility	Caller effect
Weaken precondition	Safe	No call site flagged
Strengthen postcondition	Safe	No call site flagged
Strengthen precondition	Breaking	Call-site assertion unprovable
Weaken postcondition	Breaking	Caller postcondition undischarged

```
//@ requires s != null;
//@ ensures isParsable(s)  ==> \result == parseInt(s);
//@ ensures !isParsable(s) ==> \result == -1;
public static int toInt(String s) { ... }
```

Listing 5: Library method `toInt` at version v (callee contract).

A downstream client contains the call site in listing 6. Its own postcondition—that the returned count is non-negative—is discharged against version v because every branch of `toInt` returns either a parsed value guarded by the caller or the sentinel -1, which the caller maps to 0.

```
//@ ensures \result >= 0;
public int countOrZero(String s) {
    int n = LibUtil.toInt(s);   // callee contract assumed here
    return n < 0 ? 0 : n;
}
```

Listing 6: Client call site relying on the version-v contract.

At version v' the library maintainer revises `toInt` to throw on malformed input rather than return a sentinel—a change the maintainer regards as a clean-up and ships under a non-breaking version label. The revised contract is shown in listing 7: the precondition is *strengthened* to demand a parsable argument, and the success postcondition is retained while the sentinel clause is removed.

```
//@ requires s != null && isParsable(s);
//@ ensures \result == parseInt(s);
public static int toInt(String s) { ... }
```

Listing 7: Library method `toInt` at version v' (strengthened precondition).

Running the procedure of section 9.2 with $(pre_{\texttt{toInt}}, post_{\texttt{toInt}})$ replaced by the version-v' contract recomputes the obligations of `countOrZero`. The strengthened precondition `isParsable(s)` becomes a call-site proof obligation that `countOrZero` cannot discharge: nothing in the caller establishes that `s` is parsable, so the verifier reports a precondition break at the call site. The analysis therefore flags `countOrZero` as a behaviourally broken caller of the upgrade, names the unprovable obligation `isParsable(s)` as the cause, and—because no contract change to `countOrZero` would discharge the obligation without either adding a guard or strengthening `countOrZero`'s own precondition—surfaces the two candidate remediations as the induced contract change to be propagated to *its* callers. A signature-level tool would report no break whatsoever, since the signature of `toInt` is unchanged; the contract-level analysis correctly identifies the behavioural incompatibility that the non-breaking version label conceals. Table 7 summarises how the four contract-change directions classify under the analysis.

9.5 Scope, Frontier, and Honest Limits

The propagation analysis inherits both the strengths and the limits of the underlying instrument. Its soundness is the soundness of OpenJML extended static checking discharged by Z3: a caller reported as compatible is compatible over the entire specified input domain, not merely over a tested sample, but a caller for which Z3 times out—on the multiplicative reasoning, array theory, or rich-precondition cases known to defeat the solver—must be reported under the tiered-assurance scheme as *undetermined* rather than silently passed. The analysis is also only as good as the contracts it compares. For first-party code the contracts are those the pipeline already maintains; for third-party libraries the contract attached to each version is, in the brownfield setting, an *inferred* contract recovered from the library's code, and an inferred callee contract skews weak, so a genuine behavioural break may be missed when the inferred contract failed to capture the guarantee a caller actually relied upon. This is the same weak-specification limitation that motivates ratification and inherited defaults elsewhere in the pipeline, and it confines the trustworthy application of version-compatibility analysis to the API surface and the modules within the specified frontier rather than to an entire legacy dependency tree. Within that frontier, however, the analysis delivers what semantic-versioning labels and signature diffs cannot: a sound, behaviour-level, per-call-site verdict on whether a refactor or an upgrade is safe, computed without running a single test. Establishing the empirical precision and recall of this analysis against a ground-truth corpus of known behavioural breaking changes is left to the evaluation; the architectural claim here is that the capability falls directly out of compositional contracts and requires no machinery beyond the re-verification of an already-built specification graph.

10 Instantiation: A Proof-of-Concept on Apache Commons Lang

The reference architecture of section 5 is a design claim, and a design claim about a verification spine is worth little unless it can be shown to be buildable from components that already work. This section therefore does not present a finished system; it presents an *instantiation*—a concrete mapping of every architectural layer onto either a pre-existing, independently validated asset of this programme or a small, well-scoped piece of new code, together with a staged build plan and a real, three-stream demonstration corpus. The purpose of the exercise is to de-risk the architecture: to demonstrate that the trusted parts of the pipeline already exist and have been validated outside the present work, that the new parts are modest orchestration rather than novel verification machinery, and that the whole can be exercised end-to-end on a real-world library whose intent is documented in code, issue tickets, and Javadoc simultaneously. Throughout, the discipline that makes the architecture credible—that the trusted verifier never grades itself—is preserved by construction, because the verifier is a component the author did not write for this purpose and cannot tune to flatter the agent's output.

10.1 Mapping the architecture onto validated and new components

The instantiation rests on a deliberate separation between *trusted* components, which determine whether a "verified" claim is honest, and *untrusted* components, which merely propose artefacts that the trusted components then judge. The architectural insistence on independent authority (section 5) reduces, in implementation terms, to a single rule: no trusted component may have been authored, configured, or prompted by the agent whose output it evaluates. The reuse of prior, externally validated assets is what makes this rule enforceable rather than aspirational.

The *trusted oracle* below the contract pivot is the custom OpenJML fork driven by the Z3 SMT solver. OpenJML performs extended static checking of Java against JML contracts, discharging proof obligations over *all* inputs rather than a sampled subset [22, 23], and Z3 is the decision procedure that settles the resulting verification conditions [27]. The fork extends the stock tool with `define-fun-rec` encodings of `\sum`,

Table 8: Mapping of architectural layers onto validated assets and new code. Trusted components determine the honesty of a "verified" claim; new components only propose artefacts that trusted components judge.

Architectural role	Component	Status
Trusted oracle (verification)	OpenJML fork + Z3 [23, 27]	Validated (Art. 3–4)
Draft-contract bootstrap	JML-Inferrer (AST heuristics) [33]	Validated (Art. 1–3)
Compositional refinement	WP second pass [33]	Validated (Art. 3)
Surface-syntax normalisation	`AnnotationToJMLConverter`	Validated (Art. 2)
Reproducible execution	Dockerised pipeline [34]	Validated (Art. 4)
Phase sequencing & CEGIS loop	Orchestrator	New (untrusted)
Elicitation / drafting / synthesis	LLM calls (Claude)	New (untrusted)

`\product`, and `\num_of`, which unblocks a class of aggregate postconditions that the upstream release reports as unsupported. Critically, this oracle is a deductive verifier with a fixed semantics: it does not learn from, and cannot be steered by, the agent that produced the code under test. It is the embodiment of the independent-authority constraint, because its verdict is a function of the contract and the code alone.

The *bootstrap* for draft contracts is the JML-Inferrer, the Java 21 / JavaParser instrument developed across Articles 1–4 of this programme. It infers preconditions, postconditions, loop invariants, class invariants, and assignable clauses by AST heuristics, and in the brownfield setting it supplies the code-derived draft contract that the characterisation phase reconciles against the ticket and documentation streams (section 5). Its compositional second pass—the weakest-precondition refinement that recovered approximately 18,854 additional well-formed preconditions on the corpus [33]—is precisely the mechanism that the change-propagation layer promotes into a lifecycle capability. The `AnnotationToJMLConverter` normalises inferred and authored annotations into the surface JML syntax the verifier consumes, and the Dockerised pipeline of Article 4 [34] provides reproducible, isolated execution of the entire toolchain, including the pinned solver build whose choice materially affects termination.

table 8 summarises the mapping. The decisive observation is the asymmetry between the two columns: every *trusted* component is pre-existing and externally validated, whereas all *new* code is untrusted orchestration. The new code comprises an orchestrator that sequences the elaboration, synthesis, and verification phases and manages the counterexample-refinement loop, together with the LLM calls (to Claude) that perform elicitation, contract drafting, and code synthesis. None of these new components renders a verification verdict. The orchestrator routes artefacts; the LLM proposes; the OpenJML fork disposes. Consequently, a defect in the new code cannot cause OpenJML to certify falsely that code satisfies the contract it was given, because the verifier's verdict is a function of contract and code alone and the new code cannot tune it. It can, however, cause a *wrong contract* to be drafted; the independent-authority property therefore protects the code-satisfies-contract judgement, not the contract-captures-intent judgement. The latter is guarded not by the verifier but by human ratification (milestone M3) and by the `inferred-unconfirmed` provenance that an unratified drafted contract carries, so a poor contract surfaces as a discrepancy the human adjudicates rather than as a silent false certification of intent.

10.2 The demonstration corpus: a real three-stream dataset

A faithful instantiation of the brownfield workflow requires a target in which all three evidence streams of multi-source intent triangulation—code, tickets, and documentation—are genuinely present, public, and linkable. Apache Commons Lang satisfies this requirement unusually well and is already wired into the author's experiment harness, which removes incidental engineering risk from the demonstration. The *code*

Table 9: Milestones M0–M3. Each isolates and de-risks one architectural assumption and yields a runnable artefact.

	Milestone	Assumption de-risked	Artefact
M0	Verify-as-a-service	Trusted oracle wraps cleanly	`verify(src)` endpoint
M1	Synthesis loop, fixed contract	CEGIS loop converges	Closed-loop synthesiser
M2	Contract generation from NL	Drafting is tractable	NL/code → contract
M3	Conversational ratification	Behaviour-level ratify works	Socratic + example loop

stream is the library's mature, heavily exercised Java source, dominated by self-contained utility methods over strings, numbers, arrays, and primitives. The *ticket* stream is the public Apache JIRA `LANG` project, whose `LANG-####` issues record why behaviours exist, which edge cases were once wrong, and how boundary decisions were ultimately resolved—exactly the "why" that code alone cannot supply [68, 72]. The *documentation* stream is the library's rich Javadoc, which states declared intent at method granularity and is therefore the natural source of doc-stated obligations.

The value of triangulation is disagreement detection, and Commons Lang offers concrete disagreements rather than contrived ones. Consider `StringUtils` and `NumberUtils`, two classes whose null-handling and boundary conventions have been the subject of repeated clarification in both the Javadoc and the issue tracker. A code-derived draft contract for a method such as `StringUtils.substring` or `NumberUtils.createNumber` encodes what the present implementation does at the empty-string, null, and out-of-range boundaries; the Javadoc states what the method is *declared* to do; and the `LANG` tickets frequently record cases where the two once diverged. Where the code-derived clause and the doc-derived or ticket-derived clause disagree, the instantiation surfaces a *candidate bug* or a stale-documentation finding, which is precisely the success outcome of the characterisation phase rather than a failure of it. Reconciling these streams is noisy—linking a ticket or a Javadoc sentence to the method it constrains depends on references, commit links, and path heuristics that are imperfect at scale [8, 14, 95]—and the demonstration accordingly makes no claim of corpus-wide recall. It claims that, on a curated module surface, the reconciliation produces real, inspectable discrepancies with per-clause provenance, which is sufficient to validate the architecture's central brownfield assertion: that code-derived verification is characterisation, and that correctness-relative-to-intent emerges only where the code is confronted with the ticket and documentation streams.

10.3 Staged build: milestones M0–M3

The build is staged so that each milestone de-risks one independent assumption of the architecture before the next is attempted, and so that every milestone produces a runnable artefact. table 9 states each milestone, the assumption it tests, and the artefact it yields.

M0: verify-as-a-service. The first milestone wraps the existing Dockerised pipeline behind a single function, `verify(src)`, that returns either `ok` or a concrete counterexample. This isolates and proves the load-bearing assumption that the trusted oracle can be invoked as a black-box service by untrusted orchestration without the orchestration acquiring any influence over the verdict. M0 establishes the independent-authority boundary as an executable interface: the orchestrator hands a contract and an implementation across the boundary and receives a verdict back, and nothing it does on its side of the boundary can change a `fail`

into an `ok`. Because the underlying toolchain is already validated, M0 carries essentially no verification risk; its risk is purely in the interface, and proving the interface is the point.

M1: synthesis loop against a fixed contract. The second milestone exercises the CEGIS-shaped synthesis–verification loop in isolation, against a fixed, hand-written contract attached to a method class that the inferrer already verifies green. The synthesis agent emits an implementation; `verify` judges it; on failure the counterexample—a concrete witness, a strictly more informative signal than "a test failed" [13]—is returned to the agent, which regenerates, capped at N iterations. M1 de-risks the loop mechanics independently of the harder problem of producing a good contract: because the contract is fixed and known-dischargeable, any failure to converge is attributable to the synthesis agent or the loop plumbing, not to a defective oracle or an unverifiable specification. This is the cleanest possible test of the closed-loop claim, and it deliberately mirrors the CEGIS structure that the synthesis lineage established [3, 91].

M2: contract generation from natural language. The third milestone introduces contract drafting. In the greenfield direction it generates a JML contract from NL intent by few-shot prompting an LLM, grounded in the author's own corpus of verified JML as exemplars—a use of in-context examples consistent with recent NL-to-specification work [24, 35, 65]. In the brownfield direction the JML-Inferrer supplies the code-derived draft directly, and the LLM's role narrows to reconciling that draft with the ticket and documentation streams. M2 de-risks the elaboration layer's tractability: it tests whether a draft contract good enough to enter the M1 loop can be produced at all, and it does so without yet requiring the contract to be correct, because every clause it produces is provenance-tagged and, absent human ratification, marked *inferred-unconfirmed* or *doc-stated* rather than treated as a trusted oracle.

M3: conversational ratification. The final milestone closes the loop on human authority by implementing the Socratic, decision-surfacing dialogue and the example-pinning ratification mechanism. Rather than asking the human to confirm formal text—the move that would make the ratification gap (section 5) fatal—the orchestrator renders each contested clause as a concrete, checkable example ("given `x = -1`, this contract says the result is `0`; is that correct?") and surfaces only the boundary decisions the drafting agent had to resolve: null versus empty, overflow, ordering, duplicates, error versus exception. Ratified examples become pinned oracles that the contract must continue to satisfy. M3 de-risks the one part of the architecture that cannot be validated by tooling alone, namely that a non-formalist human can exercise genuine authority over a formal contract by ratifying behaviour rather than logic.

10.4 Target method classes

The instantiation deliberately begins with method classes that the inferrer already produces dischargeable specifications for, so that the early milestones test the loop and the dialogue rather than the open research problem of verification scalability. These are: null-safety and emptiness contracts, which dominate `StringUtils` and verify reliably; array-bounds obligations; simple arithmetic postconditions over primitives, as found across `NumberUtils`; and the loop patterns for which the inferrer already emits adequate invariants—maximum and minimum, linear search, and summation. listing 8 shows a representative target: a null-and-bounds contract of the kind the inferrer generates and OpenJML discharges, and which therefore serves as a fixed, green starting point for the M1 synthesis loop.

```
//@ requires str != null;
//@ requires 0 <= start && start <= str.length();
//@ ensures \result != null;
//@ ensures \result.length() == str.length() - start;
public static String substring(String str, int start) { ... }
```

Listing 8: A representative green target: a null-and-bounds contract of the class the inferrer already discharges, used as the fixed contract for the M1 synthesis loop.

Confining the proof-of-concept to these classes is an honest scoping decision, not an evasion. The known decidability and scalability limits of the verifier—Z3 timeouts on multiplicative reasoning, array-theory queries, and rich preconditions—are real and documented in this programme, and the tiered-assurance mechanism of the architecture exists precisely to degrade gracefully when they bite. Starting from green method classes lets the instantiation validate the loop, the contract-drafting pipeline, and the ratification dialogue on solid ground, while the frontier-adoption strategy of section 5—specify the module under change and the public API surface, treat the remainder as weakly defaulted—defines how the same machinery extends outward without pretending the whole library can be verified at once. What the instantiation establishes, accordingly, is feasibility and realism: that the reference architecture can be assembled from validated parts, exercised on a genuine three-stream corpus, and made to surface real code-versus-intent discrepancies, with full empirical validation on external production codebases and an industrial case study left, as section 14 sets out, to future work.

11 Evaluation Design

This article *proposes* a reference architecture and *instantiates* it as a proof of concept; it does not claim a completed empirical validation. Accordingly, this section presents an evaluation *plan* rather than results: for each research question stated in section 1 it specifies a study design, the corpus on which the study will run, the baselines against which the pipeline will be compared, the metrics that will be collected, and the analysis that will be applied to them. The intent is to make the architecture falsifiable—to state in advance what observations would count as evidence for, and against, each of the claims the pipeline makes—and to fix the operational definitions of the metrics so that later work, by these authors or others, can execute the plan without reinterpreting it. Where the proof-of-concept instantiation of section 10 already exercises part of a study, this is noted as evidence of feasibility; full effect-size estimation on external production codebases, together with an industrial case study, is deferred to future work for the reasons set out in section 11.9.

Two design principles govern the whole plan. First, every metric must be *tool-derived and objective* wherever possible: the pipeline records provenance, assurance level, counterexamples, iteration counts, and verification outcomes as a by-product of its normal operation, so the primary measurements are extracted from machine logs rather than from human judgement. Where human judgement is unavoidable—confirming that a candidate bug is real, or that a ratified example reflects true intent—the protocol fixes the adjudication procedure in advance and reports inter-rater agreement. Second, the trusted verifier is never permitted to grade its own inputs: the OpenJML extended static checker (ESC) discharged by the Z3 SMT solver, together with the custom fork adding `define-fun-rec` for `\sum`, `\product`, and `\num_of`, is a fixed instrument reused unchanged from the authors' prior work, and all measurements that depend on it are reported relative to that fixed oracle.

11.1 Corpus

The primary corpus is Apache Commons Lang, chosen because it is the rare real-world artefact that exposes all three intent streams the brownfield workflow depends upon. Its source supplies the *code* stream; the public Apache JIRA project (issues of the form `LANG-####`) supplies the *ticket* stream, including reported defects and discussed edge cases; and its rich Javadoc supplies the *documentation* stream. The library is already integrated into the authors' experimental harness from prior work [33, 34], so the inference instrument, the converter, and the Dockerized verification pipeline run on it without modification, and the corpus

is large enough to exercise propagation and characterisation at scale while remaining small enough to permit per-clause human adjudication on a curated subset. Studies that require per-method ground truth are scoped to the `StringUtils` and `NumberUtils` classes, where code, tickets, and Javadoc are densest and where the method classes the inferrer already handles well—null-safety, array bounds, simple arithmetic postconditions, and the loop patterns (MAX/MIN, linear search, sum) for which it emits invariants—are best represented. Two supplementary corpora support external-validity checks: a held-out set of public GitHub Java projects not seen during heuristic development, used to test whether characterisation and propagation generalise beyond the tuning corpus, and the SWE-bench issue set [53] as a point of comparison for the agentic synthesis baselines defined below. The held-out corpora are used only for feasibility and generalisation observations in this article; controlled measurement on them is future work.

11.2 RQ1: Ratification effort and the ratification gap

The first study asks whether the *ratify-behaviour-not-logic* mechanism (section 6) lets a human own the contract at acceptable cost, given that the human chose NL precisely because they cannot or will not read formal text. The design is a within-subjects task study in which participants elaborate a fixed set of method specifications under two conditions: a *formal-text* baseline, in which the participant confirms the JML contract directly, and the *example-pinned* condition, in which the agent renders the contract as concrete checkable examples ("given `x=-1` this contract says `result=0`; correct?") and the participant ratifies behaviour rather than logic. The corpus is the curated `StringUtils`/`NumberUtils` subset, ordered to balance difficulty across conditions.

The primary metric is *ratification effort*, defined as obligations confirmed per minute of participant time, measured from interaction logs; the secondary metric is the *fraction auto-defaulted*, the proportion of obligations resolved by inherited default contracts (section 6) without participant adjudication, since the architecture's claim is that defaults absorb the bulk of routine intent and leave only departures for the human. A faithfulness metric guards against the failure mode in which examples are easy but wrong: each ratified example becomes a *pinned oracle*, and the study records how many pinned oracles are later contradicted by a participant on re-presentation, a direct measure of whether behavioural ratification produces stable intent. Analysis pairs the two conditions per participant and reports the effect on effort with a non-parametric paired test and confidence intervals, alongside the auto-default fraction as a descriptive distribution. The hypothesis is that the example-pinned condition raises obligations-per-minute and that inherited defaults account for a large share of obligations, narrowing the ratification gap without inflating the contradiction rate.

11.3 RQ2: Decision surfacing and adversarial soundness

The second study evaluates the two elaboration-layer guards: Socratic decision surfacing and adversarial elaboration (section 6). It uses a seeded-decision design. For a set of methods drawn from the corpus, the authors construct ground-truth catalogues of boundary decisions—null and empty handling, integer overflow, ordering, duplicates, concurrency, and the error-versus-exception choice—that a correct elaboration must resolve. The *gap/boundary detection rate* is then the fraction of seeded decisions the agent surfaces to the human *before* code is synthesized, versus those that escape surfacing and would otherwise be discovered only at or after verification; escaped decisions are counted by replaying synthesis and observing where an unsurfaced choice manifests as a counterexample or a divergence from the pinned oracles.

Adversarial elaboration is evaluated on its ability to catch *spec weakening*. The study seeds contracts with known weakenings—most importantly the canonical case in which "sort" is degraded to "is a permutation", which the identity function trivially satisfies—and measures the *spec-weakening catch rate* of the second, independent agent that searches for scenarios consistent with the NL intent but violating the proposed contract. Two further detectors are measured against seeded faults: the *vacuity rate*, the fraction

of contracts flagged because a postcondition is trivially true or a precondition is unsatisfiable, repurposing the authors' prior "`ensures true`" punt detection as a weakening signal; and the resulting precision and recall over the seeded weakening and vacuity faults. Baselines are a single-agent elaboration with no adversarial pass and a syntactic vacuity check alone. The analysis reports detection rate, catch rate, and precision/recall with binomial confidence intervals, and the hypothesis is that the adversarial pass and vacuity detector together raise weakening recall substantially over the single-agent baseline at acceptable precision.

11.4 RQ3: Provenance, assurance, and independent authority

The third study asks whether the contract that code is verified against is honestly and auditably constituted—that is, whether the independent-authority constraint and the provenance/assurance tagging make the "verified" claim defensible (section 5). The unit of observation is the obligation at merge time. For every merged contract the pipeline emits each obligation's provenance tag (human-ratified, inherited-default, inferred-unconfirmed, example-pinned, ticket-stated, doc-stated, or code-inferred) and assurance tag (proved, bounded-checked, tested, or unchecked). The primary reported artefact is the *provenance distribution at merge time*, a contingency table over provenance $\times$ assurance, which makes visible what fraction of a merged contract rests on human-ratified, proved obligations versus inferred-unconfirmed, merely tested ones. A healthy distribution concentrates the load-bearing, oracle-defining obligations in the human-ratified and example-pinned rows; a distribution dominated by code-inferred, unconfirmed clauses signals a contract that is characterisation, not specification (section 11.6).

Two integrity checks accompany the distribution. An *authorship-independence audit* verifies, from the orchestration log, that no obligation in the merged contract carries an author identity matching the implementer role that produced the code it verifies; any violation is a circularity defect and is reported as a count. A *degradation-accounting* check verifies that every obligation that could not be proved is recorded at a strictly weaker assurance level (bounded-checked or tested) rather than silently dropped, by reconciling the set of attempted obligations against the set of merged obligations. The analysis is descriptive: the contingency table, the independence-violation count (target zero), and the no-silent-drop reconciliation. Table 10 summarises the metric for each research question.

11.5 RQ4: The synthesis–verification (CEGIS) loop

The fourth study evaluates the counterexample-guided synthesis loop (section 8). For each target method, the agent emits code, OpenJML verifies the code against the fixed contract, and on failure the concrete counterexample witness is returned to the synthesis agent, which regenerates, capped at N iterations; on persistent failure the obligation is degraded per the tiered-assurance policy and reported. The corpus is stratified by *method class*: null-safety, array bounds, simple arithmetic postconditions, and the inferrer-supported loop patterns, since the architecture's claim is class-dependent convergence rather than uniform success.

The primary metrics are the *success rate by method class*—the fraction of methods for which a contract-satisfying implementation is found within the iteration cap—and the *distribution of loop iterations to convergence*, reported per class so that easy classes (null-safety, bounds) are not averaged together with hard ones (multiplicative arithmetic, rich preconditions). To isolate the contribution of the counterexample channel, the principal baseline replaces the concrete witness with a test-failure signal ("an input failed") of the kind the agentic baselines of section 3.1 rely on [20, 88]; a second baseline is open-loop sampling with no verification gate, evaluated against a strong test suite [62] to expose the plausible-but-incorrect outputs the closed loop is designed to reject. Milestone M1 of the instantiation—the synthesis loop run against a fixed hand-written contract on an already-green method class—supplies an initial feasibility data point for this study and de-risks the loop before contract generation is introduced. The analysis compares success

rate and median iterations across conditions per class, with the hypothesis that the counterexample channel converges in fewer iterations and at higher success rate than the test-failure channel, and that the gap widens on harder classes where a concrete witness carries more diagnostic information than "some test failed".

11.6 RQ5: Brownfield characterisation and candidate-bug yield

The fifth study evaluates the startup/characterisation phase and the multi-source intent triangulation it performs (section 7). The phase ingests code, JIRA tickets, and Javadoc; infers a draft contract from the code; extracts intent claims from tickets and docs; and reconciles them into a candidate contract with per-clause provenance, flagging disagreements between streams. The study runs this phase over the `StringUtils`/`NumberUtils` subset and treats the disagreements as the output of interest, in keeping with the principle that this phase cannot be gated on zero errors: finding code-versus-intent violations is the success outcome, not a failure.

The primary metric is *characterisation discrepancy yield*: the number of candidate bugs surfaced as disagreements—either code violating its own inferred pattern (an over-strong inferred clause) or code disagreeing with ticket- or doc-derived intent—partitioned into *confirmed* and *false* by a fixed adjudication protocol in which two raters inspect each candidate against the originating ticket or Javadoc and the source, with disagreements resolved by discussion and inter-rater agreement reported. Because the corpus exposes historical `LANG-####` defects, a recall proxy is available: the fraction of historically documented edge-case defects in the subset that the discrepancy stream re-surfaces. The verification-as-characterisation caveat is measured explicitly rather than asserted: the study reports the fraction of merged clauses that are code-derived (and therefore nearly circular, proving faithful description and toolchain sanity but not correctness) separately from the fraction grounded in ticket/doc intent plus human ratification, making precise what the verification does and does not establish. A secondary measurement records *traceability noise*—the precision of the heuristic and retrieval-augmented links between tickets/docs and code [8, 14]—since the credibility of the triangulation rests on those links and they are known to be noisy. The expected outcome is a non-trivial confirmed candidate-bug yield on a mature, well-tested library, demonstrating that disagreement detection finds real defects rather than merely re-describing the code.

11.7 RQ6: Compositional change propagation and version compatibility

The sixth study evaluates the lifecycle capability that links this article to the thesis core: because contracts compose, a callee-spec change recomputes caller proof obligations and flags which callers break without running them, so that an internal refactor and a dependency or library version bump are the same operation (section 9). This promotes the authors' prior compositional refinement [33] from a one-off inference result to a continuous blast-radius analysis. The design uses real version pairs from Commons Lang and from the broader Maven ecosystem, where behavioural breaking changes between releases have been catalogued in prior empirical work [51, 74, 77]. For a callee whose contract changes between versions, the pipeline computes the set of call sites whose recomputed obligations no longer discharge—the predicted breaking callers—and this set is compared against ground truth derived from the catalogued behavioural incompatibilities and from differential execution of the affected clients.

The primary metrics are *propagation precision and recall* over flagged breaking callers: precision penalizes false alarms that would erode developer trust, and recall penalizes missed breakages that defeat the purpose of the analysis. Baselines are signature-level breaking-change detectors [15] and semantic-versioning declarations [82, 84], neither of which reasons about behavioural contracts, so the hypothesis is that contract-level propagation recovers behavioural breakages these miss while not flagging benign signature-compatible changes. A cost metric records the analysis time per call site, since the value of computing the blast radius "without running them" depends on it being cheaper than differential execution. The analysis reports precision, recall, and their harmonic mean against each baseline, broken down by whether the change is an

Table 10: Evaluation plan summary: primary metric and baseline per research question.

RQ	Mechanism evaluated	Primary metric	Principal baseline
RQ1	Behavioural ratification	Obligations confirmed/min; auto-default fraction	Formal-text confirmation
RQ2	Surfacing + adversarial pass	Boundary detection rate; weakening catch rate	Single-agent elaboration
RQ3	Provenance + independent authority	Provenance×assurance distribution	— (auditability check)
RQ4	CEGIS synthesis loop	Success rate & iterations by class	Test-failure feedback signal
RQ5	Brownfield characterisation	Confirmed candidate-bug yield	Code-only inference
RQ6	Compositional propagation	Breaking-caller precision/recall	Signature diff; semver

internal refactor or a cross-version library bump, to substantiate the claim that the two are handled by one mechanism.

11.8 Cross-cutting metric: verification cost and graceful degradation

Every study above depends on the trusted oracle, whose cost and decidability limits are real and known—Z3 times out on multiplicative reasoning, array theory, and rich preconditions, and not all intent is formally specifiable. The plan therefore collects, across all studies, the *verification cost* (wall-clock discharge time per obligation, under the settled strict configuration of code-math `safe` and spec-math `bigint`) and the *timeout rate* (the fraction of obligations on which the solver does not terminate within the budget), reported per method class so that the cost profile of the architecture is transparent. These two measurements feed the tiered-assurance accounting of section 11.4: a timeout is not a verification failure but a trigger to degrade full proof to bounded check to test, recording the assurance level. Reporting cost and timeout rate alongside the per-RQ metrics ensures that any feasibility claim is qualified by the honest statement that verification does not always terminate, and that the architecture's graceful-degradation response is itself measured rather than assumed.

11.9 Planned industrial case study (future work)

The studies above can be executed on public corpora by the authors alone, but the architecture's central claims concern a *lifecycle*—how teams elaborate, ratify, synthesize, and maintain contracts over time—which an offline corpus cannot fully exercise. The plan therefore includes an industrial case study, framed explicitly as future work, conducted as *action research* within a partner organisation that adopts the pipeline on a live codebase. The methodological commitment is that the case study measures *objective, tool-derived metrics*—the same ratification-effort, provenance-distribution, candidate-bug-yield, CEGIS-convergence, and propagation-precision quantities defined above, extracted from the running pipeline's logs—rather than self-report instruments such as developer-satisfaction surveys administered by a participant's manager, whose validity is compromised by the power relationship. Human subjects are involved only to the extent of consenting to log analysis and participating in the ratification dialogue, and the study will proceed under approval from a Human Research Ethics Committee (HREC) with informed consent and data-handling protocols fixed in advance. The retroactive-then-prospective loop reordering of section 7 structures the case study: the first phase resolves recovered discrepancies on existing code to establish a trusted baseline and to let the tool earn trust before it changes anything, and only subsequent phases apply the pipeline to new features. Consistent with the limitations enumerated throughout this article, the case study aims to establish *feasibility and realism in situ*—that the pipeline functions, that its metrics are obtainable, and that practitioners can operate it—rather than a controlled effect size, which would require a larger multi-site programme beyond the scope of this work.

12 Threats to Validity

The contribution of this article is a reference architecture, validated by instantiation rather than by a controlled empirical study, and the honest assessment of what that instantiation can and cannot establish is itself part of the contribution. This section organizes the threats along the conventional four axes—construct, internal, external, and conclusion validity—and, for each threat, names the design mechanism intended to contain it. The recurring theme is that several of the architecture's most distinctive features—provenance- and assurance-tagged obligations, the independent-authority constraint, behaviour-level ratification, and tiered assurance—exist precisely because the threats below are real and were anticipated, rather than discovered after the fact. Where a threat cannot be eliminated, the design aims to make it *visible and auditable* rather than to claim it away.

12.1 Construct Validity

Construct validity concerns whether the artefacts the pipeline produces actually measure what the lifecycle requires of them—namely, a faithful, machine-checkable encoding of intent. Three constructs are at risk.

The most fundamental is that *inferred specifications skew weak*. Heuristic AST-based inference, and dynamic invariant detection of the Daikon lineage [36] more generally, describe what the code *does*, not what it *should* do; a draft contract recovered from an implementation that silently truncates on overflow will faithfully record the truncation as intended behaviour. A weak contract is a poor oracle: code can satisfy it while still violating the human's real intent. The architecture does not pretend that inference closes this gap. Instead it treats the inferred contract as an explicitly *code-inferred*, *unconfirmed* provenance class that *drives* the human dialogue rather than gating it, and it strengthens weak drafts through three channels: inherited default contracts that impose organisation-level policy (for example, public methods reject `null` unless stated) the elaboration only needs to override on departure; adversarial elaboration, the elaboration-layer dual of counterexample-guided inductive synthesis (CEGIS) [91], in which a second agent searches for scenarios consistent with the natural-language intent but admitted by the proposed contract, flagging weakenings such as "sort" degraded to "is a permutation"; and vacuity and non-triviality checks that repurpose the author's own "ensures true" punt detection as a weakening signal. The residual threat—that no automated check substitutes for intent the human never articulated—is mitigated only by ratification, which is real labor and is itself bounded below.

A dual construct threat is that inferred specifications are sometimes *wrong or over-strong*, producing *false candidate bugs*. An over-fitted clause derived from a single code path may declare a behavioural pattern the method does not in fact guarantee, so that legitimate inputs appear to violate it. Because the brownfield startup phase reports disagreements between code and its own inferred pattern as *candidate* bugs, never as confirmed defects, and surfaces them as the opening agenda of the human dialogue rather than as a gate, an over-strong clause costs a conversation, not a false alarm in production. The provenance tagging makes the source of each clause auditable, so a reviewer can discount a *code-inferred* candidate appropriately.

The third construct threat is that the *natural-language-to-formal translation is imperfect*. Converting tickets and documentation into formal clauses without a human in the loop is an open research problem [24, 35, 65], and a mistranslation injects a defective oracle. The design constrains the blast radius of this threat structurally: any clause derived from a natural-language source without human confirmation retains *ticket-stated* or *doc-stated* provenance and *unconfirmed* assurance, so it never silently becomes a trusted oracle—it enters the ratification dialogue as a question, and only behaviour-level confirmation promotes it.

12.2 Internal Validity

Internal validity concerns whether the causal mechanism the article claims—that an independent, machine-checkable contract supplies the verification spine the surveyed agentic lifecycles lack—actually holds within the instantiation, rather than being an artefact of how the pieces are wired together.

The dominant internal threat is *circularity*, in two forms. The first is the self-grading hazard endemic to closed-loop LLM systems, in which the same model authors both the implementation and the oracle against which it is checked [64, 94]. If contract and code share an author, a model that misreads the intent produces an oracle that excuses its own error, and apparent verification is vacuous. The architecture answers this with the *independent-authority constraint*: the contract the code is verified against must not be authored by the same agent instance or role that writes the code. The elaborator role is maximally faithful and strict, never sees the implementation, and the implementer role is tasked only with satisfying a contract it did not write. Crucially, the verifier is OpenJML with Z3 [23, 27], a trusted symbolic oracle that never grades itself, so the soundness arbiter is structurally outside the LLM loop. Provenance tags make the constraint *auditable*: a verification claim resting on a clause whose provenance is the implementer's own role is detectable and rejectable. The residual threat is that role separation can be subverted by shared context or prompt leakage; the design treats this as a configuration invariant to be checked, not as a property assumed for free.

The second, subtler internal threat is *verifying against the wrong specification*—the case in which the contract is internally consistent, the code provably satisfies it, and yet the contract does not encode the human's intent. A wrong spec verified against is worse than no spec, because it launders an incorrect implementation behind a green proof. This is exactly why behaviour-level ratification and the brownfield triangulation are load-bearing rather than ornamental. The pipeline never asks the human to confirm formal text—which would make the human authority fake, since the human resorted to natural language precisely because they cannot or will not read contracts (the *ratification gap*)—but instead renders the contract as concrete, checkable examples ("given $x = -1$ this contract says the result is 0; is that correct?"), and ratified examples become *pinned oracles* the contract must continue to satisfy. In the brownfield setting, correctness relative to intent is sought not from the code-derived spec alone—verifying code against a code-derived spec is *characterisation*, a faithful-description baseline and toolchain sanity check that is nearly circular and proves nothing about absence of bugs—but from *disagreement* among the three evidence streams: code, tickets, and documentation. A defect surfaces as code violating its own inferred pattern or, more tellingly, code disagreeing with ticket- or doc-derived intent.

A further internal threat is the *noisiness of ticket- and doc-to-code traceability*. Linking a Jira ticket or a Javadoc paragraph to the method it concerns relies on commit links, path heuristics, and information-retrieval techniques that are known to be imperfect [8, 14, 61], and comment-code inconsistency is itself well documented [95, 99]. Mislinked intent corrupts the disagreement signal on which candidate-bug detection depends. The design contains this threat by retrieval-augmented heuristics whose output is, again, never a trusted oracle: a traced clause is *ticket-stated* and *unconfirmed* until ratified, so a spurious link produces a question for the human, not a silent corruption. The article additionally restricts strong claims to a curated module surface where traceability is credible, rather than asserting corpus-wide link accuracy.

12.3 External Validity

External validity concerns the generality of the demonstration beyond the conditions under which it was produced.

The principal external threat is the *single-corpus* basis of the proof-of-concept. The instantiation is exercised on Apache Commons Lang, chosen because it couples a real public issue tracker (the Apache JIRA `LANG-####` project) with rich Javadoc, yielding a genuine three-stream code–tickets–docs dataset of the kind the brownfield phase requires. A single, well-maintained, mature Java library is not representative of

the breadth of production software: its API discipline, documentation quality, and ticket hygiene are higher than typical, which flatters traceability and characterisation alike. The article therefore claims *feasibility and realism* of the reference architecture, not a controlled effect size, and explicitly defers validation on external production codebases and an industrial case study—action research under human-research-ethics approval—to future work.

A second external threat is the *decidability and scalability ceiling of verification*, which bounds the class of programs to which the pipeline's strongest assurance applies. Extended static checking discharged by an SMT solver does not always terminate: Z3 times out on multiplicative reasoning, array-theory queries, and rich preconditions, as the author's own prior experiments document, and a custom OpenJML fork was required merely to model recursive `\sum`, `\product`, and `\num_of`. The instantiation consequently targets first the method classes that already verify green—null-safety, array-bounds, simple arithmetic post-conditions, and the loop patterns (max/min, linear search, sum) for which the inferrer already produces invariants—and adopts *frontier adoption* for legacy scale, specifying only the module under change, the public API surface, and modules referenced by active tickets while treating the remainder as unspecified under weak inherited defaults. The mitigation for non-termination is *tiered assurance with graceful degradation*: full proof degrades to bounded check, then to test, with the achieved assurance level *recorded* on each obligation and never silently dropped, so a timeout downgrades a claim rather than invalidating the pipeline.

12.4 Conclusion Validity

Conclusion validity concerns whether the inferences drawn from the demonstration are warranted, and is governed here chiefly by *LLM nondeterminism*. The elicitation, contract-drafting, and synthesis steps invoke a stochastic model, so a single run is not a stable measurement: a contract drafted, an ambiguity surfaced, or an implementation synthesized on one invocation may differ on the next, and reported behaviour could reflect a favorable sample. The architecture localizes the consequences of this nondeterminism by routing every consequential output through the deterministic verifier and the deterministic provenance ledger: a nondeterministic implementation is admitted only if it satisfies the contract under OpenJML, and a nondeterministically drafted clause remains *unconfirmed* until ratified into a pinned oracle, so variability degrades to retry-until-verified rather than to silent acceptance of an unsound result. The synthesis–verification loop is capped at N iterations, after which the failing obligation is reported under tiered assurance rather than masked. What this does *not* neutralize is the threat to any quantitative claim: the article accordingly frames its results as an existence demonstration of the loop on a curated corpus, and any future effect-size claim will require repeated runs with reported variance, multiple corpora, and the ethics-approved case study noted above. Taken together, the mitigations in this section convert threats that would otherwise be silent failure modes into visible, provenance-tagged, assurance-graded artefacts—an outcome the architecture treats as the realistic target, since the alternative of eliminating the threats outright is not available.

13 Discussion

The reference pipeline described in this article is not, in the first instance, a verification technique; it is a proposal to reorganise the software development *process* around a machine-checkable specification. The technical mechanisms—inference, elaboration, the synthesis–verification loop, compositional propagation—are in service of a process change, and it is that change, rather than any single algorithm, that constitutes the article's central contribution to the software development process. This section therefore steps back from the architecture and asks what the lifecycle looks like once a formal contract layer sits at the pivot between human intent and machine output: what the unit of work becomes, where trust is anchored, how dependency

evolution is handled, how the approach is adopted incrementally, and—honestly—where it will and will not repay the effort.

13.1 The process shift: from reviewing code to reviewing specifications

The most consequential implication is a relocation of the human review activity. In the prevailing process, the durable record of intent is the code itself, and quality control is exercised by reading code in review. When an agent can regenerate an implementation cheaply, the implementation becomes a disposable build output and the scarce, durable artefact is the contract it was built and verified against. Review then migrates up the stack: the reviewer inspects the *specification delta*, not the diff of generated statements. This is a qualitatively easier object to reason about, because a contract states *what* must hold over all inputs rather than *how* one particular realisation computes it, and because the implementation has already been proved to satisfy whatever the contract says by OpenJML's extended static checking [23], covering all inputs rather than the sampled inputs a test suite exercises.

Correspondingly, the unit of change becomes a *spec-delta plus a discharged proof obligation* rather than a code patch. A pull request in this model carries the changed obligations, their provenance and assurance tags, and the verifier's verdict that the new implementation satisfies them. The reviewer adjudicates the contract change—is this the behaviour we want?—and delegates the question of whether the code conforms to the verifier. This is the sense in which trust becomes *transferable*. In conventional review, trust is established by a qualified human eyeballing the implementation, an act that does not compose and does not transfer: a reviewer's approval is an opinion, not a checkable artefact. When conformance is established by a discharged proof obligation, the trust is attached to the artefact and can be re-checked by anyone, including a machine. It is precisely this property that would let an agent merge its own work without a human in the conformance loop: not because the agent is trusted, but because its output carries an independently re-verifiable certificate that it satisfies a human-ratified contract. The human authority is exercised once, over the contract; it is not re-spent on every implementation.

Two caveats keep this from becoming a slogan. First, transferable trust is only as strong as the contract: a discharged proof against a weak or wrong contract certifies nothing useful, which is why the independent-authority constraint and behaviour-level ratification described in the elaboration layer are load-bearing rather than decorative. Second, the verdict is honest only if it carries its assurance level. A merge gate must distinguish an obligation that was `proved` from one that was merely `bounded-checked` or `tested` after the proof timed out; the tiered-assurance tagging exists so that "verified" on a pull request is an auditable claim rather than a reassuring word.

13.2 Dependency evolution as a spec-diff event

The second process change follows from compositionality. Because contracts compose, a change to a callee's contract mechanically recomputes the proof obligations of its callers, and the verifier reports which callers no longer discharge—without executing any of them. Internal refactoring and external dependency upgrades are then the same operation viewed at different scopes: in both cases a callee specification moves and the blast radius is the set of call sites whose obligations break. This recasts the perennial and poorly served question "which of my call sites does this library upgrade break behaviourally?" as a first-class, answerable lifecycle event. The existing literature on breaking changes is overwhelmingly signature-level—tools such as APIDiff and the Maven-Central replication studies detect that a method's *shape* changed [15, 77]—while the harder and more common problem is the *behavioural* breaking change, where the signature is stable but the contract is not, and which the same line of work identifies as the dominant unsolved case [51, 74]. Semantic versioning encodes a promise about compatibility but cannot check it [82, 84]. A contract-diff over composed specifications turns that promise into a verification condition: an

upgrade is behaviourally compatible for a given client exactly when the client's obligations still discharge against the new callee contract. This is the lifecycle expression of the thesis link, with the compositional refinement validated in our prior work [33] promoted from an offline corpus measurement to an online change-impact capability, and it is the point at which the formal-methods-adoption and library-compatibility strands of the programme meet in a single mechanism.

13.3 An adoption path that earns trust before it demands it

A reference architecture is worthless if it can only be entered greenfield, and the overwhelming majority of software is brownfield. The adoption path is therefore designed to be incremental along two axes. The first is *scope*: verification does not scale to an entire legacy application, so the contract layer is introduced at a frontier—the module under active change, the public API surface, and the modules referenced by live tickets—while the remainder is treated as unspecified and governed only by weak inherited default contracts. The boundary contracts at this frontier are exactly where compositional propagation attaches, so the specified region can grow outward along call edges rather than requiring a big-bang formalization. The second axis is *sequence*. The first pass over a brownfield codebase is *retroactive*: the startup phase recovers a draft contract from the code, triangulates it against tickets and documentation, and surfaces the disagreements as candidate bugs and stale docs to be resolved with a human; only after this establishes a trusted baseline do *prospective* passes introduce new behaviour. The tool thus earns its keep on existing code—finding real discrepancies in a system the team already owns—before it is permitted to change anything. Inherited default contracts carry much of the adoption burden: organisation-level policies (public methods reject null unless stated, monetary values are never negative, returned collections are never null) are specified once and inherited everywhere, so that the per-method ratification labor is spent only on departures from default. The human role, across all of this, moves up the stack: away from writing and reviewing implementations and toward adjudicating contracts, ratifying behaviour through concrete examples, and arbitrating the disagreements the triangulation surfaces.

13.4 What is genuinely novel

It is important to be precise about what distinguishes this proposal from the substantial bodies of work it builds on, because individually most of its ingredients are not new. Counterexample-guided synthesis is decades old [3, 91]; closed-loop verified code generation with a language model and a deductive verifier has recent and capable instances [71, 75, 94]; specification inference, both dynamic and learned, is a mature field [35, 36, 65]; and spec-driven agentic lifecycles are emerging in industry [4, 5, 44]. The novelty is not in any one of these but in three commitments that the emerging agentic lifecycles in particular do not make. The first is insistence on a *machine-checkable* contract at the pivot, in place of the natural-language specification those lifecycles treat as ground truth; this is what closes the otherwise open generate-then-eyeball loop with a machine gate that ranges over all inputs. The second is the *independent-authority constraint*: the contract a code is verified against must not be authored by the agent that wrote the code, dissolving the circularity that afflicts self-grading approaches where the same model produces an implementation and the tests that judge it—a hazard the test-oracle literature has begun to flag explicitly for LLM-generated oracles [13, 64]. The third is the elaboration layer's treatment of the *ratification gap*: rather than asking a human to confirm formal text they used natural language precisely to avoid, the contract is ratified at the level of behaviour through pinned, human-confirmed examples, and its strength is policed by an adversarial elaborator that hunts for scenarios consistent with the stated intent yet permitted by an over-weak contract. The combination—a verification spine, an authority separation, and a behaviour-level human gate—is, to our knowledge, not assembled in prior agentic or spec-driven work, and it is that assembly, instantiated on validated components, that constitutes the contribution.

13.5 Where the approach will and will not pay off

Candor about applicability is owed to the reader, both because the claim of this article is feasibility rather than measured effect and because the failure modes are predictable. The approach pays off where behaviour is precisely statable and mechanically checkable: data-structure and library code, numeric and bounds contracts, null- and emptiness-discipline, and the loop patterns—maximum, minimum, linear search, summation—for which the inference instrument already produces discharging invariants. These are the method classes that verify green today and the natural first targets for instantiation. It pays off least where intent resists formalization (subjective or aesthetic requirements, distributed and timing-dependent behaviour, performance properties outside a functional contract) and where the underlying decision procedures do not terminate; the Z3-bound timeouts on multiplicative reasoning, array theory, and rich preconditions observed across our verification corpus are real, known, and not papered over—they are precisely the cases the tiered-assurance mechanism exists to degrade gracefully rather than to deny.

Several limitations bear directly on the process claims above and must be stated plainly. Inferred contracts skew weak, describing what code does rather than what it should, so ratification is genuine human labor; the mitigation is inherited defaults plus API-surface prioritisation, not an assertion that the labor disappears. Inferred contracts are sometimes wrong or over-strong, which manifests as false candidate bugs; this is why every recovered clause is offered as a *candidate* and why human ratification, not the inference, owns the truth. Translating ticket and documentation intent into formal clauses without a human is imperfect, so those clauses remain unconfirmed and drive the dialogue rather than gating it, and linking tickets and docs to code is noisy enough that it is credible only over a curated module rather than a whole repository. Above all, a wrong contract verified against is worse than no contract, because it launders an error into a proof; this single observation is why the independent authority and behavioural ratification are not optional refinements but the load bearing structure of the whole proposal. Finally, what verification establishes over a brownfield codebase must not be overstated: proving code against a code-derived contract is characterisation—a faithful-description baseline and a toolchain sanity check—and is nearly circular; it does not prove the absence of bugs. Correctness relative to intent emerges only from the ticket and documentation streams and the human ratification on top of them, and bugs appear as *disagreements* between what the code does and what those streams say it should. The full empirical adjudication of these claims on external production codebases, including an industrial action-research case study under appropriate ethics review, is future work; this article claims that the architecture is realistic and instantiable on validated components, not that its benefits have yet been measured at scale.

14 Conclusion and Future Work

This article began from an observation about where the economics of software construction are heading. As agentic, "spec-driven" lifecycles such as AWS AI-DLC [4], GitHub Spec Kit [44], and Kiro [5] mature, the implementation increasingly becomes a disposable build output that an agent can regenerate on demand, and the durable, scarce artefact becomes the specification of intent. Yet the specifications that drive these emerging lifecycles share three structural defects. The specification is natural language (NL), so the specification-to-code "compilation" performed by the agent is unverifiable; the loop is open, in that code is generated and then inspected by eye with no machine gate; and where verification is present at all it is circular, because the same model that writes the code also writes the tests or oracle against which the code is judged [62, 64]. The lifecycle is being reorganised around the specification as the source of truth, but the specification it is being organised around cannot bear that weight.

14.1 Summary of the contribution

The contribution of this article is a reference architecture that supplies the missing verification spine: a machine-checkable formal-specification contract layer together with a verification loop. The pivot of the architecture is the formal contract, which mediates between the ambiguous human world above it and the precise machine world below it. Above the pivot lies *elaboration*, the human-confirmed translation of NL intent into a formal contract; below it lie *synthesis*, in which an agent produces code from the contract, and *verification*, in which OpenJML extended static checking [23] discharged by the Z3 SMT solver [27] certifies the code against the contract over all inputs rather than over a sampled test suite. The organising idea is the dual role of the specification: the same contract is simultaneously the input the agent builds against and the oracle its output is verified against, and heuristic inference removes the spec-authoring bootstrap cost that historically made this dual role impractical.

Around this pivot the article defines a set of mechanisms intended to make the architecture honest rather than merely plausible. The *ratification gap*—that the human used NL precisely because they will not author formal text, yet the contract must be human-owned to be a trustworthy oracle—is closed by ratifying behaviour rather than logic: the contract is rendered as concrete, checkable examples that become pinned oracles, and a Socratic elaboration process surfaces only the boundary decisions (null, empty, overflow, ordering, duplicates, error-versus-exception) the agent had to resolve. An *adversarial elaboration* agent, the elaboration-layer dual of CEGIS [91], searches for scenarios consistent with the NL intent but violating the proposed contract, detecting spec weakening and vacuity. An *independent-authority constraint* forbids the agent that writes the code from authoring the contract it is verified against, dissolving the circularity that the reliability literature documents [62, 64, 108]; *provenance- and assurance-tagged obligations* make that constraint auditable and the "verified" claim precise; and *tiered assurance* degrades a full proof to a bounded check to a test, recording the assurance level rather than silently dropping intent when verification does not terminate. For brownfield codebases—most real software—the contract is recovered rather than authored: a characterisation phase triangulates code, issue tickets, and documentation into a provenance-tagged candidate contract whose disagreements become candidate bugs and the opening agenda of the human dialogue. Crucially, this article reframes the contribution not as a measured improvement but as a development-process capability: it restores a closed, counterexample-driven loop, with an independently owned machine-checkable contract as the source of truth, to a lifecycle that had abandoned it.

14.2 The instantiation

The architecture is not merely described but instantiated. The proof-of-concept reuses the author's existing validated assets as trusted components—the JML-Inferrer for draft contracts, the custom OpenJML fork with Z3 as the oracle that never grades itself, the `AnnotationToJMLConverter`, and the Dockerized verification pipeline—so that the only new code is an orchestration layer and LLM calls for elicitation, contract drafting, and synthesis. The instantiation is exercised on Apache Commons Lang, a genuine three-stream dataset that couples source with the public Apache JIRA `LANG-####` project and rich Javadoc, allowing real code-versus-intent discrepancies to be surfaced. This grounds the architecture in the assets and results of the surrounding PhD programme: inferred specifications improve LLM unit-test generation [31]; specifications can be embedded into compiled artefacts and OpenAPI documents at full roundtrip fidelity [32]; heuristic inference admits compositional refinement across method boundaries [33]; and the inference pipeline integrates into CI/CD with diff-only analysis and fail-open semantics [34].

14.3 Limitations

The article is candid about what it does not claim. Inferred specifications skew weak, describing what code does rather than what it should, so ratification remains real labor that the architecture mitigates—but does

not eliminate—through inherited default contracts and API-surface prioritisation. Inferred specifications are sometimes over-strong and wrong, generating false candidate bugs that the human-in-the-loop must adjudicate, which is why every recovered obligation is framed as a candidate rather than a verdict. The translation of tickets and documentation into formal clauses without human confirmation is imperfect, and linking those artefacts to code is noisy, so the brownfield results are credible only on a curated module. Verification scalability and decidability limits are real and known: Z3 times out on multiplicative reasoning, array theory, and rich preconditions, which is precisely the contingency tiered assurance exists to handle. Above all, a wrong specification verified against is worse than no specification, which is why the independent-authority constraint and human ratification are load-bearing rather than ornamental. Verifying code against a code-derived specification is characterisation, not a proof of correctness; correctness relative to intent arises only from the intent streams and human ratification.

14.4 Future work

Several lines of work follow directly. The first is to complete the proof-of-concept build beyond the de-risked early milestones: milestone M2 generates contracts from NL by few-shot prompting over the author's own verified JML corpus, with the inferrer supplying brownfield drafts, and milestone M3 implements conversational ratification through the Socratic and example-pinning loop. The second is empirical validation on external production codebases beyond the demonstration corpus, to test whether multi-source triangulation and the synthesis–verification loop generalise past a curated module. The third is an industrial action-research case study, conducted with appropriate human research ethics committee (HREC) approval, in which practitioners use the pipeline on their own codebase so that the ratification labor, the candidate-bug yield, and the trust dynamics of retroactive-then-prospective adoption can be observed in situ rather than assumed. The fourth is integration with the CI/CD machinery of the programme's prior work [34], so that compositional change propagation becomes a first-class lifecycle event: because contracts compose, a callee-spec change recomputes caller proof obligations and flags which call sites break without running them, making internal refactors and dependency or library version bumps the same operation and turning "which of my call sites does this upgrade break behaviourally?" into a routine, machine-answerable query [33, 51, 84].

These threads converge on a single thesis. In a lifecycle where an agent can regenerate the implementation cheaply, the specification is no longer a precursor to the work but the work itself—and it earns that status only by playing both roles at once. As the precise input the agent builds against and the formal oracle its output is verified against, the dual-role specification is what makes an AI-native development process verifiable rather than merely productive, and it is the artefact around which such a process must be organised.

Data Accessibility

A replication package containing the proof-of-concept orchestrator, the specification inference engine, the OpenJML/Z3 verification harness, the JSON schemas for the inference and characterisation reports, and the Apache Commons Lang corpus configuration will be made available at `https://github.com/[anonymized]/jml-inferrer`.

Acknowledgments

This research was supported by the University of Queensland Cyber initiative.

Conflict of Interest

The authors declare no conflict of interest.

Author Contributions

Brendan Edmonds: Conceptualization, Methodology, Software, Validation, Investigation, Writing - Original Draft. **Mark Utting**: Conceptualization, Methodology, Writing - Review & Editing, Supervision.

References

[1] Wolfgang Ahrendt, Bernhard Beckert, Richard Bubel, Reiner Hähnle, Peter H. Schmitt, and Mattias Ulbrich. *Deductive Software Verification – The KeY Book: From Theory to Practice*, volume 10001 of *Lecture Notes in Computer Science*. Springer, 2016. doi: 10.1007/978-3-319-49812-6.

[2] Google DeepMind AlphaProof and AlphaGeometry Teams. AI achieves silver-medal standard solving international mathematical olympiad problems. Google DeepMind technical blog, 2024. URL https://deepmind.google/discover/blog/ai-solves-imo-problems-at-silver-medal-level/.

[3] Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. Syntax-guided synthesis. In *Formal Methods in Computer-Aided Design (FMCAD)*, pages 1–8, 2013. doi: 10.1109/FMCAD.2013.6679385.

[4] Amazon Web Services. AI-DLC: Ai-driven development lifecycle. https://github.com/awslabs/aidlc-workflows, 2025. Accessed: 2026-06-27.

[5] Amazon Web Services. Kiro: An AI IDE for spec-driven development. https://kiro.dev, 2025. Accessed: 2026-06-27.

[6] Glenn Ammons, Rastislav Bodík, and James R. Larus. Mining specifications. In *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 4–16, 2002. doi: 10.1145/503272.503275.

[7] Anonymous. A benchmark for vericoding: Formally verified program synthesis. In *Dafny Workshop, Symposium on Principles of Programming Languages (POPL)*, 2026. URL https://popl26.sigplan.org/details/dafny-2026-papers/13/.

[8] Giuliano Antoniol, Gerardo Canfora, Gerardo Casazza, Andrea De Lucia, and Ettore Merlo. Recovering traceability links between code and documentation. *IEEE Transactions on Software Engineering*, 28(10):970–983, 2002. doi: 10.1109/TSE.2002.1041053.

[9] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021. URL https://arxiv.org/abs/2108.07732.

[10] Ralph-Johan Back and Joakim von Wright. *Refinement Calculus: A Systematic Introduction*. Springer, New York, NY, 1998. doi: 10.1007/978-1-4612-1674-2.

[11] Haniel Barbosa, Clark Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, Alex Ozdemir, Mathias Preiner, Andrew Reynolds, Ying Sheng, Cesare Tinelli, and Yoni Zohar. cvc5: A versatile and industrial-strength SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems*

(TACAS), volume 13243 of *Lecture Notes in Computer Science*, pages 415–442. Springer, 2022. doi: 10.1007/978-3-030-99524-9_24.

[12] Mike Barnett, K. Rustan M. Leino, and Wolfram Schulte. The Spec# programming system: An overview. In *Construction and Analysis of Safe, Secure, and Interoperable Smart Devices (CASSIS 2004)*, volume 3362 of *Lecture Notes in Computer Science*, pages 49–69. Springer, 2005. doi: 10.1007/978-3-540-30569-9_3.

[13] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525, 2015. doi: 10.1109/TSE.2014.2372785.

[14] Markus Borg, Per Runeson, and Anders Ardö. Recovering from a decade: A systematic mapping of information retrieval approaches to software traceability. *Empirical Software Engineering*, 19(6): 1565–1616, 2014. doi: 10.1007/s10664-013-9255-y.

[15] Aline Brito, Laerte Xavier, Andre Hora, and Marco Tulio Valente. APIDiff: Detecting API breaking changes. In *Proceedings of the 25th IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 507–511, 2018. doi: 10.1109/SANER.2018.8330249.

[16] Aline Brito, Laerte Xavier, Andre Hora, and Marco Tulio Valente. Why and how Java developers break APIs. *Empirical Software Engineering*, 25(2):1115–1158, 2020. doi: 10.1007/s10664-019-09756-z.

[17] Saikat Chakraborty, Shuvendu K. Lahiri, Sarah Fakhoury, Madanlal Musuvathi, Akash Lal, Aseem Rastogi, Aditya Senthilnathan, Rahul Sharma, and Nikhil Swamy. Ranking LLM-generated loop invariants for program verification. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 9164–9175, 2023. doi: 10.18653/v1/2023.findings-emnlp.614.

[18] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. URL https://arxiv.org/abs/2107.03374.

[19] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys*, 51 (1):4:1–4:27, 2018. doi: 10.1145/3143561.

[20] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2304.05128.

[21] Elliot J. Chikofsky and James H. Cross. Reverse engineering and design recovery: A taxonomy. *IEEE Software*, 7(1):13–17, 1990. doi: 10.1109/52.43044.

[22] David R. Cok. OpenJML: JML for Java 7 by extending OpenJDK. In *NASA Formal Methods (NFM 2011)*, volume 6617 of *Lecture Notes in Computer Science*, pages 472–479. Springer, 2011. doi: 10.1007/978-3-642-20398-5_35.

[23] David R. Cok. OpenJML: Software verification for Java 7 using JML, OpenJDK, and Eclipse. *Electronic Proceedings in Theoretical Computer Science*, 149:79–92, 2014. doi: 10.4204/EPTCS.149.8.

[24] Matthias Cosler, Christopher Hahn, Daniel Mendoza, Frederik Schmitt, and Caroline Trippel. nl2spec: Interactively translating unstructured natural language to temporal logics with large language models. In *Computer Aided Verification (CAV 2023)*, volume 13965 of *Lecture Notes in Computer Science*, pages 383–396. Springer, 2023. doi: 10.1007/978-3-031-37703-7_18.

[25] Patrick Cousot and Radhia Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 238–252, 1977. doi: 10.1145/512950.512973.

[26] Andrea De Lucia, Fausto Fasano, Rocco Oliveto, and Genoveffa Tortora. Recovering traceability links in software artifact management systems using information retrieval methods. *ACM Transactions on Software Engineering and Methodology*, 16(4):13:1–13:50, 2007. doi: 10.1145/1276933.1276934.

[27] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, volume 4963 of *Lecture Notes in Computer Science*, pages 337–340. Springer, 2008. doi: 10.1007/978-3-540-78800-3_24.

[28] Alexandre Decan, Tom Mens, and Philippe Grosjean. An empirical comparison of dependency network evolution in seven software packaging ecosystems. *Empirical Software Engineering*, 24(1):381–416, 2019. doi: 10.1007/s10664-017-9589-y.

[29] Edsger W. Dijkstra. Guarded commands, nondeterminacy and formal derivation of programs. *Communications of the ACM*, 18(8):453–457, 1975. doi: 10.1145/360933.360975.

[30] Edsger W. Dijkstra. *A Discipline of Programming*. Prentice-Hall, Englewood Cliffs, NJ, 1976.

[31] Brendan Edmonds and Mark Utting. Do JML specifications help LLMs write better unit tests? *Journal of Software: Evolution and Process*, 2026. In preparation.

[32] Brendan Edmonds and Mark Utting. Embedding JML specifications in compiled java artefacts and OpenAPI documents. *Journal of Software: Evolution and Process*, 2026. In preparation.

[33] Brendan Edmonds and Mark Utting. Does compositional refinement strictly extend heuristic specification inference? a measurement study. In *Proceedings of the International Conference on Software Engineering (ICSE)*, 2026. Under review.

[34] Brendan Edmonds and Mark Utting. Specification inference in continuous integration: Design and a reference implementation. *Journal of Software: Evolution and Process*, 2026. In preparation.

[35] Madeline Endres, Sarah Fakhoury, Saikat Chakraborty, and Shuvendu K. Lahiri. Can large language models transform natural language intent into formal method postconditions? *Proceedings of the ACM on Software Engineering (FSE)*, 1(FSE):1889–1912, 2024. doi: 10.1145/3660791.

[36] Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69(1–3):35–45, 2007. doi: 10.1016/j.scico.2007.01.015.

[37] Michael C. Feathers. *Working Effectively with Legacy Code*. Prentice Hall, 2004. ISBN 9780131177055.

[38] Emily First, Markus N. Rabe, Talia Ringer, and Yuriy Brun. Baldur: Whole-proof generation and repair with large language models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 1229–1241. ACM, 2023. doi: 10.1145/3611643.3616243.

[39] Cormac Flanagan and K. Rustan M. Leino. Houdini, an annotation assistant for ESC/Java. In *FME 2001: Formal Methods for Increasing Software Productivity*, volume 2021 of *Lecture Notes in Computer Science*, pages 500–517. Springer, 2001. doi: 10.1007/3-540-45251-6_29.

[40] Cormac Flanagan, K. Rustan M. Leino, Mark Lillibridge, Greg Nelson, James B. Saxe, and Raymie Stata. Extended static checking for Java. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 234–245, 2002. doi: 10.1145/512529.512558.

[41] Darius Foo, Hendy Chua, Jason Yeo, Ming Yi Ang, and Asankhaya Sharma. Efficient static checking of library updates. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 791–796, 2018. doi: 10.1145/3236024.3275535.

[42] Gordon Fraser and Andrea Arcuri. EvoSuite: Automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*, pages 416–419, 2011. doi: 10.1145/2025113.2025179.

[43] Pranav Garg, Christof Löding, P. Madhusudan, and Daniel Neider. ICE: A robust framework for learning invariants. In *Computer Aided Verification (CAV)*, volume 8559 of *Lecture Notes in Computer Science*, pages 69–87. Springer, 2014. doi: 10.1007/978-3-319-08867-9_5.

[44] GitHub. Spec kit: Toolkit to help you get started with spec-driven development. https://github.com/github/spec-kit, 2025. Accessed: 2026-06-27.

[45] Orlena Gotel, Jane Cleland-Huang, Jane Huffman Hayes, Andrea Zisman, Alexander Egyed, Paul Grünbacher, Alex Dekhtyar, Giuliano Antoniol, Jonathan Maletic, and Patrick Mäder. The grand challenge of traceability (v1.0). In Jane Cleland-Huang, Orlena Gotel, and Andrea Zisman, editors, *Software and Systems Traceability*, pages 343–409. Springer, 2012. doi: 10.1007/978-1-4471-2239-5_16.

[46] Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. *ACM SIGPLAN Notices (POPL)*, 46(1):317–330, 2011. doi: 10.1145/1925844.1926423.

[47] Jin Guo, Jinghui Cheng, and Jane Cleland-Huang. Semantically enhanced software traceability using deep learning techniques. In *Proceedings of the 39th International Conference on Software Engineering (ICSE)*, pages 3–14, 2017. doi: 10.1109/ICSE.2017.9.

[48] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12 (10):576–580, 1969. doi: 10.1145/363235.363259.

[49] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2308.00352.

[50] Frank Reyes García Islam, Gabriel Bartolomei, Martin Monperrus, and Benoit Baudry. Byam: Fixing breaking dependency updates with large language models. *arXiv preprint arXiv:2505.07522*, 2025. URL https://arxiv.org/abs/2505.07522.

[51] Dhanushka Jayasuriya, Sherlock A. Ranasinghe, Ewan Tempero, Kelly Blincoe, and Jens Dietrich. Understanding the impact of APIs behavioral breaking changes on client applications. *Proceedings of the ACM on Software Engineering*, 1(FSE):1238–1261, 2024. doi: 10.1145/3643782.

[52] Kamil Jezek and Jens Dietrich. API evolution and compatibility: A data corpus and tool evaluation. *Journal of Object Technology*, 16(4):2:1–2:23, 2017. doi: 10.5381/jot.2017.16.4.a2.

[53] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

[54] Adharsh Kamath, Aditya Senthilnathan, Saikat Chakraborty, Pantazis Deligiannis, Shuvendu K. Lahiri, Akash Lal, Aseem Rastogi, Subhajit Roy, and Rahul Sharma. Finding inductive loop invariants using large language models. arXiv preprint arXiv:2311.07948, 2023.

[55] Florent Kirchner, Nikolai Kosmatov, Virgile Prevosto, Julien Signoles, and Boris Yakobowski. Frama-C: A software analysis perspective. *Formal Aspects of Computing*, 27(3):573–609, 2015. doi: 10.1007/s00165-014-0326-7.

[56] Shuvendu K. Lahiri et al. Beyond postconditions: Can large language models infer formal contracts for automatic software verification? arXiv preprint arXiv:2510.12702, 2025.

[57] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. Verus: Verifying Rust programs using linear ghost types. *Proceedings of the ACM on Programming Languages (OOPSLA)*, 7:286–315, 2023. doi: 10.1145/3586037.

[58] Gary T. Leavens, Albert L. Baker, and Clyde Ruby. Preliminary design of JML: A behavioral interface specification language for Java. *ACM SIGSOFT Software Engineering Notes*, 31(3):1–38, 2006. doi: 10.1145/1127878.1127884.

[59] K. Rustan M. Leino. Dafny: An automatic program verifier for functional correctness. In *Logic for Programming, Artificial Intelligence, and Reasoning (LPAR-16)*, volume 6355 of *Lecture Notes in Computer Science*, pages 348–370. Springer, 2010. doi: 10.1007/978-3-642-17511-4_20.

[60] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022. doi: 10.1126/science.abq1158.

[61] Jinfeng Lin, Yalin Liu, Qingkai Zeng, Meng Jiang, and Jane Cleland-Huang. Traceability transformed: Generating more accurate links with pre-trained BERT models. In *Proceedings of the 43rd International Conference on Software Engineering (ICSE)*, pages 324–335, 2021. doi: 10.1109/ICSE43902.2021.00040.

[62] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. URL https://arxiv.org/abs/2305.01210.

[63] Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, and Yiling Lou. Large language model-based agents for software engineering: A survey. *arXiv preprint arXiv:2409.02977*, 2024. URL https://arxiv.org/abs/2409.02977.

[64] Shuo Liu, Ying Zhang, and Lin Tan. Understanding LLM-driven test oracle generation. *arXiv preprint arXiv:2601.05542*, 2026. doi: 10.48550/arXiv.2601.05542.

[65] Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. SpecGen: Automated generation of formal program specifications via large language models. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025. doi: 10.1109/ICSE55347.2025.00129.

[66] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. URL https://arxiv.org/abs/2303.17651.

[67] Zohar Manna and Richard Waldinger. A deductive approach to program synthesis. *ACM Transactions on Programming Languages and Systems*, 2(1):90–121, 1980. doi: 10.1145/357084.357090.

[68] Thorsten Merten, Bastian Mager, Paul Hübner, Thomas Quirchmayr, Barbara Paech, and Simone Bürsner. Requirements communication in issue tracking systems in four open-source projects. In *Proceedings of the 6th International Workshop on Requirements Prioritization and Communication (RePriCo), co-located with REFSQ*, volume 1342 of *CEUR Workshop Proceedings*, pages 114–125, 2015. URL https://ceur-ws.org/Vol-1342/02-reprico.pdf.

[69] Bertrand Meyer. Applying "design by contract". *Computer*, 25(10):40–51, 1992. doi: 10.1109/2.161279.

[70] Bertrand Meyer. *Object-Oriented Software Construction*. Prentice Hall, Upper Saddle River, NJ, 2 edition, 1997.

[71] Md Rakib Hossain Misu, Cristina V. Lopes, Iris Ma, and James Noble. Towards ai-assisted synthesis of verified dafny methods. *Proceedings of the ACM on Software Engineering (FSE)*, 1(FSE):812–835, 2024. doi: 10.1145/3643763.

[72] Lloyd Montgomery, Clara Lüders, and Walid Maalej. An alternative issue tracking dataset of public Jira repositories. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*, pages 73–77, 2022. doi: 10.1145/3524842.3528486.

[73] Carroll Morgan. *Programming from Specifications*. Prentice Hall, Hemel Hempstead, UK, 2 edition, 1994.

[74] Shaikh Mostafa, Rodney Rodriguez, and Xiaoyin Wang. Experience paper: A study on behavioral backward incompatibilities of Java software libraries. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 215–225, 2017. doi: 10.1145/3092703.3092721.

[75] Emmanuel Mugnier et al. DafnyPro: LLM-assisted automated verification for Dafny programs. In *Dafny Workshop, Symposium on Principles of Programming Languages (POPL)*, 2026. URL https://popl26.sigplan.org/details/dafny-2026-papers/12/.

[76] Jeremy W. Nimmer and Michael D. Ernst. Static verification of dynamically detected program invariants: Integrating Daikon and ESC/Java. In *Proceedings of the First Workshop on Runtime Verification (RV)*, volume 55 of *Electronic Notes in Theoretical Computer Science*, pages 255–276, 2001. doi: 10.1016/S1571-0661(04)00256-7.

[77] Lina Ochoa, Thomas Degueule, Jean-Rémy Falleri, and Jurgen Vinju. Breaking bad? Semantic versioning and impact of breaking changes in Maven Central: An external and differentiated replication study. *Empirical Software Engineering*, 27(3):61, 2022. doi: 10.1007/s10664-021-10052-y.

[78] Ipek Ozkaya. Application of large language models to software engineering tasks: Opportunities, risks, and implications. *IEEE Software*, 40(3):4–8, 2023. doi: 10.1109/MS.2023.3248401.

[79] Carlos Pacheco, Shuvendu K. Lahiri, Michael D. Ernst, and Thomas Ball. Feedback-directed random test generation. In *Proceedings of the 29th International Conference on Software Engineering (ICSE)*, pages 75–84, 2007. doi: 10.1109/ICSE.2007.37.

[80] Kexin Pei, David Bieber, Kensen Shi, Charles Sutton, and Pengcheng Yin. Can large language models reason about program invariants? In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pages 27496–27520. PMLR, 2023. URL https://proceedings.mlr.press/v202/pei23a.html.

[81] Kexin Pei, David Bieber, Kensen Shi, Charles Sutton, and Pengcheng Yin. Can large language models reason about program invariants? In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pages 27496–27520, 2023. URL https://proceedings.mlr.press/v202/pei23a.html.

[82] Tom Preston-Werner. Semantic versioning 2.0.0. https://semver.org/, 2013. Accessed 2026-06-27.

[83] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. URL https://arxiv.org/abs/2307.07924.

[84] Steven Raemaekers, Arie van Deursen, and Joost Visser. Semantic versioning and impact of breaking changes in the Maven repository. *Journal of Systems and Software*, 129:140–158, 2017. doi: 10.1016/j.jss.2016.04.008.

[85] Tal Ridnik, Dedy Kredo, and Itamar Friedman. Code generation with AlphaCodium: From prompt engineering to flow engineering. *arXiv preprint arXiv:2401.08500*, 2024. URL https://arxiv.org/abs/2401.08500.

[86] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering*, 50 (1):85–105, 2024. doi: 10.1109/TSE.2023.3334955.

[87] Sergio Segura, Gordon Fraser, Ana B. Sánchez, and Antonio Ruiz-Cortés. A survey on metamorphic testing. *IEEE Transactions on Software Engineering*, 42(9):805–824, 2016. doi: 10.1109/TSE.2016.2532875.

[88] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. URL https://arxiv.org/abs/2303.11366.

[89] Xujie Si, Hanjun Dai, Mukund Raghothaman, Mayur Naik, and Le Song. Learning loop invariants for program verification. In *Advances in Neural Information Processing Systems 31 (NeurIPS)*, pages 7762–7773, 2018. URL https://proceedings.neurips.cc/paper/2018/hash/65b1e92c585fd4c2159d5f33b5030ff2-Abstract.html.

[90] Mohammed Latif Siddiq, Joanna C. S. Santos, Ridwanul Hasan Tanvir, Noshin Ulfat, Fahmid Al Rifat, and Vinicius Carvalho Lopes. Using large language models to generate JUnit tests: An empirical study. *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering (EASE)*, pages 313–322, 2024. doi: 10.1145/3661167.3661216.

[91] Armando Solar-Lezama. *Program Synthesis by Sketching*. PhD thesis, University of California, Berkeley, 2008. URL https://people.csail.mit.edu/asolar/papers/thesis.pdf.

[92] Armando Solar-Lezama. *Program Synthesis by Sketching*. PhD thesis, University of California, Berkeley, 2008. Technical Report UCB/EECS-2008-177.

[93] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodík, Sanjit Seshia, and Vijay Saraswat. Combinatorial sketching for finite programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 404–415, 2006. doi: 10.1145/1168857.1168907.

[94] Chuyue Sun, Ying Sheng, Oded Padon, and Clark Barrett. Clover: Closed-loop verifiable code generation. In *AI Verification (SAIV)*, volume 14846 of *Lecture Notes in Computer Science*, pages 134–155. Springer, 2024. doi: 10.1007/978-3-031-65112-0_7.

[95] Shin Hwei Tan, Darko Marinov, Lin Tan, and Gary T. Leavens. @tComment: Testing Javadoc comments to detect comment-code inconsistencies. In *Proceedings of the 5th IEEE International Conference on Software Testing, Verification and Validation (ICST)*, pages 260–269, 2012. doi: 10.1109/ICST.2012.106.

[96] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. Unit test case generation with transformers and focal context. *arXiv preprint arXiv:2009.05617*, 2020. doi: 10.48550/arXiv.2009.05617.

[97] Cody Watson, Michele Tufano, Kevin Moran, Gabriele Bavota, and Denys Poshyvanyk. On learning meaningful assert statements for unit test cases. In *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*, pages 1398–1409, 2020. doi: 10.1145/3377811.3380429.

[98] Cheng Wen, Jialun Cao, Jie Su, Zhiwu Xu, Shengchao Qin, Mengda He, Haokun Li, Shing-Chi Cheung, and Cong Tian. Enchanting program specification synthesis by large language models using static analysis and program verification. In *Computer Aided Verification (CAV)*, volume 14682 of *Lecture Notes in Computer Science*, pages 302–328. Springer, 2024. doi: 10.1007/978-3-031-65630-9_16.

[99] Fengcai Wen, Csaba Nagy, Gabriele Bavota, and Michele Lanza. A large-scale empirical study on code-comment inconsistencies. In *Proceedings of the 27th IEEE/ACM International Conference on Program Comprehension (ICPC)*, pages 53–64, 2019. doi: 10.1109/ICPC.2019.00019.

[100] Haoze Wu, Clark Barrett, and Nina Narodytska. Lemur: Integrating large language models in automated program verification. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL https://openreview.net/forum?id=Q3YaCghZNt.

[101] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *Proceedings of the 1st Conference on Language Modeling (COLM)*, 2024. URL https://arxiv.org/abs/2308.08155.

[102] Laerte Xavier, Aline Brito, Andre Hora, and Marco Tulio Valente. Historical and impact analysis of API breaking changes: A large-scale study. In *Proceedings of the 24th IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 138–147, 2017. doi: 10.1109/SANER.2017.7884616.

[103] Chenyuan Yang et al. ExVerus: Verus proof repair via counterexample reasoning. arXiv preprint arXiv:2603.25810, 2026. URL https://arxiv.org/abs/2603.25810.

[104] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024. URL https://arxiv.org/abs/2405.15793.

[105] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2306.15626.

[106] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. Evaluating and improving ChatGPT for unit test generation. *Proceedings of the ACM on Software Engineering (FSE)*, 1(FSE):1703–1726, 2024. doi: 10.1145/3660783.

[107] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 1592–1604. Association for Computing Machinery, 2024. doi: 10.1145/3650212.3680384.

[108] Ziyao Zhang, Chong Wang, Yanlin Wang, Ensheng Chen, and Zibin Zheng. LLM hallucinations in practical code generation: Phenomena, mechanism, and mitigation. *Proceedings of the ACM on Software Engineering (ISSTA)*, 2(ISSTA), 2025. doi: 10.1145/3728894.